

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

DANIELA ARALDI

**ANÁLISE DE REGRAS *SIGMA* GERADAS POR AGENTE AUTÔNOMO COM
RAG**

PATO BRANCO

2026

DANIELA ARALDI

**ANÁLISE DE REGRAS *SIGMA* GERADAS POR AGENTE AUTÔNOMO COM
RAG**

Analyzing Sigma Rules Generated by a RAG-based Autonomous Agent

Projeto de Trabalho de Conclusão de Curso de Graduação apresentado como requisito para obtenção do título de Bacharel em Engenharia de Computação do Curso de Bacharelado em Engenharia de Computação da Universidade Tecnológica Federal do Paraná.

Orientador: Prof. Me. Adriano Serckumecka

Coorientador: Prof. Dr. Jefferson Tales Oliva

PATO BRANCO

2026



[4.0 Internacional](https://creativecommons.org/licenses/by/4.0/)

Esta licença permite compartilhamento, remixe, adaptação e criação a partir do trabalho, mesmo para fins comerciais, desde que sejam atribuídos créditos ao(s) autor(es). Conteúdos elaborados por terceiros, citados e referenciados nesta obra não são cobertos pela licença.

Para meu pai (*em memória*).

AGRADECIMENTOS

Agradeço a todas as pessoas que conheci graças à graduação e cujos atravessamentos foram positivos, mas principalmente às amigadas que permaneceram.

Agradeço aos professores que me inspiraram.

Agradeço aos professores que me orientaram, por se porem disponíveis a me orientar, e a todos os professores que foram compreensivos às sensibilidades humanas que um aluno carrega.

Agradeço aos meus pais, por me proporcionarem o privilégio de frequentar a universidade e sempre me apoiarem.

Agradeço ao meu companheiro por acreditar em mim e me incentivar a finalizar o curso (me mandando ir pra terapia).

RESUMO

A crescente sofisticação e o volume das ameaças cibernéticas pressionam as equipes de segurança a reduzir o intervalo entre a divulgação de uma vulnerabilidade e a disponibilização de detecção correspondente. Neste contexto estão inseridas as regras Sigma, um padrão aberto e independente para a descrição de lógicas de detecção em sistemas de gerenciamento de informações e eventos de segurança (SIEMs), mas sua escrita manual é lenta e propensa a lacunas diante de ameaças emergentes. Este trabalho desenvolve e avalia um agente autônomo capaz de gerar regras Sigma de forma automatizada a partir de fontes diversificadas de inteligência de ameaças, como registros de vulnerabilidades, notícias de falhas recém-divulgadas e históricos de logs, com o propósito de acelerar a operacionalização de detecções e apoiar o analista de segurança, a quem cabe a validação final. O agente é implementado como uma máquina de estados orientada a grafo, articulando consulta a fontes externas de enriquecimento, geração aumentada por recuperação (RAG) sobre um repositório de regras, geração da regra por um modelo de linguagem de grande escala (LLM), validação sintática iterativa com laço automático de correção e avaliação semântica complementar por um segundo modelo no papel de juiz. A metodologia compara quatro estratégias de obtenção de regras em complexidade crescente: as regras originais escritas por especialistas, tomadas como referência; a geração direta por modelos comerciais sob instrução mínima; a geração pelos mesmos modelos sob engenharia de *prompt*; e a geração pelo agente proposto. A avaliação emprega métricas determinísticas de conformidade sintática e estrutural, métricas de similaridade semântica em relação à referência e um conjunto de testes estatísticos pareados para verificar se as diferenças observadas entre as estratégias são significativas.

Palavras-chave: regras sigma; engenharia de detecção; llm; rag; automação em cibersegurança.

ABSTRACT

Keywords: sigma rules; detection engineering; llm; rag; cybersecurity automation.

LIST OF ALGORITHMS

Algoritmo 1 – Coleta de contexto técnico (nó 2)	61
---	----

LISTA DE FIGURAS

Figura 1 – Exemplo de regra <i>Sigma</i> (SIGMAHQ, 2025c)	26
Figura 2 – Exemplo de regra <i>Sigma</i> (Roth, 2022)	27
Figura 3 – Arquitetura do agente	58
Figura 4 – Distribuição do número de tentativas até a aprovação no nó de validação determinística.	76
Figura 5 – Distribuição do.	76
Figura 6 – Distribuição dos vereditos do juiz semântico estratificada por variação de <i>prompt</i>	78
Figura 7 – Distribuição absoluta dos vereditos do juiz semântico	79
Figura 8 – Distribuição da similaridade de cosseno entre os YAMLS completos das regras geradas e das regras-gabarito, por grupo.	82
Figura 9 – Distribuição da similaridade de cosseno entre os blocos <i>detection</i> das regras geradas e das regras-gabarito, por grupo.	83

LISTA DE TABELAS

Tabela 1 – Campos do <code>GraphState</code> agrupados por etapa do <i>pipeline</i>	59
Tabela 2 – Composição final dos quatro grupos comparativos	73
Tabela 3 – Regras geradas pelo <code>sigma_agent_automatico.py</code>	73
Tabela 4 – Pareceres gerados pelo juiz do nó 6	74
Tabela 5 – Diversidade da seleção de regras <i>Sigma</i> por <i>round-robin</i>	74
Tabela 6 – Taxa de aprovação sintática do agente, global e por variação de <i>prompt</i>	75
Tabela 7 – Distribuição dos erros de validação por categoria	77
Tabela 8 – Resultados da validação por tipo de <i>input</i> e qualidade do contexto	77
Tabela 9 – Distribuição dos vereditos emitidos pelo nó 6 (juiz semântico)	78
Tabela 10 – Resultado dos vereditos nas seis dimensões de avaliação do juiz semântico	79
Tabela 11 – Cruzamento entre o status do nó 5 (validação determinística) e o veredito do nó 6 (juiz semântico)	80
Tabela 12 – Volume de comentários textuais do juiz por dimensão, segmentado por status	81
Tabela 13 – Resultados estatísticos de similaridade de cosseno global e de <i>detection</i> por grupo	82
Tabela 14 – Distribuição dos resultados em níveis (Alta, Média e Baixa) por grupo	83
Tabela 15 – Estatísticas descritivas da similaridade do bloco <i>detection</i> , considerando apenas cenários com YAML válido na regra gerada	84
Tabela 16 – Correspondência estrutural entre regras geradas e regras-gabarito, calculada sobre o total de 50 regras por grupo.	84
Tabela 17 – Correspondência estrutural restrita às regras com YAML válido por grupo.	85
Tabela 18 – Taxa de regras com YAML válido por grupo.	85
Tabela 19 – Lista completa das regras filtradas e os modelos selecionados por <i>prompt</i>	103

LISTA DE QUADROS

Quadro 1 – Modelos de linguagem utilizados no trabalho.	47
Quadro 2 – <i>Frameworks</i> , banco vetorial e ambiente de desenvolvimento.	48
Quadro 3 – Bibliotecas de validação, coleta de dados e análise estatística.	49
Quadro 4 – <i>Hardware</i> utilizado no desenvolvimento do trabalho.	49
Quadro 5 – Composição das bases de dados utilizadas no trabalho.	50

LISTA DE ABREVIATURAS E SIGLAS

Siglas

API	<i>Application Programming Interface</i> , em português Interface de Programação de Aplicações
BGL	<i>BlueGene/L</i>
CAPEC	Common Attack Pattern Enumerations and Classifications
CISA	Agência de Segurança de Infraestrutura e Cibernética, do inglês <i>Cybersecurity and Infrastructure Security Agency</i>
CNN	Rede Neural Convolucional, do inglês <i>Convolutional Neural Network</i>
CoT	<i>Chain-of-Thought</i>
CoT-R	<i>Chain-of-Thought + Reflection</i>
CVE	Vulnerabilidades e Exposições Comuns, do inglês <i>Common Vulnerabilities and Exposures</i>
CVSS	Sistema de Pontuação de Vulnerabilidades Comuns, do inglês <i>Common Vulnerability Scoring System</i>
CWE	Common Weakness Enumeration, do inglês <i>Common Weakness Enumeration</i>
DVWA	Damn Vulnerable Web Application, do inglês <i>Damn Vulnerable Web Application</i>
EPP	<i>Endpoint Protection Platforms</i>
GDPR	Regulamento Geral de Proteção de Dados, do inglês <i>General Data Protection Regulation</i>
GPU	<i>Graphics Processing Unit</i>
IA	Inteligência Artificial, do inglês <i>Artificial Intelligence</i>
IDS	<i>Intrusion Detection System</i>
IOC	Indicador de Comprometimento, do inglês <i>Indicator of Compromise</i>
IPS	<i>Intrusion Prevention System</i>
IR	<i>Intermediate Representation</i>

KQL	<i>Kusto Query Language</i>
LLM	Modelo de linguagem de grande escala, do inglês, <i>Large Language Model</i>
ML	Aprendizado de Máquina, do inglês <i>Machine Learning</i>
NIST	Instituto Nacional de Padrões e Tecnologia, do inglês <i>National Institute of Standards and Technology</i>
NVD	Banco Nacional de Vulnerabilidades, do inglês <i>National Vulnerability Database</i>
OSCTI	<i>Open-Source Cyber Threat Intelligence</i>
PCI-DSS	Padrão de Segurança de Dados da Indústria de Cartões de Pagamento, do inglês <i>Payment Card Industry Data Security Standard</i>
PLN	Processamento de Linguagem Natural, do inglês <i>Natural Language Processing</i>
QA	<i>Question Answering</i>
RAG	<i>Retrieval-Augmented Generation</i> , em português, Geração Aumentada por Recuperação
RNN	<i>Recurrent Neural Networks</i> , Redes Neurais Recorrentes.
SIEM	Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês <i>Security Information and Event Management</i>
SOC	Centro de Operações de Segurança, do inglês <i>Security Operations Center</i>
SPL	<i>Splunk Processing Language</i>
TF-IDF	Frequência de Termo–Frequência Inversa de Documento, do inglês <i>Term Frequency–Inverse Document Frequency</i>
TTP	<i>Tactics, Techniques, and Procedures</i> — em português, Táticas, Técnicas e Procedimentos
URL	<i>Uniform Resource Locators</i>
UUID	<i>Universally Unique Identifier</i>

SUMÁRIO

1	INTRODUÇÃO	14
1.1	Motivação	14
1.2	Objetivos	16
1.2.1	Objetivo geral	16
1.2.2	Objetivos específicos	16
1.3	Estrutura do trabalho	17
2	REFERENCIAL TEÓRICO	18
2.1	Segurança da Informação e SOC	18
2.1.1	SIEM	18
2.1.2	SIEMs Inteligentes	20
2.2	Bases de Conhecimento sobre Vulnerabilidades e Técnicas Adversárias	22
2.2.1	CVE e CWE	22
2.2.2	MITRE ATT&CK	23
2.2.3	NVD e CVSS	23
2.3	Regras <i>Sigma</i>	24
2.3.1	Sigma e a mitigação de ameaças	25
2.3.2	Estrutura	25
2.4	Processamento de Linguagem Natural	27
2.4.1	Modelos de Linguagem de Grande Escala (LLMs)	28
2.4.2	Arquitetura <i>Transformer</i>	29
2.4.3	Engenharia de <i>Prompt</i>	30
2.5	Geração Aumentada por Recuperação (RAG)	31
2.5.1	<i>Embeddings</i> e Bancos de Dados Vetoriais	31
2.5.2	RAG aplicado à Cibersegurança	32
2.6	Agentes Autônomos	33
2.6.1	Padrão <i>ReAct</i>	34
2.6.2	<i>LangGraph</i> como orquestrador	35
2.7	Métricas de similaridade	35
2.7.1	Índice de <i>Jaccard</i>	36
2.7.2	Similaridade do Cosseno	36

2.7.3	Aplicação	37
3	TRABALHOS RELACIONADOS	38
3.1	<i>LLMCloudHunter</i>	38
3.2	<i>RulePilot</i>	39
3.3	GRIDAI	41
3.4	Aprendizado de máquina aplicado a SIEMs e análise de <i>logs</i>	42
3.5	Posicionamento do Trabalho Proposto	43
4	MATERIAIS E MÉTODOS	46
4.1	Materiais	46
4.1.1	Base de Dados	46
4.2	Métodos	50
4.2.1	Seleção dos dados	50
4.2.2	Estratégias de geração das regras	52
4.2.2.1	Regras originais do <i>SigmaHQ</i> (Estágio 1)	52
4.2.2.2	Geração zero-shot com UUIDs comerciais (Estágio 2)	53
4.2.2.3	Geração com <i>prompt engineering</i> (Estágio 3)	53
4.2.2.4	Filtragem	54
4.2.2.5	Geração via agente com RAG (Estágio 4)	55
4.2.3	Desenvolvimento do agente	57
4.2.3.1	Visão geral da arquitetura	57
4.2.3.2	Máquina de estados (<i>LangGraph</i>), nó 1	59
4.2.3.3	Ferramentas externas (nó 2)	60
4.2.3.4	RAG (nó 3)	63
4.2.3.5	Geração da regra (nó 4)	64
4.2.3.6	Validação sintática iterativa e laço de correção (nó 5)	66
4.2.3.7	Avaliação semântica por LLL-as-a-judge (nó 6)	68
4.2.4	Métricas de Avaliação	69
4.2.4.1	Metadados	69
4.2.4.2	Métricas determinísticas (nó 5)	70
4.2.4.3	Métricas semânticas (nó 6)	71
4.2.4.4	Métricas de similaridade com o gabarito	71
5	RESULTADOS	73

5.0.1	Produtos obtidos	73
5.0.2	Desempenho determinístico do agente	75
5.0.3	Qualidade semântica das regras do agente	77
5.0.4	Similaridade com o gabarito	81
5.0.4.1	<u>Similaridade semântica</u>	81
5.0.4.2	<u>Correspondência estrutural</u>	84
5.1	Discussões	86
5.1.1	Validação sintática como condição necessária mas não suficiente	86
5.1.2	A barreira sintática como gargalo principal	86
5.1.3	Variações de <i>prompt</i>	87
5.1.4	Viés do procedimento de filtragem	87
5.2	Limitações e Trabalhos futuros	88
5.2.1	Limitações	88
5.2.2	Trabalhos futuros	89
6	CONCLUSÃO	90
	REFERÊNCIAS	91
	A EXEMPLOS DE <i>PROMPTS</i> USADOS	97
	A.1 Exemplos de <i>prompts</i> para file_event_win_malware_darkgate_autoit3_save_temp.yml	97
	A.2 Exemplos de <i>prompts</i> para web_sql_injection_in_access_logs.yml	99
	B MELHORES REGRAS GERADAS PELAS COMERCIAIS SELECIONADAS NO FILTRO POR -AS-A-JUDGE	103
	GLOSSÁRIO	96

1 INTRODUÇÃO

1.1 Motivação

A segurança cibernética contemporânea sustenta a estabilidade de serviços essenciais — como saúde, energia, finanças e administração pública, enquanto enfrenta uma pressão crescente decorrente tanto do aumento no volume de ameaças quanto da complexidade das infraestruturas tecnológicas modernas. Os ataques cibernéticos alimentam um ciclo contínuo de exploração de vulnerabilidades, comprometendo desde a integridade de dados sensíveis até a continuidade operacional de organizações inteiras. Esse ciclo demanda abordagens de defesa cada vez mais ágeis e adaptativas, capazes de acompanhar a sofisticação dos ataques e a expansão de infraestruturas tecnológicas heterogêneas (Shah *et al.*, 2022).

Por infraestrutura heterogênea, entende-se um ambiente composto por diferentes tecnologias, dispositivos, sistemas operacionais, plataformas e arquiteturas que coexistem e interagem entre si. Tal heterogeneidade amplia a complexidade de gestão e de proteção, pois cada componente possui vulnerabilidades específicas e demanda estratégias distintas de defesa. Nesse contexto, um único ponto fraco pode comprometer todo o ecossistema, o que reforça a necessidade de uma defesa cibernética inovadora e adaptativa (Shah *et al.*, 2022; PALO ALTO NETWORKS, 2025; EXABEAM, 2025).

A dimensão desse desafio pode ser quantificada pelo ritmo de divulgação de vulnerabilidades. Só em 2025 foram publicados aproximadamente 48 mil registros de Vulnerabilidades e Exposições Comuns, do inglês *Common Vulnerabilities and Exposures* (CVE)s — uma média de cerca de 133 novas vulnerabilidades por dia e um crescimento de aproximadamente 21% em relação a 2024 (Gamblin, 2026). O volume tornou-se de tal forma insustentável que, em abril de 2026, o Instituto Nacional de Padrões e Tecnologia, do inglês *National Institute of Standards and Technology* (NIST) anunciou que seu Banco Nacional de Vulnerabilidades, do inglês *National Vulnerability Database* (NVD) deixaria de enriquecer todos os CVEs recebidos, passando a priorizar apenas uma fração deles, em razão de um aumento de 263% nas submissões entre 2020 e 2025 (National Institute of Standards and Technology, 2026). Este cenário torna evidente que o esforço manual de catalogação e análise de ameaças já não acompanha a velocidade com que elas surgem.

A urgência dessa adaptação se tornou ainda mais evidente diante da diminuição progressiva do intervalo entre a divulgação de uma vulnerabilidade e sua exploração. Os s — ataques que exploram falhas ainda desconhecidas do desenvolvedor, sempre constituíram um caso particular em que a exploração precede a correção; o que se observa nos últimos anos, contudo, é a inversão da média desse intervalo. Relatórios de inteligência de ameaças documentam essa trajetória de forma quantitativa: o tempo médio para exploração (do inglês *time-to-exploit*) caiu de aproximadamente 63 dias em 2018 para cerca de 32 dias em 2023, cruzou o zero em 2024 e atingiu, em 2025, uma estimativa de -7 dias (Mandiant, 2026). Em outras palavras, a

exploração antecipando a divulgação deixou de ser exceção, característica dos s, e passou a representar o comportamento médio do ecossistema de ameaças. A consequência estratégica é direta, pois quando a exploração precede o *patch*, a aplicação de correções deixa de ser o controle decisivo, e a capacidade de detectar a atividade maliciosa em tempo hábil passa a ser o fator determinante para conter o ataque (Rapid7 Labs, 2026). A velocidade de detecção se tornou a linha de frente da defesa.

É precisamente neste contexto que se insere a engenharia de detecção, do inglês *detection engineering* — área da cibersegurança dedicada ao projeto, à validação e à manutenção sistemática das lógicas de detecção que alimentam os mecanismos de defesa. Os Sistema de Gerenciamento de Informações e Eventos de Segurança, do inglês *Security Information and Event Management* (SIEM)s, consolidaram-se como pilares dessa atividade, correlacionando grandes volumes de registros automáticos de eventos (*logs*) gerados continuamente por sistemas, aplicações, redes e dispositivos. Assim como os *firewalls*, os *Intrusion Detection System* (IDS)s e os *Intrusion Prevention System* (IPS), os SIEMs operam fundamentalmente com base em regras de detecção. Tais regras, contudo, nem sempre acompanham a evolução das técnicas maliciosas, exigindo atualização constante (Shah *et al.*, 2022; Podzins; Romanovs, 2019).

O gargalo da engenharia de detecção, entretanto, não está apenas na defasagem das regras, mas no próprio esforço humano de produzi-las e mantê-las. Pesquisas recentes do setor apontam que a fadiga de alertas, do inglês *alert fatigue*, figura entre as principais preocupações dos Centro de Operações de Segurança, do inglês *Security Operations Center* (SOC)s, e que os falsos positivos constituem o desafio mais citado na detecção de ameaças, sobrecarregando equipes já afetadas pela escassez de profissionais qualificados (SANS Institute, 2025). Cada regra mal calibrada gera ruído; cada ruído consome o tempo de analistas que precisariam estar investigando ameaças reais. Reduzir o custo humano de produzir regras de detecção precisas e atualizadas é um problema substancial da defesa cibernética moderna.

Dentre as iniciativas que buscam padronizar e acelerar essa produção, destacam-se as regras *Sigma*: um formato aberto e descritivo, escrito em YAML (linguagem de serialização de dados legível e estruturada), voltado à definição de lógicas de detecção independentes de plataforma. Por serem convertíveis para linguagens nativas de diferentes SIEMs, como *Splunk Processing Language* (SPL) ou *Elasticsearch Query*, as regras *Sigma* favorecem a colaboração entre equipes e a portabilidade das detecções (SIGMAHQ, 2025a). No entanto, sua criação e manutenção manuais permanecem lentas e propensas a lacunas, sobretudo diante de ameaças emergentes como um *ransomware* (tipo de *malware* que sequestra dados ou sistemas e exige resgate) ou um *zero-day exploit*, ataque que explora uma falha de segurança ainda desconhecida do desenvolvedor e para a qual não há correção disponível (SIGMAHQ, 2025a; EXABEAM, 2025).

Diante deste conjunto de pressões — o volume insustentável de vulnerabilidades, a compressão do tempo de exploração e o custo humano da engenharia de detecção manual, a aplicação de técnicas de Inteligência Artificial, do inglês *Artificial Intelligence* (IA) na geração au-

tomatizada de regras apresenta-se como uma direção promissora, capaz de reduzir o intervalo entre a divulgação de uma ameaça e sua cobertura defensiva, preservando a qualidade técnica das regras produzidas. O estudo de Lempinen, Pyyny e Juntunen (2023) apud ??) demonstrou que a integração entre LLM, e o SIEM *Wazuh*¹ resultou em eficácia na análise de *logs*, na detecção de padrões de ameaças e na automação de respostas de segurança, beneficiando sobretudo usuários com conhecimento limitado na área. Tal evidência sugere que LLMs podem ser adaptados para apoiar a geração de regras *Sigma* a partir de dados de ameaças.

Sendo assim, este trabalho propõe a implementação de um agente autônomo capaz de automatizar a criação de regras *Sigma* a partir de referências externas de inteligência de ameaças — tipicamente CVEs, Indicador de Comprometimento, do inglês *Indicator of Compromise* (IOC)s, *logs* de incidentes e notícias de vulnerabilidades recém-divulgadas, acessadas durante o processo de geração. A proposta é concebida como ferramenta de apoio à engenharia de detecção, na qual o analista permanece como instância final de validação no fluxo operacional, ainda que a presente investigação se concentre na avaliação da geração automática propriamente dita. O agente combina recuperação aumentada por geração (*Retrieval-Augmented Generation*, em português, Geração Aumentada por Recuperação (RAG)), sobre um repositório consolidado de regras *Sigma*, consulta a fontes externas via busca na *web* e validação iterativa (sintática e semântica) das regras produzidas por um Modelo de Linguagem de Grande Escala. Ao final, os resultados obtidos por diferentes estratégias de geração, com complexidade arquitetural crescente, são comparados de modo a avaliar o ganho efetivo proporcionado pela arquitetura agêntica proposta.

1.2 Objetivos

1.2.1 Objetivo geral

Desenvolver um agente autônomo, de estrutura local, capaz de gerar regras *Sigma* de forma automatizada a partir de fontes diversificadas de inteligência de ameaças — CVEs, (IOCs), *logs* de incidentes e notícias de vulnerabilidades recém-divulgadas, com o propósito de acelerar a operacionalização de detecções em SIEMs e, consequentemente, reduzir o intervalo entre a divulgação de uma ameaça e a sua cobertura defensiva.

1.2.2 Objetivos específicos

- Replicar regras *Sigma* a partir de LLMs;
- Gerar regras *Sigma* a partir do agente autônomo construído;

¹ O *Wazuh* é uma solução de segurança de código aberto com funcionalidades de *Host-based Intrusion Detection System* (HIDS) e SIEM (Wazuh, Inc., 2024).

- Desenvolver um agente autônomo para a geração de regras *Sigma*;
- Comparar os resultados dos métodos utilizados;
- Comparar métricas dos metadados derivados do agente.

1.3 Estrutura do trabalho

Este trabalho possui seis capítulos, organizados de forma a conduzir o leitor do embasamento teórico à abordagem metodológica que será adotada. No primeiro capítulo, apresenta-se a introdução do tema, defendendo a motivação, os objetivos e a justificativa do estudo. O capítulo 2 disserta sobre conceitos necessários para se compreender o assunto do trabalho: sistemas SIEM, regras *Sigma* e LLMs — o que são, como se configuram na segurança da informação e como se relacionam entre si diante da mitigação de ameaças. Já o capítulo 3 discute trabalhos relacionados, oferecendo uma visão crítica sobre pesquisas e soluções correlatas. O capítulo 4 descreve os materiais e os métodos utilizados, detalhando as etapas do desenvolvimento da proposta e quais serão as métricas de avaliação. No capítulo 5 estão os produtos do trabalho proposto, os resultados obtidos, a interpretação dos mesmos e discussão sobre as limitações encontradas. No último capítulo, a conclusão e sugestão de trabalhos futuros.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que sustentam o desenvolvimento do agente proposto. São discutidos os conceitos de SOC e SIEM na área de segurança da informação, as bases de conhecimento sobre vulnerabilidades e técnicas adversárias, o padrão *Sigma* para regras de detecção, os modelos de linguagem de grande escala (LLMs), a geração aumentada por recuperação (RAG) e os agentes autônomos como paradigma de orquestração de LLMs.

2.1 Segurança da Informação e SOC

Em organizações de médio e grande porte a defesa cibernética não costuma ser tarefa de uma única ferramenta. O volume e a heterogeneidade dos eventos gerados por servidores, *firewalls*, sistemas de detecção de intrusão (IDS), aplicações em nuvem e dispositivos de usuários tornam inviável a inspeção manual contínua, exigindo a composição de equipes e infraestruturas dedicadas ao monitoramento, à detecção e à resposta a incidentes (Vielberth *et al.*, 2020; Shah *et al.*, 2022). É neste contexto que surgem os SOC, unidades especializadas que centralizam pessoas, processos e tecnologias para sustentar a postura defensiva de uma organização em tempo integral.

Segundo Vielberth *et al.* (2020), um SOC bem estruturado articula três pilares interdependentes: analistas humanos com diferentes níveis de experiência, procedimentos formalizados de triagem e resposta e uma camada tecnológica capaz de coletar, correlacionar e priorizar evidências de ameaças. A operação típica se organiza em níveis funcionais, em que no *Tier 1* os analistas iniciais executam a triagem dos alertas, no *Tier 2* os investigadores intermediários conduzem a análise de incidentes e no *Tier 3* os especialistas de nível mais alto realizam a perícia forense (Tariq *et al.*, 2025). O componente tecnológico central que sustenta essa cadeia é o sistema SIEM, descrito na próxima seção.

2.1.1 SIEM

Um SIEM atua como o núcleo de inteligência operacional em ambientes de cibersegurança, agregando e correlacionando *logs* – registros automáticos de eventos gerados por sistemas, aplicações, redes e dispositivos, provenientes de toda a infraestrutura de Tecnologia da Informação. Como já citado anteriormente, as fontes típicas incluem *firewalls*, sistemas de detecção e prevenção de intrusão (IDS/IPS), servidores de aplicação e de banco de dados, controladores de domínio, *endpoints* e serviços em nuvem (González-Granadillo; González-Zarzosa; Diaz, 2021). Seu propósito reside na capacidade de correlacionar eventos aparentemente desconexos e identificar padrões que ferramentas isoladas não o fazem.

Convém distinguir o SIEM de outras soluções pontuais de defesa, como antivírus, *firewalls* ou plataformas de proteção de *endpoint* (EPP). Tais soluções operam de forma localizada – protegendo um *host*, um perímetro de rede ou um vetor específico, e geram alertas específicos de seu escopo. O SIEM, por outro lado, é uma camada agregadora: ele consome os *logs* produzidos por essas ferramentas, normaliza formatos heterogêneos para um esquema comum e aplica regras de correlação capazes de identificar, na conjunção de múltiplos eventos, padrões de ataque que permaneceriam imperceptíveis em uma análise isolada (González-Granadillo; González-Zarzosa; Diaz, 2021; Sheeraz *et al.*, 2024). Conforme destacam Podzins e Romanovs (2019), essa contextualização é decisiva em ações automatizadas contra ameaças complexas, como ataques *zero-day*, em que, por exemplo, a correlação entre uma tentativa de força bruta no *Active Directory* (serviço de diretório da *Microsoft* responsável pelo gerenciamento de identidades, recursos e permissões da rede interna) e um tráfego suspeito no *firewall* pode disparar uma resposta imediata.

Nos SOC, o SIEM assume papel operacional indispensável: consolida alertas de diferentes fontes e formatos, prioriza incidentes com base na criticidade e orquestra fluxos de resposta (*workflows*). Sem essa camada, os analistas perdem a capacidade de detectar fases avançadas de ataque, como movimento lateral e escalção de privilégios – etapas em que o invasor se desloca pelos sistemas do alvo acumulando privilégios até atingir o nível administrativo (Podzins; Romanovs, 2019). A ausência desse tipo de visão integrada está associada a violações de longa duração, como foi o caso da rede mundial de hotéis *Starwood*, em que a intrusão permaneceu não identificada por aproximadamente quatro anos e resultou no vazamento de dados de cerca de 383 milhões de hóspedes ao redor do mundo (Shepardson, 2019). Essa visão unificada transforma o SIEM no "sistema nervoso" do SOC, permitindo que analistas investiguem ameaças sistêmicas e não apenas alertas isolados.

A eficácia de um SOC, porém, está intrinsecamente ligada à robustez do SIEM e à calibração das suas regras de detecção. Regras mal calibradas geram excesso de falsos positivos e instâncias não otimizadas podem produzir mais de mil alertas diários, sobrecarregando as equipes – segundo Podzins e Romanovs (2019), um analista isolado consegue investigar com profundidade entre 5 e 15 alertas por dia, dependendo da criticidade. Esse desequilíbrio configura o fenômeno conhecido como *alert fatigue*, ou fadiga de alertas: a exposição contínua a grandes volumes de notificações de baixo valor leva os analistas à dessensibilização, ao prolongamento do tempo de resposta e, em última instância, ao *burnout* profissional, com impactos diretos na qualidade da defesa (Tariq *et al.*, 2025). A literatura recente identifica a fadiga de alertas como um dos principais desafios contemporâneos dos SOC e, justamente por isso, motiva a busca por mecanismos automatizados de triagem, correlação e geração de regras mais precisas – preocupação central do presente trabalho.

A importância estratégica do SIEM também se ancora na sua capacidade de reduzir o tempo de detecção de ameaças. Podzins e Romanovs (2019) citam o relatório de violações da *Verizon* de 2018, que apontava cerca de 68% das violações sendo descobertas meses ou

mais após o ataque inicial (Business, 2018); já o relatório de 2025 informa que o tempo médio de remediação de vulnerabilidades em dispositivos de borda (especialmente falhas de *zero-day*) é de 32 dias (Business, 2025). Essa lacuna pode ser substancialmente reduzida com um SIEM bem implementado, em virtude da correlação de eventos em tempo real. Adicionalmente, a plataforma SIEM serve de alicerce para a evolução tecnológica do SOC: é sobre ela que se integram componentes mais recentes, como módulos de inteligência artificial e aprendizado de máquina, que ampliam a detecção de ameaças sutis e habilitam a automação de respostas (Tendikov *et al.*, 2024).

Do ponto de vista econômico, embora o SIEM envolva custos elevados – com licenças baseadas em volume de *logs*, infraestrutura dedicada e equipe especializada em regime ininterrupto, o investimento se justifica diante da magnitude das perdas associadas a incidentes cibernéticos. Em 2019, a *Accenture* projetava perdas globais da ordem de US\$ 5,2 trilhões para os 5 anos seguintes (Accenture, 2019 apud Podzins; Romanovs, 2019), tendência que se confirmou nos anos posteriores: o relatório anual da IBM de 2024 ((IBM Security, 2024)) registrou um custo médio global de US\$ 4,88 milhões por violação de dados, com aumento de 10% em relação ao ano anterior. O último, de 2025, no entanto, trouxe uma pequena reviravolta: o custo médio global caiu 9% em relação a 2024, motivado por melhorias na identificação e contenção de incidentes (IBM Security, 2025). Para organizações com dados sensíveis ou sujeitas a regulações como o Regulamento Geral de Proteção de Dados, do inglês *General Data Protection Regulation* (GDPR), da União Europeia, ou o PCI-DSS – norma globalmente reconhecida para a segurança de dados de cartões de pagamento, o SIEM deixa de ser uma opção e torna-se requisito operacional. Em síntese, o SIEM mantém-se como pilar irrevogável da segurança corporativa, não apenas pela sua tecnologia isolada, mas por construir o ecossistema integrado de detecção e resposta sobre o qual se constroem as defesas modernas.

2.1.2 SIEMs Inteligentes

Apesar de sua centralidade operacional, o o sistema SIEM tradicional opera majoritariamente sobre regras estáticas – definições escritas por especialistas que descrevem condições de alertas a partir de padrões conhecidos. Essa abordagem apresenta duas limitações sistêmicas: a dificuldade em detectar ameaças cuja assinatura ainda não foi formalizada, como ataques , e a alta taxa de falsos positivos quando as regras não acompanham a dinâmica do ambiente protegido (González-Granadillo; González-Zarzosa; Diaz, 2021; Nurusheva *et al.*, 2024). González-Granadillo, González-Zarzosa e Diaz (2021) observam, especificamente, que pouquíssimas soluções de SIEM dispõem de mecanismos de correlação avançada capazes de realizar uma análise histórica e de desvios necessária, por exemplo, para a detecção posterior de ataques . Portanto, é a partir da integração de técnicas de aprendizado de máquina Aprendizado de Máquina, do inglês *Machine Learning* (ML) e IA, buscando superar tais limitações, que se originam os chamados SIEMs inteligentes.

De acordo com Saddi *et al.* (2024), a IA generativa é capaz de produzir dados sintéticos e simular cenários de ataque – replicando, por exemplo, a atuação de um invasor ou a propagação de *malware*, o que permite tanto treinar modelos de detecção quanto avaliar defesas antes de sua implantação, sustentando uma postura proativa de identificação de ameaças emergentes. Essa capacidade é amplificada por algoritmos de ML que analisam padrões contextuais em grandes volumes de *logs*, correlacionam eventos aparentemente desconexos – por exemplo, tráfego anômalo combinado com tentativas de força bruta, e identificam comportamentos compatíveis com técnicas adversárias documentadas. Tendikov *et al.* (2024) reforçam que a integração de IA e ML em SIEMs aprimora a correlação de grandes volumes de eventos em tempo real e permite que esses sistemas aprendam a partir de dados históricos e se adaptem dinamicamente a ameaças emergentes. Esse aprimoramento incide diretamente sobre a fadiga dos analistas discutida na seção anterior: Tariq *et al.* (2025) documentam que mecanismos de automação e triagem assistida por ML reduzem a carga cognitiva associada ao alto volume de alertas e ao excesso de falsos positivos, principais causas do esgotamento nos SOCs.

Nurusheva *et al.* (2024) demonstram que algoritmos de detecção de anomalias, análise preditiva e limiares adaptativos permitem identificar atividades sutis que escapariam das regras estáticas ou à análise manual, e propõem um sistema autoadaptativo, capaz de identificar ameaças inéditas em tempo real sem depender de regras predefinidas. Um aspecto central desse paradigma é a possibilidade de reduzir a dependência de bases previamente rotuladas ¹: como destacado por Tariq *et al.* (2025), algoritmos não supervisionados conseguem modelar o comportamento típico de uma rede e detectar anomalias sem a necessidade de conjuntos de dados rotulados, o que os torna particularmente eficazes na identificação de ameaças novas – embora ao custo de uma taxa de falsos positivos superior à dos métodos supervisionados. Do ponto de vista conceitual, esse comportamento decorre das próprias características do aprendizado de máquina, cuja capacidade de aprender padrões a partir de dados, sem programação explícita, fundamenta tanto as abordagens supervisionadas quanto as não-supervisionadas (Jordan; Mitchell, 2015). Ao combinar esses dois tipos de modelo, o SIEM com ML ajusta-se continuamente às mudanças no ambiente e antecipa novos vetores de ataque (Sheeraz *et al.*, 2024).

Por fim, compreende-se que depender exclusivamente de analistas humanos é inviável diante do crescimento exponencial dos dados e da sofisticação das ameaças. A incorporação de IA e ML em SIEMs representa, portanto, um avanço significativo da cibersegurança, conferindo precisão, adaptabilidade e automação a um cenário cada vez mais dinâmico e hostil. Esse movimento, no entanto, ainda enfrenta desafios relevantes, sobretudo no que tange à interpretabilidade dos modelos, à dependência de dados de qualidade para o treinamento e à necessidade de regras formais que estabeleçam o vocabulário comum entre as diferentes ferramentas. Seguindo este último ponto, nas próximas seções se discutirá sobre a efetividade

¹ Uma base rotulada (do inglês *labeled dataset*) condiz a uma coleção de pares entrada-saída, onde cada amostra de entrada (por exemplo, linha de *log* ou fluxo de rede) recebeu uma anotação confiável sobre o que se é de fato

de qualquer SIEM; sendo inteligente ou não, depende de regras de detecção formalizadas em padrões abertos e interoperáveis, ancoradas em bases consolidadas de conhecimento sobre vulnerabilidades e técnicas adversárias.

2.2 Bases de Conhecimento sobre Vulnerabilidades e Técnicas Adversárias

A construção de regras de detecção em SIEMs depende de um vocabulário comum para descrever vulnerabilidades, fraquezas e comportamentos adversários. Esse vocabulário é fornecido por bases abertas, mantidas por organizações de pesquisa e agências governamentais, que catalogam de forma sistemática os elementos de interesse para a defesa cibernética. Sem esses padrões, regras escritas por diferentes equipes seriam incomparáveis, e a interoperabilidade entre ferramentas seria inviável (Roy *et al.*, 2023). O agente proposto neste trabalho consulta quatro dessas bases ao longo de sua execução: CVE, Common Weakness Enumeration, do inglês *Common Weakness Enumeration* (CWE), NVD e ATT&CK. As subseções a seguir apresentam cada uma delas e justificam seu papel na geração das regras *Sigma*.

2.2.1 CVE e CWE

O CVE é um catálogo público de vulnerabilidades, operado pelo desde 1999 sob patrocínio da Agência de Segurança de Infraestrutura e Cibernética, do inglês *Cybersecurity and Infrastructure Security Agency* (CISA) — agência norte-americana de segurança cibernética (The MITRE Corporation, 2024)). Cada vulnerabilidade divulgada recebe um identificador único no formato CVE-AAAA-NNNN, em que AAAA é o ano da publicação e NNNN um número sequencial. Em 2024, o catálogo já ultrapassava 250 mil registros (The MITRE Corporation, 2025b). A principal função do CVE é eliminar ambiguidade: quando duas ferramentas mencionam "CVE-2021-44228", há certeza de que ambas se referem à mesma falha — aqui, a vulnerabilidade *Log4Shell*, na biblioteca *Apache Log4j*. Neste trabalho, o CVE atua como uma das entradas possíveis para o agente: quando o usuário fornece um identificador, ou quando a regra parte de uma descrição textual que o menciona, o agente consulta o catálogo para extrair contexto descritivo.

O CWE, também mantido pelo , opera em outro nível de abstração. Enquanto o CVE cataloga vulnerabilidades específicas em produtos identificáveis, o CWE classifica as categorias gerais de fraqueza subjacentes — *buffer overflow*, falhas de validação de entrada, exposição de dados sensíveis e centenas de outras. Uma vulnerabilidade CVE costuma estar associada a um ou mais CWEs que descrevem a natureza do problema (The MITRE Corporation, 2025a).

Para a geração de regras, essa abstração é valiosa: padrões CWE generalizam-se para famílias inteiras de vulnerabilidades, incluindo variantes ainda não catalogadas. O agente pro-

posto obtém os CWEs associados a um CVE por meio da consulta ao NVD, descrita na subseção 4.2.3.3.

2.2.2 MITRE ATT&CK

Em complemento aos catálogos de vulnerabilidades e fraquezas, o MITRE ATT&CK (*Adversarial Tactics, Techniques, and Common Knowledge*) fornece uma taxonomia de *Tactics, Techniques, and Procedures* — em português, Táticas, Técnicas e Procedimentos (TTP) efetivamente utilizados por adversários em incidentes reais (Strom *et al.*, 2018). A diferença é importante: CVE e CWE descrevem o que pode ser explorado; o ATT&CK descreve como os atacantes agem ao longo das fases de uma intrusão, desde o reconhecimento inicial até a exfiltração de dados.

A matriz *Enterprise*, variante mais utilizada, organiza essas TTPs em 14 táticas e centenas de técnicas, cada uma identificada por um código — por exemplo, T1190 para *Exploit Public-Facing Application*. Em uma revisão sistemática, (Roy *et al.*, 2023) identificaram mais de 50 contribuições científicas que utilizam o ATT&CK como base para pesquisas em inteligência de ameaças, detecção de intrusão e resposta a incidentes, destacando-o como referência consolidada na academia e na indústria.

No agente proposto, o ATT&CK é utilizado para validar as *tags* de técnicas que o LLM inclui na regra *Sigma* gerada, visto que modelos de linguagem podem inventar identificadores inexistentes (por exemplo, *attack.exploitation* em vez do real *attack.initial-access*); a consulta à base oficial do ATT&CK permite remover ou sinalizar referências inválidas antes da entrega final da regra.

Cabe registrar que o MITRE mantém outras bases relacionadas, em particular o Common Attack Pattern Enumerations and Classifications (CAPEC), que cataloga padrões de ataque em um nível de abstração mais alto que o ATT&CK. Este catálogo não foi incorporado ao agente neste trabalho porque sua granularidade não corresponde ao formato das regras *Sigma*, que operam sobre técnicas concretas e mapeáveis a eventos de *log* — exatamente o nível em que o ATT&CK atua.

2.2.3 NVD e CVSS

O NVD é o repositório oficial do governo norte-americano para vulnerabilidades de segurança, mantido pelo NIST, e atua como camada de enriquecimento do CVE. A cada vulnerabilidade identificada, o NVD acrescenta metadados estruturados como produtos e versões afetados, mapeamentos para CWE e pontuações de severidade calculadas segundo o Sistema de Pontuação de Vulnerabilidades Comuns, do inglês *Common Vulnerability Scoring System* (CVSS) (National Institute of Standards and Technology, 2025; Booth; Rike; Witte, 2013). O

CVSS, por sua vez, é um sistema híbrido; produz uma pontuação numérica de 0,0 a 10,0 a partir de métricas como vetor de ataque, complexidade, privilégios requeridos e impacto sobre confidencialidade, integridade e disponibilidade, e mapeia essa pontuação em categorias qualitativas (*None, Low, Medium, High e Critical*), oferecendo simultaneamente critério quantitativo para ordenação e classificação intuitiva para consumo humano (Forum of Incident Response and Security Teams, 2019). No agente proposto, o NVD faz uma requisição para obter a descrição da vulnerabilidade, os CWEs associados e a pontuação CVSS — dados que alimentam tanto o conteúdo descritivo da regra quanto a calibração do campo *level* (que indica a criticidade da regra *Sigma*).

2.3 Regras *Sigma*

Sigma (*Generic Signature Format for SIEM Systems*, “Assinatura em formato genérico para sistemas SIEM” em tradução livre (SIGMAHQ, 2025b)) é o projeto de código aberto de um padrão genérico de descrição de *logs* de eventos, aplicável a qualquer tipo de *log*, com o intuito de compartilhar a lógica de detecção de ameaças ou anomalias entre diferentes SIEMs. Ao operar com dados de ameaças de múltiplas fontes, o *Sigma* capacita os caçadores de ameaças a correlacionar eventos que passariam despercebidos por abordagens convencionais. Dessa forma, se obtém conhecimentos estratégicos sobre dados confiáveis e antecipa-se tentativas de ataque em estágios iniciais desta cadeia de eventos (McGowan, 2025).

Os mantenedores da iniciativa consideram que excelentes análises podem ser encontradas na *internet*, incluindo IOCs e regras YARA² para detectar arquivos ou ferramentas maliciosos, por exemplo, mas que muitas vezes limitam-se a descrições específicas, funcionais para poucos *softwares*. Portanto, da necessidade de haver um método de descrição específico ou genérico de eventos de *logs*, com portabilidade para vários SIEMs, a fim de aprimorar os recursos de detecção para todos, surgiu o *Sigma* (SIGMAHQ, 2025b).

O projeto de código aberto foi criado em 2017 por Florian Roth e Thomas Patzke (SIGMAHQ, 2025a), inspirados em padrões já existentes nas áreas de análise de *malware* e monitoramento de rede. Como já dito, o padrão *Sigma* se diferencia por lidar com vários tipos de esquemas de *logs*, unindo padrões que já existem e se diferenciam entre si (McGowan, 2025).

O projeto carrega essa denominação pois se considera um ecossistema — envolve três componentes: o formato *Sigma*, a ferramenta *Sigma* e a coleção de regras *Sigma*. Este último diz respeito ao repositório no *Github*, que possui mais de 3 mil regras de detecção de diferentes tipos; por ser de código aberto, pode receber sugestões de qualquer pessoa disposta a contribuir (SIGMAHQ, 2025b). Já a ferramenta *Sigma*, refere-se a um conversor de regras disponível para *download* — o *Sigma Command Line Interface* (CLI). Com ele é possível converter as regras *Sigma* para as regras usadas no SIEM de interesse do usuário; *Elasticsearch*, *CrowdStrike*,

² Uma regra YARA é um arquivo de extensão “.yar” contido de expressões lógicas usadas para classificar e identificar amostras de *malware* (PICUS SECURITY, 2025).

IBM QRadar AQL e *Splunk* são algumas marcas registradas para as quais essa integração está disponível.

2.3.1 Sigma e a mitigação de ameaças

As regras *Sigma* formam um conjunto estruturado de princípios e métodos essenciais para o sucesso da investigação focada em determinar as ameaças que podem prejudicar um ou mais sistemas (*Threat Hunting*), oferecendo um padrão uniforme que orienta os profissionais de segurança na elaboração de regras criteriosas para detecção de ameaças ou anomalias. Segundo Dimitrov e Nikolov (2025), essas regras otimizam a identificação, avaliação e a resposta a incidentes, proporcionando decisões mais precisas e oportunas, além de uma base sólida para controlar vetores de ameaças, minimizar riscos e prevenir violações de dados. Dessa forma, as regras *Sigma* constituem o alicerce de uma estratégia de segurança proativa e resiliente, capaz de antecipar e mitigar possíveis ataques de forma sustentável.

A caça a ameaças é um processo defensivo proativo que visa descobrir e neutralizar invasores, mapeando vulnerabilidades e antecipando movimentos adversos por meio da análise contínua de comportamentos anômalos e incidentes em curso. Nesse cenário, as regras *Sigma*, desde sua padronização, em 2017, desempenham um papel fundamental ao oferecer uma linguagem comum para correlação de eventos em múltiplas plataformas SIEM, unificando dados de sistemas heterogêneos e acelerando a detecção de padrões suspeitos. Conforme demonstrado por Dimitrov e Nikolov (2025), essa unificação reduz significativamente a carga de trabalho manual dos analistas, permitindo identificar ameaças ainda na fase inicial de comprometimento, elevando a eficácia das operações de *Threat Hunting*, ou “caça a ameaças”.

No entanto, ainda que a aplicação das regras *Sigma* viabilize inúmeros cenários benquistas, vale ressaltar que, ainda assim, os SIEMs continuarão sendo heterogêneos, pois utilizam formatos e padrões de dados diferentes — situação inclusive de eventos de diferentes dispositivos. As regras *Sigma* surgem como uma forma de expressar a regra ou a ameaça e o que deve ser buscado, mas ainda precisa ser convertida para o SIEM de destino (através do CLI), que já possui suas próprias regras.

2.3.2 Estrutura

As regras *Sigma* são arquivos YAML contidos de informações que viabilizam a detecção de comportamento estranho ou malicioso quando da inspeção de arquivos de *logs* dentro do contexto de um SIEM (SIGMAHQ, 2025c).

A compreensão da estrutura de uma regra *Sigma* e como é usada para detecção faz-se importante em razão de que parte da metodologia envolverá selecionar as informações mais críticas das regras e avaliá-las.

As regras são divididas em três grupos principais de instruções (SIGMAHQ, 2025c):

- Detecção (*detection*): qual comportamento malicioso a regra está procurando;
- Fonte do log (*logsource*): quais tipos de logs a detecção deve procurar; e
- Metadados (*title*, *id*, *status*, etc.): detalhes da regra.

A Figura 1 é um exemplo de regra *Sigma* que está disponível na página oficial do projeto.

Figura 1 – Exemplo de regra *Sigma* (SIGMAHQ, 2025c)

```

1  # ./rules/cloud/okta/okta_user_account_locked_out.yml
2  title: Okta User Account Locked Out
3  id: 14701da0-4b0f-4ee6-9c95-2ffb4e73bb9a
4  status: test
5  description: Detects when a user account is locked out.
6  references:
7    - https://developer.okta.com/docs/reference/api/system-log/
8    - https://developer.okta.com/docs/reference/api/event-types/
9  author: Austin Songer @austinsonger
10 date: 2021-09-12
11 modified: 2022-10-09
12 tags:
13   - attack.impact
14 logsource:
15   product: okta
16   service: okta
17 detection:
18   selection:
19     displaymessage: Max sign in attempts exceeded
20     condition: selection
21 falsepositives:
22   - Unknown
23 level: medium

```

Fonte: Captura de tela da página *Sigma Rules* em SigmaHQ (2025c).

A seção de detecção é o coração da regra *Sigma*, pois especifica o alvo da regra no vasto volume de *logs*. Adiante neste trabalho este componente será mencionado como um dos principais para a análise da qualidade das regras geradas pelas IAs. Vale mencionar que, dentro da seção de detecção, existem instruções para a escrita, assim como qualquer linguagem de programação; no entanto, num primeiro momento, basta compreender o esqueleto principal. Sendo assim, a segunda peça mais importante da regra é a fonte dos *logs*, o campo *logsource*; nele se especifica qual o tipo de *log* deve ser canalizado pela regra — ao invés de varrer todos os *logs* do SIEM. Por fim, as demais informações, os metadados, servem para guiar o usuário sobre o assunto da regra (SIGMAHQ, 2025c).

Para fins de orientação quando da criação de uma nova regra, é disponibilizado um esqueleto da regra com instruções (Figura 2) no repositório *Sigma* (Roth, 2022).

Figura 2 – Exemplo de regra *Sigma* (Roth, 2022)

```

1 title: a short capitalised title with less than 50 characters
2 id: generate one here https://www.uuidgenerator.net/version4
3 status: experimental
4 description: A description of what your rule is meant to detect
5 references:
6 |   - A list of all references that can help a reader or analyst understand the meaning of a triggered rule
7 tags:
8 |   - attack.execution # example MITRE ATT&CK category
9 |   - attack.tl059      # example MITRE ATT&CK technique id
10 |  - car.2014-04-003    # example CAR id
11 author: Michael Haag, Florian Roth, Markus Neis # example, a list of authors
12 date: 2018/04/06 # Rule date
13 logsource: # important for the field mapping in predefined or your additional config files
14 |   category: process_creation # In this example we choose the category 'process_creation'
15 |   product: windows          # the respective product
16 detection:
17 |   selection:
18 |       |   FieldName: 'StringValue'
19 |       |   FieldName: IntegerValue
20 |       |   FieldName|modifier: 'Value'
21 |   condition: selection
22 fields:
23 |   - fields in the log source that are important to investigate further
24 falsepositives:
25 |   - describe possible false positive conditions to help the analysts in their investigation
26 level: one of five levels (informational, low, medium, high, critical)

```

Fonte: Captura de tela do código genérico na página *Rule Creation Guide* no repositório Sigma(Roth, 2022).

2.4 Processamento de Linguagem Natural

O Processamento de Linguagem Natural, do inglês *Natural Language Processing* (PLN) é a área da computação que busca fazer com que máquinas compreendam e produzam a linguagem humana. Segundo Liddy (2001), trata-se de uma abordagem teórica e tecnologicamente fundamentada para analisar textos, com o objetivo de alcançar um processamento de linguagem semelhante ao humano. Para isso, o PLN permite ao computador analisar um texto sistematicamente em sete níveis de representação: fonológico (padrões de entonação e fonemas), morfológico (estrutura das palavras), léxico (significado individual das palavras), sintático (estrutura gramatical das frases), semântico (significado literal das frases), discursivo (coesão e estrutura dos textos) e pragmático (contexto) liddy2001nlp, jurafsky2009speech. Ao longo do tempo, as técnicas evoluíram dos sistemas simbólicos baseados em regras manuais para abordagens estatísticas e, mais recentemente, conexionistas (inspiradas no funcionamento do cérebro humano), impulsionadas pelo aumento do poder computacional e pela disponibilidade de grandes coleções de texto (Liddy, 2001; Chowdhary, 2020).

Sobre essa base, o PLN passou a sustentar aplicações que se tornaram centrais na computação contemporânea. A Recuperação de Informação (RI) encontra, em uma grande coleção de documentos, aqueles mais relevantes a uma consulta, ordenando-os por similaridade semântica em vez de apenas casar palavras idênticas (Manning; Raghavan; Schütze, 2008). A Extração de Informação (EI) parte de texto livre e deste são retirados dados estruturados, como entidades, datas, relações, igualmente quando um anúncio de emprego em texto corrido é convertido em campos de uma tabela (Chowdhary, 2020). A Tradução Automática (TA) converte um texto entre idiomas preservando o significado original, mantendo uma correspondência pró-

xima entre origem e destino. Já a Sumarização condensa documentos longos em versões mais curtas que retêm o essencial, podendo ser extrativa (seleciona frases do original) ou abstrativa (reescreve em novas palavras) (Jurafsky; Martin, 2009).

Os Sistemas de Pergunta-Resposta (QA) ocupam um nível distinto: em vez de transformar diretamente o texto de entrada, interpretam uma pergunta em linguagem natural e produzem uma resposta direta, frequentemente combinando RI, EI e sumarização como etapas internas (Jurafsky; Martin, 2009). A diferença essencial é o tipo de saída entregue ao usuário — enquanto a RI devolve uma lista de documentos para que ele mesmo leia e conclua, o QA devolve a conclusão já formulada. Todas essas aplicações enfrentam dificuldades comuns relacionadas à integração de conhecimento de mundo e ao raciocínio de senso comum, necessários para resolver ambiguidades contextuais (Chowdhary, 2020). Esse cenário começou a mudar no final da década de 2010, com o surgimento dos modelos de linguagem baseados em redes neurais profundas, em especial os Modelos de Linguagem de Grande Escala, descritos na próxima seção, cuja escala de dados e parâmetros permitiu superar boa parte dessas limitações sem depender de regras escritas à mão (Hasanov *et al.*, 2024).

Várias dessas aplicações reaparecem, de forma combinada, no agente desenvolvido neste trabalho. A recuperação de informação fundamenta a busca por regras Sigma semelhantes em uma base vetorial — núcleo da estratégia de geração aumentada por recuperação descrita na Seção 2.5. A extração de informação atua na identificação de indicadores técnicos, como identificadores CVE e *hashes*, a partir de texto livre. E a sumarização orienta a condensação do material coletado de fontes externas em uma descrição curta e factual da ameaça. Como se verá nas subseções seguintes, é justamente a possibilidade de unir essas tarefas em um único sistema, por meio de modelos de linguagem de grande escala, que torna essa arquitetura viável.

2.4.1 Modelos de Linguagem de Grande Escala (LLMs)

Os Modelos de Linguagem de Grande Escala (*Large Language Models*, LLMs) são sistemas de inteligência artificial treinados com bilhões ou trilhões de parâmetros, capazes de compreender e gerar textos em linguagem natural, ou seja, em formato de comunicação humano (Hasanov *et al.*, 2024). Sua arquitetura é baseada em *Transformers*: redes neurais que empregam mecanismos de *self-attention* (autoatenção) para focar simultaneamente em diferentes partes do texto, capturando relações contextuais de longo alcance entre as palavras (Vaswani *et al.*, 2017). Esses modelos aprendem por meio de aprendizado auto-supervisionado (*self-supervised learning*), estratégia em que o próprio texto fornece o sinal de treinamento: dada uma sequência incompleta, o modelo tenta prever o elemento ausente, extraindo padrões linguísticos a partir de imensos *corpora*³ textuais sem rótulos explícitos (Hasanov *et al.*, 2024).

³ *Corpora* é plural de *corpus* — termo em latim para "corpos" e no contexto de PLN significa coleções extensas e estruturadas de textos autênticos.

O mecanismo operacional central de um LLM é a predição sequencial de *tokens* — as menores unidades de texto em que o modelo divide sua entrada. Dada uma sequência de entrada, o modelo estima a probabilidade do próximo token, e essa habilidade é refinada continuamente ao longo do pré-treinamento. Ao operar sobre janelas de contexto de milhares de *tokens*, o modelo mantém coerência semântica em textos longos e responde a instruções formuladas em linguagem natural (Brown *et al.*, 2020; Jurafsky; Martin, 2009).

Uma propriedade particularmente relevante para o presente trabalho é a capacidade dos LLMs de produzir saídas em formatos estruturados, como YAML, JSON ou código-fonte. É essa capacidade que permite ao modelo gerar regras *Sigma*, que são justamente "artefatos" estruturados em YAML, com campos formalmente definidos, e a arquitetura que viabilizou esse comportamento é o *Transformer*, descrita a seguir, enquanto as estratégias para orientar o modelo sem retreinamento são objeto da subseção 2.4.3.

2.4.2 Arquitetura *Transformer*

A arquitetura *Transformer*, proposta por Vaswani *et al.* (2017) no artigo "*Attention is All You Need*", é o substrato técnico que viabilizou os modelos de linguagem contemporâneos. Antes dela, o processamento de texto era dominado por redes neurais recorrentes (RNN), que processavam o texto palavra por palavra, em sequência, o que dificultava o paralelismo computacional e limitava a captura de dependências entre elementos distantes.

O *Transformer* substitui a recorrência por um mecanismo denominado *self-attention* (autoatenção). Em vez de processar um *token* de cada vez, o modelo analisa todos os *tokens* da entrada simultaneamente e calcula, para cada um, o quanto os demais *tokens* da sequência são relevantes para sua representação. O resultado é uma representação contextualizada de cada *token*, que incorpora informação de toda a sequência (Vaswani *et al.*, 2017).

Essa formulação trouxe duas consequências práticas que explicam por que o *Transformer* se tornou a base dos LLMs. A primeira é o paralelismo: como todas as relações são calculadas simultaneamente, o treinamento torna-se massivamente paralelizável, viabilizando modelos com bilhões de parâmetros. A segunda é a captura de dependências de longo alcance: a relação entre dois tokens distantes é calculada com o mesmo custo da relação entre tokens adjacentes, eliminando o viés que penalizava informações distantes nas RNNs (Vaswani *et al.*, 2017). A arquitetura original combina um encoder, voltado à compreensão da entrada, e um decoder, voltado à geração da saída; os LLMs empregados neste trabalho derivam de sua porção decoder, especializada na geração de texto.

2.4.3 Engenharia de *Prompt*

Diferentemente dos modelos clássicos de aprendizado de máquina, que precisam ser retreinados para cada nova tarefa, os LLMs podem ser condicionados em tempo de inferência por meio de instruções em linguagem natural — os *prompts*. A formulação dessas instruções constitui o ramo da "engenharia de *prompt*", cujos paradigmas mais estabelecidos foram caracterizados por Brown *et al.* (2020).

O primeiro paradigma é o *zero-shot*, em que o modelo recebe apenas a descrição da tarefa e a entrada, sem nenhum exemplo de saída esperada. Instruir o modelo a "gerar uma regra *Sigma* para detectar a vulnerabilidade CVE-2021-44228" é uma chamada *zero-shot*: o modelo precisa inferir, a partir do que aprendeu no pré-treinamento, o formato esperado e os detalhes a incluir. O segundo paradigma é o *few-shot*, em que o *prompt* inclui um pequeno número de exemplos (tipicamente entre dois e dez) que ilustram a transformação desejada entre entrada e saída. Esses exemplos atuam como demonstração contextual: o modelo identifica o padrão a partir das amostras e o reproduz, sem que qualquer parâmetro seja atualizado (Brown *et al.*, 2020).

A diferença entre os dois tem implicações práticas. O *zero-shot* depende inteiramente do conhecimento incorporado no pré-treinamento, sendo mais sensível a domínios pouco representados nos dados de origem; o *few-shot* fornece especificação adicional sem custo de treinamento, mas consome parte do orçamento de *tokens* da janela de contexto. Brown *et al.* (2020) demonstraram que o ganho do *few-shot* sobre o *zero-shot* é mais expressivo em modelos de maior escala.

Estratégias mais elaboradas combinam essas modalidades com instruções estruturais. O *Chain-of-Thought* (CoT), proposto por Wei *et al.* (2022), instrui o modelo a externalizar seu raciocínio em passos intermediários antes de produzir a resposta final, o que demonstrou ganhos significativos em tarefas que exigem composição lógica. Em vez de gerar diretamente a saída, o modelo é induzido a "pensar em voz alta", decompondo o problema e justificando decisões parciais, o que tende a reduzir erros em tarefas estruturadas.

No presente trabalho, esses paradigmas constituem objeto direto de investigação experimental. O Estágio 2 da metodologia emprega geração *zero-shot* como linha de base; o Estágio 3 aplica engenharia de *prompt* estruturada, com instruções refinadas, seções nomeadas e restrições de formato; e o Estágio 4 incorpora recuperação aumentada de exemplos via RAG, configurando uma forma de *few-shot* contextualizada, combinada a uma etapa explícita de CoT, em que o agente é instruído a raciocinar sobre a fonte de *log*, os campos relevantes, os modificadores apropriados e a técnica ATT&CK correspondente antes de emitir a regra *Sigma* final. A comparação controlada entre essas estratégias permite quantificar o ganho marginal de cada camada de complexidade, conforme detalhado no Capítulo 4.

2.5 Geração Aumentada por Recuperação (RAG)

Os Modelo de linguagem de grande escala, do inglês, *Large Language Model* apresentam duas limitações estruturais quando aplicados a domínios técnicos especializados. A primeira é o corte de conhecimento (*knowledge cutoff*), já que o modelo só conhece informações disponíveis até a data em que foi treinado, e tudo o que surge depois (um novo CVE, uma nova técnica de ataque, uma atualização de norma) fica fora do seu alcance. A segunda é a tendência à alucinação: na ausência de informação confiável, o modelo pode produzir respostas plausíveis em forma, mas incorretas em conteúdo. Para mitigar ambos os problemas, Lewis *et al.* (2020) introduziram o paradigma da *Retrieval-Augmented Generation*, em português, Geração Aumentada por Recuperação (RAG), que combina o LLM com uma base externa de documentos consultada em tempo de inferência. O funcionamento do RAG pode ser dividido em duas etapas. Primeiro, dado um pedido feito pelo usuário, um mecanismo de recuperação seleciona, de uma base previamente indexada, os documentos mais relevantes para responder àquele pedido. Em seguida, esses documentos recuperados são fornecidos como contexto adicional ao modelo de linguagem, que executa a etapa de geração se apoiando simultaneamente em seu conhecimento interno e nas evidências externas recém-fornecidas (Lewis *et al.*, 2020). Este arranjo implica que o conhecimento do sistema pode ser atualizado sem retreinar o modelo, bastando acrescentar documentos novos à base; que as respostas se tornam rastreáveis aos documentos que as embasaram, oferecendo um caminho para a diminuição de alucinações; e que o domínio de especialização pode ser adaptado com a substituição da base de documentos, embora a qualidade do sistema permaneça fortemente dependente da curadoria e indexação dessa base (Lewis *et al.*, 2020).

A próxima subseção detalha os mecanismos técnicos que tornam essa recuperação possível. Em particular, como textos são convertidos em representações numéricas que permitem comparação semântica. E a subseção seguinte discute aplicações específicas de RAG em cibersegurança.

2.5.1 *Embeddings* e Bancos de Dados Vetoriais

A recuperação eficaz de documentos relevantes depende de quão confiável é a medição do quanto dois textos tratam do mesmo assunto. Buscas baseadas em correspondência literal de palavras-chave costumam falhar, porque dois textos podem descrever o mesmo conceito usando vocabulários diferentes — um analista pode escrever "escalada de privilégios", enquanto a documentação técnica usa "*privilege escalation*". Modelos clássicos de recuperação já tratavam essa limitação representando documentos como vetores em um espaço de termos (*Vector Space Model*), com peso de relevância calculado por esquemas como Frequência de Termo–Frequência Inversa de Documento, do inglês *Term Frequency–Inverse Document*

Frequency (TF-IDF) (Manning; Raghavan; Schütze, 2008). Sistemas modernos de RAG, no entanto, recorrem a representações vetoriais densas e contextualizadas (os *embeddings*).

A ideia central dos *embeddings* modernos é que textos com sentido próximo são posicionados próximos uns dos outros no espaço vetorial, mesmo quando usam palavras diferentes. Para gerar essas representações, modelos especializados são treinados a partir de redes neurais *Transformer* adaptadas: Reimers e Gurevych (2019) propuseram o *Sentence-BERT* (SBERT), uma das arquiteturas mais usadas para esse fim, que estende o modelo BERT em uma rede siamesa para produzir, com baixo custo computacional, *embeddings* fixos por sentença. A partir desse trabalho derivou-se uma família de modelos amplamente adotada em sistemas de RAG, incluindo o `all-MiniLM-L6-v2` e o `BAAI/bge-small-en-v1.5` utilizados neste trabalho.

Uma vez que cada documento esteja representado por um vetor, a similaridade entre dois textos pode ser quantificada por uma métrica geométrica. A mais comum é a similaridade de cosseno, que mede o ângulo entre dois vetores: vetores próximos no espaço produzem cosseno próximo de 1 (alta similaridade), enquanto vetores ortogonais resultam em cosseno próximo de 0 (sem relação). Essa métrica é especialmente adequada para a comparação semântica de textos porque ignora a magnitude dos vetores e foca apenas na direção que eles apontam — uma propriedade almejada quando se compara conteúdo semântico em documentos de diferentes comprimentos (Manning; Raghavan; Schütze, 2008). Porém, cabe registrar que a eficácia da recuperação por similaridade depende fortemente da qualidade dos *embeddings* utilizados. Representações inadequadas ao domínio podem aproximar documentos semanticamente distantes, ou afastar documentos relevantes, comprometendo a etapa subsequente de geração.

À medida que a base cresce, percorrer todos os vetores a cada consulta se torna inviável. Bancos de dados vetoriais especializados foram desenvolvidos para resolver esse problema, indexando os *embeddings* em estruturas que permitem a recuperação aproximada dos vizinhos mais próximos em tempo sublinear. Em particular, Johnson, Douze e Jégou (2019) demonstraram que técnicas de busca por similaridade podem ser escaladas para bilhões de vetores quando combinadas a aceleração por *Graphics Processing Unit* (GPU), viabilizando aplicações práticas de RAG em larga escala. Entre as implementações mais utilizadas estão o FAISS, do *Meta AI Research*, e o *ChromaDB*, banco vetorial leve e de código aberto adotado neste trabalho. O *ChromaDB* armazena os *embeddings* das regras *Sigma* indexadas e responde, para cada consulta, com os documentos mais semanticamente próximos, prontos para serem fornecidos como contexto ao *LLM gerador*.

2.5.2 RAG aplicado à Cibersegurança

O paradigma de RAG vem sendo investigado em cibersegurança como uma alternativa promissora à atualização contínua de LLMs por retreinamento. Há razões estruturais do pró-

prio domínio que motivam essa investigação. O cenário de ameaças evolui de forma contínua, com novas vulnerabilidades e técnicas adversárias surgindo diariamente, ritmo que torna economicamente inviável manter um LLM atualizado por retreinamento frequente. Além disso, o conhecimento especializado em segurança da informação está distribuído em múltiplas fontes (bases padronizadas, relatórios técnicos, regras públicas de detecção, entre outras), formato adequado a um sistema de recuperação. Reconhecendo essas afinidades, o uso de LLMs em cibersegurança vem crescendo rapidamente, com aplicações documentadas em detecção de ameaças, análise de vulnerabilidades e suporte a SOCs (Hasanov *et al.*, 2024).

Investigações recentes exploram essa direção em diferentes frentes. Rajapaksha, Rani e Karafili (2024) propuseram um sistema de pergunta-resposta baseado em RAG para investigação e atribuição de ataques cibernéticos, reportando redução de alucinações e rastreabilidade das fontes utilizadas na geração das respostas, com desempenho superior (em avaliação sobre sua base de conhecimento curada) ao uso isolado de GPT-3.5 e GPT-4o. Os autores também mostram que o modelo RAG pergunta-resposta gera respostas melhores quando recebe exemplos *few-shot* em vez de instruções *zero-shot*.

Outras aplicações descritas na literatura recente incluem o enriquecimento de inteligência de ameaças, a geração assistida de cenários de simulação de ataque e o suporte automatizado a analistas em SOCs. Cabe notar, contudo, que esses ganhos não são universais: a qualidade da recuperação permanece como gargalo recorrente, podendo deixar de fora o documento correto mesmo quando ele existe na base, e a sensibilidade do método à curadoria do , à escolha do modelo de *embeddings* e à formulação da consulta torna o desempenho do RAG fortemente dependente das decisões de projeto.

Neste trabalho, o RAG opera sobre uma base de 3.134 regras *Sigma* do repositório oficial *SigmaHQ*, previamente convertidas em *embeddings* e armazenadas no *ChromaDB*. A cada novo pedido, seja a partir de um CVE, de um *hash*, de um texto sobre uma ameaça recente ou de uma combinação dessas entradas, o agente consulta o banco vetorial e recupera as regras mais semanticamente próximas, que servem como exemplos contextuais durante a geração da nova regra. Esse mecanismo materializa, na prática, a aplicação simultânea das três tarefas clássicas de PLN discutidas na Seção 2.4: recuperação de informação (busca por regras semelhantes), extração de informação (identificação de CVE e hash na entrada) e sumarização (condensação do material coletado em uma descrição factual). A avaliação experimental do impacto efetivo dessa estratégia, comparada a abordagens alternativas, é objeto do Capítulo 5. A integração de todos esses componentes em um único fluxo automatizado é tema da próxima seção, sobre Agentes Autônomos.

2.6 Agentes Autônomos

Um agente autônomo é, na definição clássica de Russell e Norvig (2020), um sistema computacional capaz de perceber seu ambiente por meio de sensores, raciocinar sobre essas

percepções e atuar sobre o ambiente por meio de atuadores para alcançar objetivos definidos, com mínima ou nenhuma intervenção humana entre a entrada e a saída. Atualmente, agentes autônomos baseados em LLMs tem se consolidado como uma abordagem promissora, na qual o LLM atua como “núcleo cognitivo” do sistema, sendo responsável pelo raciocínio em linguagem natural, enquanto componentes externos, como ferramentas, memórias e bases de conhecimento, ampliam suas capacidades para além do que o modelo conseguiria realizar isoladamente. Para este tipo de agente, Wang *et al.* (2024) propõem um arcabouço unificado que articula quatro módulos: perfil (a identidade e o papel atribuídos ao agente), memória (representação do contexto e do histórico), planejamento (decomposição de tarefas em passos) e ação (execução por meio de ferramentas).

A distinção entre utilizar um LLM e construir um agente é fundamental para compreender a contribuição do presente trabalho. Em uma chamada direta ao LLM, o usuário envia uma pergunta e obtém uma resposta única e estática, sem que o modelo consulte fontes externas, corrija sua própria saída ou integre múltiplas etapas de raciocínio. Em um agente, por outro lado, o LLM é orquestrado em um fluxo que pode envolver sucessivas chamadas, recuperação de informações externas, validação de resultados intermediários e ajustes iterativos — comportamentos sustentados pelos módulos de memória, planejamento e ação descritos por Wang *et al.* (2024).

As duas subseções seguintes abordam, respectivamente, o padrão de raciocínio *ReAct* e a biblioteca adotada para implementar a arquitetura do agente proposto neste trabalho.

2.6.1 Padrão *ReAct*

O padrão *ReAct* (*Reasoning and Acting*), proposto por Yao *et al.* (2023), é um esquema para a construção de agentes baseados em LLMs. Sua ideia central é intercalar, em uma mesma sequência, traços de raciocínio (*Thought*) e ações (*Action*) sobre ferramentas externas, intercaladas com observações (*Observation*) dos resultados dessas ações. O ciclo opera em três passos, iniciando com o raciocínio do modelo sobre o estado atual do problema, em seguida decide qual ação executar (se consulta uma *Application Programming Interface*, em português Interface de Programação de Aplicações (API) ou faz busca em uma base, por exemplo), e por fim incorpora a observação do resultado ao seu raciocínio subsequente, repetindo o ciclo até concluir a tarefa (Yao *et al.*, 2023).

A contribuição do *ReAct* foi mostrar que combinar raciocínio e ação no mesmo fluxo pode reduzir alucinações e melhorar a precisão em tarefas que exigem informação externa, em comparação ao uso isolado de cada um dos dois mecanismos (Yao *et al.*, 2023). O agente desenvolvido neste trabalho incorpora parcialmente esse paradigma, pois o LLM é instruído a produzir raciocínio explícito antes de gerar a regra *Sigma*, no formato *Thought-Action-Observation* descrito por Yao *et al.* (2023). Porém, diferentemente do *ReAct* original, a sequência de ações do agente como consultar APIs externas, recuperar contexto com RAG, gerar a regra, validar e

refazer se necessário, não é decidida dinamicamente pelo modelo, mas orquestrada por uma máquina de estados externa, descrita na próxima subseção.

2.6.2 *LangGraph* como orquestrador

O *LangGraph* é uma biblioteca de código aberto do *framework* de orquestração de código aberto *LangChain* ⁴, voltada para a construção de agentes em forma de grafos dirigidos (LangChain Team, 2024b). Cada nó do grafo representa uma etapa funcional (uma chamada ao LLM, uma consulta a uma API ou uma rotina de validação) e as arestas determinam a ordem e as condições de transição entre nós. Todos os nós operam sobre um estado compartilhado, tipicamente estruturado como um dicionário tipado (*Graphstate*), que cada etapa pode ler e atualizar. Essa formulação permite representar fluxos com ramificações condicionais, ciclos de correção e roteamento dinâmico, enquanto se mantém controle determinístico sobre a estrutura geral do agente (LangChain, 2024).

O agente proposto neste trabalho é construído sobre esta biblioteca e sua arquitetura se organiza em seis nós sequenciais: classificador de entrada, consulta a APIs externas, recuperação por RAG, geração da regra, validação sintática e julgamento semântico. Todos os nós estão conectados por arestas fixas, exceto entre a validação e a geração, em que uma aresta condicional implementa um ciclo de auto-correção: caso a regra gerada seja rejeitada pelo validador, o estado é devolvido ao nó de geração com o erro registrado, permitindo nova tentativa enriquecida pela informação do erro anterior. Essa configuração caracteriza o sistema como um agente autônomo baseado em estado (*stateful autonomous agent*), conforme a taxonomia de Wang *et al.* (2024), porque possui núcleo de raciocínio (o LLM), memória estruturada (estado compartilhado), acesso a ferramentas externas (APIs e banco vetorial) e capacidade de auto-correção sem intervenção humana. Cabe registrar, no entanto, os limites de autonomia da implementação adotada: a sequência de ferramentas consultadas é fixa, e o agente não implementa experiências entre execuções — características presentes em arquiteturas mais avançadas (agentes treinados) e identificadas como direções de trabalho futuro.

2.7 Métricas de similaridade

Métricas de similaridade são funções matemáticas que recebem dois objetos e produzem um valor numérico que quantifica o quanto eles se parecem. São ferramentas fundamentais em recuperação da informação, mineração de texto e aprendizado de máquina, e cobrem aplicações que vão desde a busca por documentos relevantes em uma base até a comparação entre a saída de um modelo gerador e uma referência de avaliação (Manning; Raghavan; Schütze, 2008). Esta seção apresenta as métricas índice de *Jaccard* e similaridade do cosseno, que se-

⁴ www.langchain.com

rão empregadas neste trabalho para comparar as regras *Sigma* geradas pelo agente proposto contra o conjunto de regras de referência. Elas operam sobre representações distintas dos dados — conjuntos discretos e vetores contínuos, e oferecem perspectivas complementares sobre o conceito de “semelhança”.

2.7.1 Índice de Jaccard

O índice de *Jaccard* foi proposto pelo botânico suíço Paul *Jaccard* em 1912, no estudo da distribuição de espécies vegetais nos Alpes, como um coeficiente para quantificar a semelhança entre comunidades vegetais a partir das espécies que tinham em comum (Jaccard, 1912). Desde então, foi adotado em diversas áreas da ciência da computação para comparar conjuntos discretos. Dados dois conjuntos finitos A e B , o índice é definido como a razão entre o tamanho da interseção e o tamanho da união dos dois conjuntos:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

O valor de $J(A, B)$ varia entre 0 e 1. Quando os conjuntos não compartilham nenhum elemento, a interseção é vazia e o índice vale 0; quando os conjuntos são idênticos, a interseção é igual à união e o índice vale 1; valores intermediários indicam graus parciais de sobreposição.

Para efeitos de ilustração do conceito, considere dois conjuntos de palavras-chave associados a duas regras de detecção: $A = \{\text{powershell}, \text{encoded}, \text{base64}\}$ e $B = \{\text{powershell}, \text{encoded}, \text{obfuscated}, \text{macro}\}$. A interseção contém dois elementos (`powershell` e `encoded`), e a união contém cinco — portanto, $J(A, B) = 2/5 = 0,4$. A leitura é que as duas regras compartilham 40% dos termos considerados (Manning; Raghavan; Schütze, 2008). Essa interpretabilidade direta torna o *Jaccard* particularmente útil quando se deseja responder a perguntas pontuais sobre quais elementos estão ou não em comum.

2.7.2 Similaridade do Cosseno

A similaridade do cosseno é a métrica padrão em modelos de espaço vetorial, paradigma que representa objetos, tipicamente documentos, como vetores em um espaço de muitas dimensões (Manning; Raghavan; Schütze, 2008).

Dado dois vetores $\mathbf{v}$ e $\mathbf{w}$, a similaridade é definida como o cosseno do ângulo entre eles, calculado pelo produto escalar dos vetores normalizado pelos seus comprimentos (normas):

$$\cos(\mathbf{v}, \mathbf{w}) = \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|} \quad (2)$$

Para vetores com componentes não negativos, como vetores TF-IDF ou *embeddings* de texto normalizados, o resultado varia entre 0 e 1: vetores que apontam na mesma direção

produzem cosseno próximo de 1, o que indica alta similaridade, e vetores ortogonais produzem cosseno próximo de 0, indicando nenhuma relação. A propriedade fundamental dessa métrica é que ela depende apenas da direção dos vetores, ignorando suas magnitudes — aspecto útil na comparação de textos com extensões distintas, nos quais a norma do vetor se altera conforme o comprimento do documento, enquanto a direção reflete o conteúdo semântico de forma consistente (Manning; Raghavan; Schütze, 2008). Por essa razão, a similaridade do cosseno se tornou a métrica de referência em sistemas de recuperação semântica, incluindo o mecanismo de RAG descrito na Seção 2.5.1.

2.7.3 Aplicação

A aplicação das métricas neste trabalho será na etapa de avaliação das regras *Sigma* geradas pelo agente de modo a compará-las com as regras de referência extraídas do repositório oficial *SigmaHQ*. A similaridade do cosseno é calculada entre os *embeddings* da regra gerada e da regra de referência, oferecendo uma medida global de proximidade semântica entre as duas.

Em contrapartida, o índice de *Jaccard*, será aplicado separadamente nos três conjuntos extraídos das regras: as *tags* de técnicas MITRE ATT&CK declaradas, os campos do bloco *detection* e o par (*category*, *product*) do *logsource*. Apresentar os três valores de *Jaccard* separadamente, em vez de combiná-los em uma única média, revela o *tipo* de erro que a regra gerada apresenta: uma regra pode acertar a fonte de *log* mas declarar técnicas equivocadas, ou fazer o contrário. Apesar de um eventual desempenho médio similar, as duas falham por razões diferentes e exigiriam correções diferentes na prática.

Demais procedimentos operacionais e critérios de interpretação estão detalhados no Capítulo 4.

3 TRABALHOS RELACIONADOS

3.1 *LLMCloudHunter*

O *LLMCloudHunter* é uma proposta de Schwartz *et al.* (2025), um *framework* que utiliza o modelo -4o para automatizar a geração de regras *Sigma* a partir de relatórios de código aberto de inteligência de ameaças, voltados a ambientes em nuvem (*Open-Source Cyber Threat Intelligence* (OSCTI)). Os autores identificaram três limitações em trabalhos anteriores que motivaram a pesquisa: a ausência de saídas acionáveis, prontas para serem aplicadas em sistemas de segurança; o desperdício de informações visuais presentes em relatórios de ameaças, como capturas de tela e diagramas de fluxo de ataque; e o foco em ambientes *on-premises* (infraestrutura local), em detrimento dos ambientes em nuvem. Para superar essas lacunas, o *framework* utiliza uma abordagem multimodal, ou seja, com módulos que processam texto e imagens de forma conjunta.

A metodologia se organiza em três fases. Na fase de pré-processamento, a ferramenta é empregada no *web scraping* do código HTML do relatório, onde se descartam elementos irrelevantes – como barras laterais e propagandas; o conteúdo restante é convertido em um formato de texto unificado, no qual cabeçalhos, listas e tabelas têm sua estrutura hierárquica preservada. Em seguida, o componente *Image Analyzer* interpreta cada imagem do relatório por meio das capacidades visuais nativas do -4o, transcreve seu conteúdo para texto e o reintegra à posição original no documento.

Na segunda fase o processamento é em nível de parágrafo, onde acontece a extração de entidades (pedaço de informação relevante) e a geração inicial dos candidatos à regra final. O componente *API Call Extractor* atua em duas etapas: primeiro extrai chamadas de API explicitamente citadas no texto e, em seguida, infere chamadas mencionadas de forma implícita. A inferência é necessária porque relatórios de ameaças frequentemente descrevem operações de alto nível que não correspondem diretamente a eventos registrados nos *logs*. Conforme ilustram os próprios autores, "descrições operacionais, como a realização de uma ação de sincronização (*sync*) em um *bucket* S3, devem ser mapeadas para as chamadas de API *ListBuckets* e *GetObject*" (Schwartz *et al.*, 2025, p. 1925). Para mitigar alucinações e omissões do modelo, esse componente incorpora um mecanismo de votação por maioria, executando a extração múltiplas vezes e mantendo apenas as ocorrências que ultrapassam um valor de referência pré-definido. Depois, o *TTP Extractor* mapeia cada chamada de API às respectivas táticas, técnicas e subtécnicas do MITRE ATT&CK, e o *Rule Generator* sintetiza essas informações na estrutura sintática das regras *Sigma*. Por fim, o *Rule Validator* higieniza os candidatos gerados, remove campos excessivamente específicos e ajusta a sintaxe ao padrão *Sigma*.

Na terceira fase, de processamento em nível de OSCTI, o escopo de refinamento é global – o *framework* olha todos os candidatos juntos para tomar decisões. O *Rule Optimizer* executa duas operações lógicas: unificação, que mescla campos de seleção que compartilham

os mesmos critérios de detecção; e separação, que divide campos com lógicas divergentes incorretamente agrupados. Em seguida, o *Set Refiner* elimina redundâncias decorrentes do processamento paralelo dos parágrafos, selecionando, para cada chamada de API repetida em múltiplos candidatos, a regra mais adequada e descartando ou ajustando as demais. Paralelamente, o *IoC Extractor* identifica endereços IP e *User Agents* citados no relatório, e em seguida o *IoC Enhancer* os integra ao conjunto final como campos de seleção adicionais, conectados à lógica original por meio de operadores condicionais, sem comprometer a estrutura de detecção previamente estabelecida.

Os resultados de Schwartz *et al.* (2025) demonstram que, na identificação das chamadas de API (sequência de comandos) atribuídas ao agente da ameaça nos relatórios analisados, o *framework* acertou 83% das extrações e recuperou 99% das ocorrências presentes nos relatórios. Para os indicadores de comprometimento (IOC), os índices foram de 99% e 97%, respectivamente. A avaliação foi conduzida sobre uma base de vinte relatórios reais de ameaças em nuvem, previamente anotados de forma manual pela equipe de pesquisa. Além disso, 99,18% das regras *Sigma* geradas pelo *framework* foram compiladas com sucesso e convertidas em consultas para a plataforma, o que indica que a saída produzida atende ao padrão esperado. O estudo dos autores confirmou ainda a importância do processamento das imagens contidas nos relatórios, evidenciando que a leitura conjunta de texto e elementos visuais é uma estratégia adequada para lidar com a diversidade de formatos encontrados em relatórios de ameaças em nuvem (Schwartz *et al.*, 2025, seção 4.3).

3.2 RulePilot

O *RulePilot*, derivado do estudo de Wang *et al.* (2026), trata-se de um agente baseado em LLM para geração e conversão automatizada de regras de detecção em sistemas SIEMs proprietários, com foco principal em regras escritas na linguagem do (SPL), com suporte adicional à linguagem do *Microsoft Sentinel* (a *Kusto Query Language* (KQL)). Nas motivações do trabalho, os autores justificam o foco em regras específicas de cada SIEM sob a argumentação de que formatos genéricos como o *Sigma*, apesar de sua portabilidade entre diferentes plataformas e fornecedores, dependem majoritariamente de correspondência de padrões por expressões regulares e não suportam consultas complexas como agregações estatísticas, junções multi-fonte ou cálculos em janelas temporais, tendo, portanto, limitações na eficácia de detecção de ataques sofisticados que envolvem sequências de eventos. Wang *et al.* (2026) citam como referência de trabalhos anteriores de geração automática de regras o próprio *LLM-CloudHunter* (Schwartz *et al.*, 2024) – citado na seção anterior do presente trabalho. Os autores sustentam que *frameworks* como o citado contém tais limitações por restringirem-se ao formato *Sigma*. Portanto, a motivação central do *RulePilot* é suprir a lacuna entre descrições de alto nível em linguagem natural e as regras complexas, específicas de cada fornecedor, requeridas por SIEMs em produção.

A arquitetura do *framework RulePilot* é organizada em três componentes principais. O primeiro é o módulo de raciocínio em cadeia (*Chain-of-Thought* (CoT)), no qual a descrição de entrada é decomposta em seis etapas estruturadas: interpretação dos objetivos de segurança, identificação das fontes de *logs*, definição de filtros iniciais, extração de campos relevantes, agregação de dados e otimização da regra. O segundo é a representação intermediária (*Intermediate Representation* (IR)), uma abstração estruturada que separa a lógica semântica da sintaxe específica do SIEM-alvo: `<KEYWORD> | PARAMS {k_i=v_i} | MODULES {m_j}`; *KEYWORD* designa a função da etapa (como *FILTER*, *EXTRACT*, *AGGREGATE* e *JOIN*); *PARAMS* define as configurações obrigatórias e *MODULES* descreve a intenção analítica. Essa separação permite que o LLM se concentre no raciocínio semântico antes de se comprometer com a sintaxe final. O terceiro componente é o mecanismo de reflexão e otimização iterativa, que avalia a regra gerada em três dimensões – consistência lógica, correção sintática e viabilidade de execução, e dispara dois tipos de refinamento: regeneração de etapas da cadeia de raciocínio e a integração com a API do real para validar com execução de *logs* verdadeiros. Para a funcionalidade de conversão entre linguagens SIEM, o *RulePilot* incorpora ainda um mecanismo de RAG alimentado pela documentação oficial de migração da *Microsoft*, que mapeia comandos do SPL para seus equivalentes em KQL por meio de busca vetorial combinada à re-ranqueamento lexical. O *RulePilot* foi avaliado utilizando três LLMs distintos como motor – -4o, -V3 e -3, sobre 1.699 regras oficiais do repositório *Splunk Security Content* e 229.968 *logs* coletados de 61 testes atômicos do ATT&CK.

Os resultados de Wang *et al.* (2026) afirmam que o *RulePilot* supera os LLMs autônomos (sem o *framework*) em até 107,4% na similaridade textual com as regras de referência, medida pelas métricas BLEU, ROUGE-1, ROUGE-L e METEOR¹. Na avaliação de execução real sobre o , o *RulePilot* atingiu precisão de 100% em diversas táticas do ATT&CK, incluindo reconhecimento, acesso inicial e coleta, com desempenho superior ao -4o autônomo. Tais ganhos, contudo, acompanham certo custo computacional: o *RulePilot* consome cerca de dez vezes mais *tokens* de *prompt* e aproximadamente seis vezes mais tempo de geração que o LLM cru, em razão de seu raciocínio em múltiplas etapas e dos ciclos de refinamento iterativo. O estudo de ablação serve para entender o papel de cada parte do *RulePilot*. A ideia é simples: tira-se uma parte de cada vez para ver o quanto o resultado piora, assim se descobre para quê serve cada uma. O *RulePilot* tem duas partes principais: a primeira é a representação intermediária (IR), que ajuda a montar a regra com a sintaxe certa; quando ela é removida, o produto é uma regra com mais erros de escrita e de difícil leitura – a nota média cai de 0,55 para 0,44. A segunda parte é o raciocínio em cadeia com revisão (*Chain-of-Thought + Reflection* (CoT-R)) – ela ajuda a regra a ter sentido do começo ao fim. Quando se remove este bloco de raciocínio em

¹ BLEU (Papineni *et al.*, 2002), ROUGE (Lin, 2004) e METEOR (Banerjee; Lavie, 2005) são métricas automáticas que medem a similaridade entre um texto gerado e uma referência pela sobreposição de n-gramas. BLEU prioriza precisão e ROUGE prioriza revocação; METEOR acrescenta sinônimos, radicais e ordem das palavras. Valores mais altos indicam maior alinhamento textual.

cadeia e reflexão, o que mais piora é a consistência lógica e a cobertura de condições. A nota média cai para 0,41. Quando as duas partes são removidas juntas, o resultado é o pior de todos: a nota cai para 0,35. Ou seja, as duas se completam – cada uma cuida de um problema diferente. No fim, os autores testaram se o *RulePilot* consegue traduzir regras de uma plataforma para outra. Em regras simples, ele acertou tudo (nota 1,000). Em regras mais complicadas, que juntam dados de várias fontes, acertou quase tudo (nota 0,919). Isso mostra que dá para migrar regras de um sistema para outro sem grandes perdas. Por fim, os autores avaliaram a conversão entre linguagens SIEM, e o *RulePilot* atingiu nota 1,000 em , nas regras de agregação e de listagem. Já nas regras com junções de várias fontes, que são mais complexas, o foi de 0,919. Esse resultado mostra que a migração entre plataformas SIEM diferentes é tecnicamente viável.

3.3 GRIDAI

O estudo de Li *et al.* (2025) (ainda apenas disponível como pré-print) propõe o GRIDAI, um *framework* de ponta a ponta para geração e reparo automatizado de regras de detecção de intrusão em sistemas baseados em assinaturas, como o e . São citados alguns trabalhos anteriores que buscaram automatizar a criação dessas regras, como AutoCombo (Du; Hu; Hewlett, 2021) e Syrius (Alcantara *et al.*, 2021); entretanto, Li *et al.* (2025) argumentam que limitação central deles é que todos geram uma regra nova para cada amostra de ataque recebida e precisam de tráfego de rede normal como referência, sem nunca verificar se uma regra já existente poderia simplesmente ser atualizada. Com o tempo, isso gera um acúmulo de regras redundantes e um volume crescente de alertas duplicados, dificultando a manutenção do sistema. O GRIDAI propõe uma lógica diferente: antes de agir, ele avalia a natureza de cada amostra. Se o ataque for inédito, cria uma nova regra; se for uma variação de algo já conhecido, atualiza a regra existente para que ela passe a cobrir também esse novo caso. A arquitetura é composta por quatro módulos especializados coordenados por uma lógica central. O primeiro módulo (*Relation-Assess Agent*) funciona em dois passos: primeiro testa a amostra diretamente no motor de detecção e, somente se nenhum alerta for gerado, consulta o modelo de linguagem para decidir o que fazer. O segundo (*New-Rule-Generation Agent*) cria regras novas analisando o conteúdo do ataque em etapas, gerando e testando múltiplos candidatos. Já o terceiro (*Existing-Rule-Repair Agent*), ajusta regras existentes e garante que a versão modificada continue funcionando tanto para o ataque original quanto para a nova variante. E o quarto (*Memory-Update Agent*) registra todas as alterações. Para lidar com erros gerados pelos modelos de linguagem, cada regra produzida é validada em tempo real no motor de detecção; se as falhas ultrapassarem um limite pré-definido, o sistema desfaz a operação e tenta novamente.

Os experimentos foram realizados em dois conjuntos de dados – um sintético (com sete tipos de ataques) e um real (com quarenta e três tipos), além de usar o -4.1 como modelo principal. O sistema também foi testado com outros quatro modelos de linguagem (-4o, -3.5-turbo, -4-Plus e -4-Flash), demonstrando que o GRIDAI tem um bom funcionamento na utilização de

modelos distintos. Quando na tarefa de identificar corretamente o que fazer com cada amostra, o sistema acertou 100% das decisões no conjunto sintético e 96,9% dos casos de reparo no conjunto real; a acurácia para geração de novas regras foi de 60,5%, em virtude de algumas regras do conjunto de referência compartilharem características muito similares entre si. Frente aos três trabalhos tomados como base de comparação por Li *et al.* (2025), o GRIDAI detectou 100% dos ataques no conjunto sintético e 95,4% no conjunto real, sem gerar nenhum falso alarme. O experimento de ablação, em que os autores removeram individualmente cada um dos principais mecanismos do sistema para medir o impacto de cada componente no desempenho final, mostrou que sem o mecanismo de reparo, a taxa de alertas duplicados sobe de 12,7% para 66,8%, confirmando que reaproveitar e atualizar regras existentes é muito mais eficiente do que criar novas regras a cada variante detectada.

3.4 Aprendizado de máquina aplicado a SIEMs e análise de logs

A aplicação de aprendizado de máquina para análise automatizada de logs e a identificação de ameaças foi explorada de forma convergente por Shah *et al.* (2022) e Nurusheva *et al.* (2024). Ambos trabalhos partem da constatação de que o volume e a heterogeneidade dos logs gerados por sistemas modernos inviabilizam a inspeção manual, e as abordagens tradicionais baseadas somente em regras estáticas falham na identificação de ataques novos ou ofuscados². Para superar essa limitação, os dois estudos recorrem a técnicas de ML, embora com metodologias distintas – empírico-quantitativa com foco em agrupamento no primeiro trabalho e com foco arquitetural e demonstrativo no segundo.

Shah *et al.* (2022) propõe um modelo que enfrenta o desafio da ausência de dados rotulados, problema recorrente em cenários reais de monitoramento. A metodologia inicia com pré-processamento dos registros, mantendo apenas o conteúdo textual das mensagens. Em seguida, os autores aplicam a vetorização TF-IDF e executam o algoritmo de *clustering* não-supervisionado *K-means*. A partir dos agrupamentos obtidos, os pesquisadores rotularam os registros e geraram um conjunto de dados para treinar um classificador supervisionado *Naive Bayes*. Avaliado sobre o *dataset BlueGene/L* (BGL), o modelo obtido alcançou acurácia média de 98% na predição de eventos anômalos, demonstrando viabilidade para ambientes que exigem triagem rápida.

Já Nurusheva *et al.* (2024), propõem a integração direta de algoritmos com ML supervisionado a um sistema SIEM. A validação foi conduzida em uma arquitetura construída sobre a pilha ELK, com cenários de ciberataques simulados na aplicação propositalmente vulnerável chamada *Damn Vulnerable Web Application*, do inglês *Damn Vulnerable Web Application* (DVWA). Os autores treinaram modelos clássicos – *Support Vector Machine*, *Random Forest*, *Decision Tree* e *AdaBoost*, além de uma Rede Neural Convolucional, do inglês *Convolutional*

² Ofuscação é a técnica de mascarar a aparência de um código, comando ou tráfego de rede para torná-lo incompreensível aos sistemas de detecção e analistas, sem alterar a função executável original.

Neural Network (CNN) adaptada à classificação de eventos de segurança, utilizando amostras de ataques baseadas no *framework*. Como resultado, relataram ganhos operacionais: a correlação de eventos reduziu falsos positivos e a automação da resposta diminuiu a carga sobre os analistas de segurança. Diferentemente do estudo anterior, o trabalho de Nurusheva *et al.* (2024) possui caráter predominantemente descritivo, com foco na arquitetura operacional, em detrimento de uma avaliação baseada em métricas comparativas rigorosas.

Ao analisar conjuntamente as duas pesquisas, vê-se um dilema prático na modernização e evolução da segurança cibernética: o equilíbrio entre o refinamento algorítmico isolado e a aplicabilidade sistêmica. Enquanto a abordagem de Shah *et al.* (2022) comprova que é possível contornar a escassez de dados rotulados por meio de *pipelines* híbridos de aprendizado de máquina, alcançando alta acurácia em ambientes controlados, a proposta de Nurusheva *et al.* (2024) evidencia que o verdadeiro desafio está na orquestração contínua desses modelos dentro de ecossistemas corporativos reais. Portanto, a consolidação do uso de aprendizado de máquina na triagem de eventos de segurança permanece condicionada à superação dessa fronteira: trabalhos como o de Shah *et al.* (2022) demonstram a viabilidade técnica do componente preditivo, enquanto Nurusheva *et al.* (2024) ilustram a integração operacional desses componentes em SIEMs – mas raramente os dois aspectos são tratados de forma articulada em um único estudo.

3.5 Posicionamento do Trabalho Proposto

Os trabalhos analisados nas seções anteriores expõe duas vertentes que se complementam na pesquisa em automação da detecção de ameaças. A primeira vertente se concentra em técnicas de aprendizado de máquina aplicadas à triagem e classificação de *logs* de segurança, sem produzir artefatos sintáticos diretamente acionáveis em sistemas SIEM. Nesta vertente encontram-se Shah *et al.* (2022) e Nurusheva *et al.* (2024), que combinam métodos supervisionados e não supervisionados para reduzir a carga de alertas em ambientes corporativos. A segunda vertente se concentra na geração automatizada das próprias regras de detecção, valendo-se das capacidades generativas dos LLMs, e representa uma evolução natural da primeira: uma vez consolidada a aplicabilidade de aprendizado de máquina e inteligência artificial na análise de eventos de segurança, abre-se caminho para que essas mesmas técnicas atuem não apenas na classificação dos eventos, mas também na produção dos próprios mecanismos de detecção. Nesta segunda vertente está o GRIDAI ((Li *et al.*, 2025)), voltado para regras de sistemas de detecção de intrusão de rede; o *RulePilot* ((Wang *et al.*, 2026)), voltado para regras nativas das linguagens SPL e KQL; e o *LLMCloudHunter* ((Schwartz *et al.*, 2025)), voltado para regras *Sigma* em ambientes de nuvem. O presente trabalho melhor se insere na segunda vertente, compartilhando com Schwartz *et al.* (2025) o objetivo de geração de regras *Sigma*.

A análise comparativa dos estudos revela três observações fundamentais para o posicionamento da presente pesquisa. Num primeiro momento, dentre os cinco trabalhos analisados, apenas o *LLMCloudHunter* de Schwartz *et al.* (2025) compartilha exatamente o mesmo formato-alvo de saída (regras *Sigma*). No entanto, mesmo neste caso, há diferenças relevantes de escopo: o *LLMCloudHunter* opera sobre relatórios OSCTI como entrada e produz regras específicas à detecção em ambientes de nuvem AWS, enquanto o presente trabalho opera a partir de registros CVE estruturados e notícias de vulnerabilidade, visando a produção de regras *Sigma* não restritas a um provedor de nuvem específico. Em segundo, os trabalhos voltados a outras linguagens de regra – Li *et al.* (2025) e Wang *et al.* (2026), não competem diretamente no domínio *Sigma*, mas oferecem precedentes arquiteturais relevantes para a estruturação do agente proposto neste trabalho. O uso de validação iterativa em motor de detecção real ((Li *et al.*, 2025)) e o uso de raciocínio em cadeia com reflexão ((Wang *et al.*, 2026)) são padrões metodológicos que se aproximam da arquitetura empregada nesta pesquisa, ainda que a aplicação tenha contextos sintáticos distintos. Em terceiro, os trabalhos da primeira vertente – Shah *et al.* (2022) e Nurusheva *et al.* (2024), embora não produzam regras como saída, são fundamentais para contextualizar a presente pesquisa: comprovam o quão útil é o uso de técnicas de aprendizado de máquina e inteligência artificial na detecção de ameaças em sistemas SIEM, fundamentando a premissa de que tais técnicas, hoje já consolidadas, estendem-se para tarefas sofisticadas como a geração automatizada de regras de detecção.

Há ainda uma divergência teórica interessante entre os trabalhos analisados, que merece saliência. Wang *et al.* (2026) argumentam que o formato *Sigma* é insuficiente para a detecção de ataques sofisticados que envolvem sequências de eventos, por restringir-se sobretudo à correspondência de padrões por expressões regulares, criticando trabalhos que adotam *Sigma* como saída – citando diretamente o *LLMCloudHunter*. O presente trabalho optou por manter o foco no formato *Sigma* em decorrência de sua portabilidade entre múltiplos desenvolvedores de SIEMs, da existência de um repositório público bem consolidado de regras (o) que serve como base para o mecanismo de RAG, e da posição do formato como padrão aberto mantido pela comunidade de *detection engineering*. Essa escolha posiciona o trabalho aqui proposto em alinhamento com Schwartz *et al.* (2025), mas com clara consciência das limitações expressivas do formato apontadas por Wang *et al.* (2026). Vale ressaltar igualmente que a própria arquitetura proposta para o agente desenvolvido neste trabalho – fundamentada no uso de RAG sobre o repositório , pode apresentar limitações na tarefa específica de geração de regras *Sigma*, conforme indícios observados ao longo do desenvolvimento. Portanto, investigação empírica conduzida neste trabalho de conclusão de curso pretende vislumbrar o quanto essa estratégia se sustenta frente às alternativas avaliadas, contribuindo para o esclarecimento dessa questão metodológica.

Ante o exposto, dessa análise emerge o recorte ocupado pelo presente trabalho. Enquanto o *LLMCloudHunter* demonstra a viabilidade técnica da geração de regras *Sigma* via LLM em um domínio específico, e enquanto GRIDAI e *RulePilot* exploram arquiteturas agênti-

cas em outras linguagens de regra, permanece em aberto a questão metodológica de quanto a complexidade arquitetural – desde a invocação direta de uma LLM em formato até a orquestração de um agente autônomo com RAG e validação iterativa, efetivamente contribui para a qualidade das regras *Sigma* geradas a partir de fontes públicas de inteligência sobre ameaças. O estudo aqui exposto investiga essa questão por meio da comparação controlada entre três estratégias incrementais — , engenharia de *prompt* e agente autônomo com RAG – submetidas a teste estatístico de hipóteses. Diferentemente dos cinco trabalhos analisados, que tipicamente comparam uma única abordagem proposta contra LLMs autônomos ou métodos baseados em mineração de dados, o presente trabalho se propõe a quantificar empiricamente o ganho marginal proporcionado por cada camada de complexidade arquitetural na tarefa de geração de regras *Sigma*, fornecendo subsídios para decisões de *design* em trabalhos futuros na linha de pesquisa.

4 MATERIAIS E MÉTODOS

Este capítulo descreve os materiais e os métodos que serão utilizados para conduzir esta pesquisa. Apresentar-se-ão, o conjunto de ferramentas e bibliotecas empregadas no desenvolvimento; a base de dados de regras *Sigma* utilizada como referência, juntamente com seu particionamento em base de conhecimento e base de teste; as três estratégias de geração de regras que constituem o objeto de comparação deste trabalho — geração, engenharia de *prompt* e agente autônomo com RAG; as métricas de avaliação adotadas; e, por fim, a formulação das hipóteses e o procedimento de análise estatística.

4.1 Materiais

Conforme apresentado no Quadro 4, para o desenvolvimento do trabalho se utilizou dois equipamentos. O *laptop*, com GPU integrada e recursos de memória mais limitados, foi utilizado no início do trabalho em virtude de problemas técnicos com o *desktop*. Graças a este incidente, a fase de prototipação do agente se deu com o modelo *Qwen 2.5 1.5B*, que demanda menos computacionalmente e permitiu iterar rapidamente sobre a arquitetura. O *desktop*, equipado com GPU NVIDIA RTX 3060 e 12 GB de VRAM dedicada, foi empregado na execução dos experimentos finais e na geração das regras *Sigma* com o modelo *Llama 3.1 8B*.

Os materiais apresentados no 1, atendem a duas finalidades distintas. As LLMs comerciais *ChatGPT*, *Gemini*, *Claude*, *Microsoft Copilot* e *DeepSeek* foram utilizadas exclusivamente nas duas primeiras fases do experimento (geração *zero-shot* e engenharia de *prompt*), em que cada modelo recebe o mesmo conjunto de entradas e gera regras *Sigma* de forma independente, permitindo a comparação entre diferentes provedores. Já os modelos de código aberto (*Llama 3.1* e *Qwen 2.5*), juntamente com os *frameworks* *LangChain*, *LangGraph* e *ChromaDB*, constituem a infraestrutura do agente autônomo desenvolvido na terceira fase, sendo responsáveis pela geração, recuperação de contexto via RAG, e validação iterativa das regras. Os modelos de *embeddings* e *re-ranking* (*bge-small-en-v1.5* e *ms-marco-MiniLM-L-12-v2*) são componentes internos do agente, utilizados na etapa de recuperação e não participam diretamente da geração de texto.

4.1.1 Base de Dados

A fundação da base de dados do trabalho é o repositório oficial *SigmaHQ*. No entanto, apenas cinco subdiretórios interessam, pois são elas que contém as regras-alvo:

- *rules*;
- *rules-compliance*;

Quadro 1 – Modelos de linguagem utilizados no trabalho.

Ferramenta	Versão	Disponível em	Finalidade
<i>ChatGPT</i>	—	https://chat.openai.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
<i>Gemini</i>	—	https://gemini.google.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
<i>Claude</i>	—	https://claude.ai/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
<i>Microsoft Copilot</i>	—	https://copilot.microsoft.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
<i>DeepSeek</i>	—	https://chat.deepseek.com/	LLM comercial utilizada nas fases de geração <i>zero-shot</i> e engenharia de <i>prompt</i> .
<i>Llama 3.1 Instruct</i>	8B	https://ollama.com/library/llama3.1	LLM de código aberto utilizada como geradora de regras <i>Sigma</i> nas três estratégias avaliadas e como modelo principal do agente autônomo.
<i>Qwen 2.5</i>	1.5B	https://ollama.com/library/qwen2.5	LLM de código aberto; pouco consumo de memória, utilizada no início da modelagem do agente por motivos de restrições de <i>hardware</i> ; usada no destilador de texto (nó 3).
<i>Qwen 2.5</i>	14B	https://ollama.com/library/qwen2.5:14b	LLM com mais parâmetros usada como juiz das regras no nó 6.
<i>bge-small-en-v1.5</i>	—	https://huggingface.co/BAAI/bge-small-en-v1.5	Modelo de <i>embeddings</i> (384 dimensões) utilizado para vetorização das regras <i>Sigma</i> e para o cálculo da similaridade semântica.
<i>bge-reranker-base</i>	—	https://huggingface.co/BAAI/bge-reranker-base	Modelo <i>cross-encoder</i> utilizado na etapa de <i>re-ranking</i> , refinando o ordenamento dos candidatos recuperados por similaridade.

Fonte: Autoria própria (2026).

- *rules-emerging-threats*;
- *rules-placeholder*; e
- *rules-threat-hunting*.

Esta distinção é feita no código, que poderá ser conferido no repositório do *Github* agente-autonomo-rag-regras-sigma¹ — mais explicações na Seção 4.2.1.

¹ Disponível em: www.github.com/daninotagente-autonomo-rag-regras-sigma.git – **incompleto, será organizado**

Quadro 2 – *Frameworks*, banco vetorial e ambiente de desenvolvimento.

Ferramenta	Versão	Disponível em	Finalidade
<i>langchain-core</i>	1.4.1	https://pypi.org/project/langchain-core/	Núcleo do <i>framework</i> LangChain; fornece as abstrações de mensagens utilizadas pelo gerador.
<i>langchain-community</i>	0.4.2	https://pypi.org/project/langchain-community/	Integrações comunitárias; fornece os <i>loaders</i> de documentos utilizados na construção do banco vetorial.
<i>langchain-ollama</i>	1.1.0	https://pypi.org/project/langchain-ollama/	Integração entre o LangChain e o servidor Ollama; fornece a classe <i>ChatOllama</i> .
<i>langchain-chroma</i>	1.1.0	https://pypi.org/project/langchain-chroma/	Integração entre o LangChain e o ChromaDB, utilizada pelo mecanismo de RAG.
<i>langchain-huggingface</i>	1.2.2	https://pypi.org/project/langchain-huggingface/	Integração entre o LangChain e a Hugging Face; carrega o modelo de <i>embeddings</i> .
<i>langgraph</i>	1.2.4	https://langchain-ai.github.io/langgraph/	Biblioteca utilizada para modelar o agente como uma máquina de estados, com nós, arestas condicionais e estado compartilhado.
<i>sentence-transformers</i>	5.5.1	https://sbert.net/	Biblioteca que provê as classes utilizadas para carregar os modelos de <i>embeddings</i> e o <i>cross-encoder</i> .
Ollama	—	https://ollama.com/	Servidor local para inferência de LLMs de código aberto, permitindo executar o modelo gerador em <i>hardware</i> próprio.
<i>chromadb</i>	1.5.9	https://www.trychroma.com/	Banco de dados vetorial utilizado para persistir os <i>embeddings</i> das regras <i>Sigma</i> e suportar a recuperação por similaridade.
<i>Visual Studio Code</i>		https://code.visualstudio.com/	Ambiente de desenvolvimento integrado utilizado para a implementação.

Fonte: Autoria própria (2026).

Para a elaboração das avaliações a partir das gerações das regras, optou-se por uma base de teste oficial (*test_cases*) composta por 50 regras oficiais e um conjunto de teste extra com outras 10 regras, este último selecionado para testes repetitivos de aferição durante o desenvolvimento do agente. Decidiu-se por retirar, ao todo, 85% de regras do repositório *Sigma*² para uso no trabalho. Sendo assim, para o desenvolvimento do agente com RAG, que requer uma base de conhecimento (*rag_knowledge*), foram destinados 83,62% das regras selecionadas (Tabela 5).

Como já citado anteriormente, para confrontar a qualidade das regras geradas pelo agente desenvolvido, empregou-se a avaliação de cada regra do conjunto *test_cases* por cinco

² Disponível em www.github.com/SigmaHQ/sigma

Quadro 3 – Bibliotecas de validação, coleta de dados e análise estatística.

Ferramenta	Versão	Disponível em	Finalidade
Python	3.12.3	https://www.python.org/	Linguagem utilizada para implementar o agente, os <i>scripts</i> de geração de <i>datasets</i> e os procedimentos de avaliação.
<i>pySigma</i>	1.3.3	https://github.com/SigmaHQ/pySigma	Biblioteca oficial mantida pela SigmaHQ, utilizada na validação semântica das regras geradas.
<i>PyYAML</i>	6.0.3	https://pyyaml.org/	Biblioteca para análise sintática (<i>parsing</i>) de YAML, utilizada na validação de primeiro nível das regras geradas.
<i>requests</i>	2.34.2	https://requests.readthedocs.io/	Biblioteca utilizada para requisições HTTP a fontes públicas de inteligência sobre ameaças.
<i>beautifulsoup4</i>	4.15.0	https://www.crummy.com/software/BeautifulSoup/	Biblioteca utilizada para <i>parsing</i> de HTML das páginas de referência recuperadas pelo agente.
<i>ddgs</i>	9.14.4	https://pypi.org/project/ddgs/	Biblioteca utilizada para busca <i>web</i> complementar quando as referências diretas são insuficientes.

Fonte: Autoria própria (2026).

Quadro 4 – Hardware utilizado no desenvolvimento do trabalho.

Componente	Desktop (principal)	Laptop (prototipação)
Modelo	ASUS TUF H310M-PLUS GAMING/BR	Lenovo IdeaPad 3 15ALC6
Processador	Intel Core i7-9700K @ 3.60 GHz (8 núcleos)	AMD Ryzen 7 5700U @ 1.80 GHz (8 núcleos / 16 threads)
Memória RAM	16 GB DDR4 2133 MHz (2×8 GB)	12 GB DDR4 3200 MHz (8 GB + 4 GB)
GPU	NVIDIA GeForce RTX 3060 (12 GB GDDR6 dedicados)	AMD Radeon Lucienne (integrada, sem VRAM dedicada)
Armazenamento	SSD Kingston SA400S3 480 GB (SATA)	SSD NVMe SSSTC 512 GB
Sistema operacional	Linux Ubuntu 24.04 LTS	Linux Ubuntu 24.04 LTS

Fonte: Autoria própria (2026).

LLMs comerciais. Este arranjo de regras é segmentado em quatro grupos, sendo o primeiro composto pelas 50 regras originais do repositório oficial *SigmaHQ*, que funcionam como referência para todas as comparações subsequentes. O segundo grupo é composto pelas 250 regras-produto da replicação das cinco LLMs comerciais, utilizando *prompts zero-shot*. No grupo três, o procedimento foi repetido nas mesmas cinco LLMs, porém com *prompts* elaborados via *prompt engineering*, resultando em mais 250 regras. Por fim, o quarto grupo é composto pelas 50 regras geradas pelo agente autônomo.

Quadro 5 – Composição das bases de dados utilizadas no trabalho.

Conjunto	Quantidade	% do <i>SigmaHQ</i>	Origem
Subconjuntos derivados do repositório <i>SigmaHQ</i> válidas (3.748 regras)			
Base de teste (<i>test_cases</i>)	50	1,34%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório <i>SigmaHQ</i> .
Base de teste estendida (<i>extra_test_cases</i>)	10	0,27%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório <i>SigmaHQ</i> .
Base de conhecimento RAG (<i>rag_knowledge</i>)	3.134	83,62%	Selecionada por amostragem estratificada <i>round-robin</i> a partir do repositório <i>SigmaHQ</i>
Subtotal selecionado	3.194	85,2%	—
Regras não utilizadas	554	14,8%	Restante do repositório, não incluído nos subconjuntos.
Regras geradas pelas LLMs comerciais (a partir de <i>test_cases</i>)			
Subconjunto	Quantidade total	Quantidade pós seleção	—
Regras geradas com prompt <i>zero-shot</i>	250	50	50 regras da <i>test_cases</i> replicadas por 5 LLMs comerciais.
Regras geradas com engenharia de <i>prompt</i>	250	50	50 regras da <i>test_cases</i> replicadas por 5 LLMs comerciais.
Regras geradas pelo agente autônomo	50	50	50 regras da <i>test_cases</i> processadas pelo agente <i>LangGraph</i> .
Subtotal de regras geradas	550	150	—

Fonte: Autoria própria (2026).

Para as avaliações será realizada uma filtragem das melhores regras do grupo 2 e do grupo 3, utilizando a técnica de *LLM-as-a-judge*, o que resultará em 150 regras geradas no total, como pode ser visto na Tabela 5. Os métodos serão explicados na **seção X**.

4.2 Métodos

A seguir serão apresentadas as etapas que compõem o método utilizado nesta pesquisa, detalhando cada fase do processo, desde a seleção e divisão do acervo de regras *Sigma*, o desenvolvimento do agente proposto, a composição comparativa entre as estratégias de geração investigadas e o protocolo de avaliação que será aplicado às regras geradas.

4.2.1 Seleção dos dados

A separação dos dados em conjuntos de treino e teste é um requisito metodológico fundamental para garantir a validade científica da comparação entre os quatro estágios desta pesquisa, em particular do Estágio 4. Em sistemas baseados em RAG, o risco de *data leakage*

— definido como “a introdução não intencional de informação preditiva sobre o alvo proveniente do processo de coleta, agregação ou preparação dos dados” por Kaufman *et al.* (2012), pode se manifestar de forma particularmente traiçoeira: caso a mesma regra figure simultaneamente na base recuperável pelo agente e no gabarito contra o qual sua saída será comparada (regras originais), o agente poderia, durante a etapa de *retrieval*, recuperar a própria regra de referência como contexto, invalidando as métricas de similaridade e produzindo uma superestimação artificial em seu desempenho. Esse problema é amplamente reconhecido na literatura como uma das causas mais frequentes de resultados inflados em *pipelines* de ML (Kaufman *et al.*, 2012).

Para evitar o *data leakage*, adotou-se a estratégia de bases disjuntas por construção: as regras utilizadas como gabarito de teste (conjuntos `test_cases` e `extra_test_cases`) foram fisicamente removidas do conjunto de regras disponibilizado ao RAG do agente antes da seleção deste último (`rag_knowledge`), de modo que nenhuma regra do teste pode ser recuperada como exemplo durante a geração, na execução do agente. A divisão foi implementada por meio do *script* `gerador_datasets_unificado.py`³, desenvolvido especificamente para este trabalho. O *script* funciona assim:

O algoritmo opera de forma determinística, com a semente do gerador pseudoaleatório fixada em 42 (`random.seed(42)`), garantindo a reprodutibilidade dos experimentos: qualquer execução posterior do *script* produz exatamente a mesma partição dos dados. A varredura se restringe aos cinco subdiretórios do repositório oficial *SigmaHQ* que concentram as regras de produção (seção 4.1.1), excluindo-se de forma deliberada o diretório *deprecated* (já que contém regras descontinuadas), bem como pastas de exemplos, documentação e ferramentas auxiliares contidas no repositório.

A seleção das regras foi feita por amostragem estratificada balanceada com base no *logsource*, e não por amostragem aleatória simples: a partir de cada regra do repositório, o algoritmo deriva uma assinatura que consiste na tripla `categoria_produto_serviço`, oriunda do campo `logsource` (por exemplo, `process_creation_windows` ou `network_connection_linux`), e então agrupa as regras conforme estas categorias. Em seguida, a seleção acontece via *round-robin*, onde a ordem das categorias é inicialmente embaralhada, para eliminar algum viés de ordenação do dicionário; o algoritmo, então, percorre ciclicamente as categorias embaralhadas e, a cada passo, sorteia-se uma regra aleatória pertencente à categoria vigente, até se atingir-se a quantidade desejada (50 para o `test_cases`, por exemplo). Esse procedimento garante que os conjuntos resultantes contemplem uma maior diversidade possível de fontes de *log* dentro do conjunto amostral, com o objetivo de evitar a sobre-representação de categorias numerosas (como *Windows process creation*) que ocorreria sob amostragem aleatória simples — vale ressaltar que a aparição de mais regras *Windows* é justificável sob a lógica de que existem mais regras deste tipo no repositório original *Sigma*.

O processo produz três conjuntos disjuntos, descritos na Tabela 5. O conjunto `test_cases`, composto por 50 regras, funciona como *baseline* de referência (gabarito, padrão-ouro)

³ Disponível em: .

para os quatro estágios. O conjunto *extra_test_cases*, com 10 regras adicionais, serviu para o empreendimento de testes e tomadas de decisões durante a construção do agente. Por fim, o conjunto *rag_knowledge*, correspondente a 83,62% das regras restantes no *pool* disponível, é selecionado também via *round-robin* e constitui a base de conhecimento consultada pelo agente do Estágio 4 (Seção 4.2.2.5) durante a etapa de *retrieval*. As regras remanescentes não são utilizadas em nenhum dos estágios, ficando reservadas como margem para experimentos posteriores.

4.2.2 Estratégias de geração das regras

A avaliação proposta neste trabalho compara quatro estratégias distintas de obtenção de regras *Sigma* e se delineiam em ordem crescente de complexidade arquitetural: (i) a coleta direta de regras escritas por especialistas humanos (Estágio 1, regras de referência); (ii) geração automática por LLMs comerciais sob instrução mínima (*zero-shot*, Estágio 2); (iii) a geração pelas mesmas LLMs sob *prompts* elaborados via *prompt engineering* (Estágio 3); e (iv) a geração por um agente autônomo composto de RAG e ferramentas externas (Estágio 4). Cada estágio é descrito a seguir conforme a estrutura: objetivo, *input* ou entrada, processo e saída, de modo a facilitar a comparação direta entre eles. As regras produzidas pelos Estágios 2 e 3 são posteriormente submetidas a uma filtragem por *LLM-as-a-judge*, conforme detalhado na Seção 4.2.2.4, restando ao final 150 regras geradas para confronto com as 50 regras de referência.

4.2.2.1 Regras originais do *SigmaHQ* (Estágio 1)

O Estágio 1 não constitui propriamente uma estratégia de geração, mas a referência (*baseline*) contra a qual as demais serão comparadas. Seu objetivo é fornecer um conjunto de 50 regras *Sigma* convencionadas como corretas por construção — escritas, revisadas e mantidas pela comunidade especializada do projeto *SigmaHQ*, que servirão como gabarito para todas as métricas de avaliação descritas na Seção 4.2.4. Como entrada, este estágio utiliza diretamente o repositório oficial *SigmaHQ*, conforme descrito na Seção 4.1.1. Não há etapa de geração: o processo consiste exclusivamente na seleção determinística das 50 regras que compõem o conjunto *test_cases*, conduzida pelo *script* `gerador_datasets_unificado.py` segundo o procedimento de amostragem estratificada balanceada por *logsource* apresentado na Seção 4.2.1. A saída deste estágio são as 50 regras armazenadas no diretório `data/test_cases/`, cada qual em um arquivo `.yaml` nomeado segundo a convenção do repositório de origem. Essas regras cumprem dupla função no protocolo experimental: (i) servem de gabarito para o cálculo das métricas de similaridade e correspondência estrutural; e (ii) fornecem as referências externas (campos `references`, `description` e `tags`) que são extraídas e utilizadas como base para os *prompts* dos Estágios 2, 3 e 4.

4.2.2.2 Geração zero-shot com RAGs comerciais (Estágio 2)

O Estágio 2 tem como objetivo reproduzir regras *Sigma* a partir de LLMs comerciais sob instrução mínima, replicando o cenário no qual um analista de segurança pode recorrer a uma LLM de uso geral sem investir em elaboração de *prompts*. O paradigma adotado é o de *zero-shot prompting*, definido por Brown *et al.* (2020) como a ocasião em que “o modelo recebe apenas uma descrição em linguagem natural da tarefa, sem qualquer demonstração ou exemplo”. Trata-se da forma mais simples e direta de interação com uma LLM: o *prompt* contém apenas a instrução do que deve ser feito, cabendo ao modelo inferir, a partir de seu treinamento prévio, como executar a tarefa.

Como entrada deste estágio utiliza-se, para cada uma das 50 regras-alvo do conjunto *test_cases*, um *prompt zero-shot* (nomeado `prompt1.txt` curto e padronizado, contendo apenas o texto “Generate ONE SIGMA rule (<https://sigmahq.io/docs/basics/rules.html>) for this event:” seguido do(s) *link(s)* presentes no campo *references* da regra original — a URL entre parêntesis é uma referência crua para a LLM posicionar-se a respeito de qual regra está se falando (exemplos em Apêndice A). Não são fornecidos exemplos de regras *Sigma* previamente escritas, instruções sobre a estrutura YAML esperada, nem orientações específicas sobre o uso de *tags* MITRE ATT&CK ou definição de `logsource`.

O processo consiste em submeter cada um dos 50 `prompt1` para cinco LLMs comerciais distintas — *ChatGPT*, *Claude*, *DeepSeek*, *Gemini* e *Microsoft Copilot*, totalizando 250 interações independentes. Cada LLM é instruída a gerar uma única regra *Sigma* válida em formato YAML. A diversidade de modelos comerciais foi adotada para evitar que os resultados reflitam particularidades de uma única arquitetura ou política de treinamento. Logo, a saída deste estágio são 250 regras *Sigma*, organizadas em arquivos YAML dentro do diretório de cada caso de teste, junto à respectiva regra original e ao *prompt* utilizado. Essas regras compõem, juntamente com as do Estágio 3, o conjunto candidato à filtragem por *LLM-as-a-judge* descrita na Seção 4.2.2.4.

4.2.2.3 Geração com *prompt engineering* (Estágio 3)

O propósito do Estágio 3 é investigar em que medida a aplicação sistemática de *prompt engineering*, conforme conceituada na Seção 2.4.3, eleva a qualidade das regras geradas pelas mesmas LLMs comerciais do Estágio 2. Ao contrário do paradigma *zero-shot* adotado no Estágio 2, em que a instrução é mínima e o modelo opera apenas com seu conhecimento prévio, no Estágio 3 a instrução é enriquecida com elementos que orientam o modelo quanto ao papel a ser assumido, ao formato esperado da saída e ao contexto técnico da ameaça. Como entrada deste estágio utiliza-se, para cada uma das 50 regras-alvo, um *prompt* construído a partir de um *template* padronizado no qual variam apenas as informações específicas da ameaça. Tal *template* foi elaborado com o auxílio do modelo *Claude* em processo iterativo, enviando a re-

gra original *Sigma* e solicitando o retorno do *prompt2*⁴ correspondente e que incorpora cinco técnicas reconhecidas na literatura (Liu *et al.*, 2023):

- Atribuição de papel: o *prompt* inicia instruindo o modelo a operar como se fosse um “*Senior Threat Detection Engineer*”, engenheiro sênior de detecção de ameaças, em português;
- Restrição de formato de saída: a instrução determina que o modelo produza *exatamente* uma regra *Sigma* válida em YAML;
- Ancoragem por referência: inclui-se a *Uniform Resource Locators (URL)s* da especificação oficial *Sigma*;
- Injeção de contexto estruturado: o conteúdo da requisição é organizado em seções nomeadas — `Threat to detect`, `Target environment` e `References`;
- Enriquecimento de domínio: além das referências externas, o *prompt* fornece descrição textual da ameaça, identificação do produto/serviço alvo e categorização de fonte de *log* esperada.

O processo de replicação se repete como no Estágio 2: cada um dos 50 *prompts* elaborados é submetido manualmente, via interface *web*, às cinco LLMs comerciais escolhidas, totalizando 250 interações independentes. Os Estágios 2 e 3 compartilham deliberadamente os mesmos modelos, regras-alvo e canal de submissão. A única variável que muda entre eles é a estratégia de *prompting*, de modo que as diferenças observadas nos resultados podem ser atribuídas exclusivamente a esse fator. Por fim, a saída deste estágio são 250 regras *Sigma*, armazenadas no mesmo diretório de cada caso de teste paralelo às regras do Estágio 2. Em conjunto com as regras do capítulo anterior, essas regras compõem o universo de 500 candidatos que serão submetidos a um processo de filtragem por *LLM-as-a-judge*.

4.2.2.4 Filtragem

Após a geração dos 500 cenários com as cinco LLMs comerciais, 250 para o *prompt1* e 250 para o *prompt2*, optou-se por filtrar as 50 melhores de cada conjunto. Os cinco LLMs foram os próprios juízes, e para isso foi necessário organizar um teste minimamente cego, alterando o nome das regras para letras de A a E. Para cada cenário (regra original), em cada uma das cinco LLM, o *input* foi: as 5 regras (A.yml, B.yml, C.yml, D.yml e E.yml), a regra original e um *prompt* de instrução:

⁴ Ver exemplo de *prompt2* em Apêndice A.

prompt filtro

```

You are a Senior Threat Detection Engineer.
Five attached files (A, B, C, D, E) are Sigma rule candidates
generated for the same threat.
The reference rule from SigmaHQ and the original prompt are also
attached. Pick the BEST candidate based on:
1. Syntactic validity (well-formed YAML, required fields, follows
Sigma pec);
2. Structural correctness (correct logsource, valid MITRE tags,
appropriate detection approach);
3. Detection quality (covers the threat, avoids
over/underdetection). Output ONLY:
WINNER: <letter>
WHY: <one sentence>

```

Após o modelo devolver a resposta de qual considera melhor, segundo os critérios do *prompt*, os resultados foram anotados manualmente em uma tabela — que está disponível no repositório `agente-autonomo-rag-regras-sigma`⁵. Depois de contabilizar qual LLM recebeu mais votos por regra, obteve-se uma regra vencedora, para todas as regras do Estágio 2 e 3.

A Tabela 19, disponível no Apêndice B, revela o resultado da filtragem por *LLM-as-a-judge*; as regras utilizadas nos experimentos do Capítulo 5 correspondem às geradas pelas LLMs correspondentes.

4.2.2.5 Geração via agente com RAG (Estágio 4)

O quarto e último estágio tem como finalidade gerar regras *Sigma* por meio do agente autônomo desenvolvido neste trabalho, configurando a estratégia de maior complexidade arquitetural dentre as quatro. Diferentemente dos Estágios 2 e 3, que recorrem a LLMs comerciais de grande porte acessadas via interface *web*, no Estágio 4 há o emprego de um modelo de linguagem aberto e de menor porte, executado localmente, cuja saída é apoiada por recuperação de contexto (RAG), consulta a fontes externas de inteligência de ameaças e validação iterativa. A descrição detalhada da arquitetura do agente, de seus componentes e funcionamento interno é apresentada na Seção 4.2.3; nesta seção, descreve-se apenas o papel do agente como uma das estratégias de geração avaliadas.

Neste estágio, se utilizam, como entrada, três variantes de *prompt* para cada uma das 50 regras-alvo, identificadas como *prompt1*, *prompt2* e *prompt3*. As duas primeiras correspon-

⁵ Especificamente em www.github.com/daninot/agente-autonomo-rag-regras-sigma/blob/main/data

dem, respectivamente, ao *prompt zero-shot* do Estágio 2 e ao *prompt* com *prompt engineering* do Estágio 3; a terceira, *prompt3*, é uma versão refinada do *prompt2* em texto corrido, sem divisão em seções nomeadas. A supressão dos cabeçalhos estruturais (*Threat to detect*, *Target environment* e *References*) já é implantada no código: o agente realiza internamente, por meio de seus nós de classificação e enriquecimento, a função de identificar o tipo de entrada, consultar fontes externas e recuperar exemplos relevantes, tornando redundante a estruturação manual exigida das LLMs comerciais nos estágios anteriores. O *prompt3* é, portanto, a configuração principal do agente — desenhado especificamente para suas características arquiteturais, e é a variante adotada na comparação entre os quatro estágios. As variantes *prompt1* e *prompt2* são executadas em paralelo para subsidiar a análise complementar de sensibilidade descrita ao final desta seção. O processo de geração é totalmente automatizado e conduzido pelo *script* `sigma_agent_automatico.py`⁶, que submete cada uma das 50 regras-alvo às três variantes de *prompt*, totalizando 150 execuções independentes do agente. Cada execução produz três: (i) o parecer do juiz semântico (nó 6) sobre essa regra, em formato JSON; (ii) e uma linha em um arquivo CSV centralizado de metadados, contendo informação sobre a configuração da execução, seus resultados quantitativos e ponteiros para os arquivos (i) e (ii). A estrutura completa desse CSV e o uso de cada um de seus campos para as análises subsequentes são descritos no Capítulo 5.

O conjunto de saída deste estágio possui 150 regras *Sigma*, das quais 50 (geradas a partir do *prompt3*) compõem o conjunto oficial do Estágio 4 a ser confrontado com o gabarito de referência. As 100 regras restantes, geradas a partir de *prompt1* e *prompt2*, não participam da comparação entre os quatro estágios, mas são preservadas para a análise de sensibilidade entre os três tipos de *prompts*. Diferentemente dos Estágios 2 e 3, as regras do Estágio 4 não passam pela filtragem *LLM-as-a-judge* entre modelos comerciais, uma vez que o agente produz uma única regra por regra-alvo/*prompt*. Mas vale salientar que em seu fluxo interno, após a etapa de validação sintática, no Nó 6, é incorporado julgamento semântico com *LLM-as-a-judge*, descrito na Seção 4.2.3.7.

Dois eixos de análise são, portanto, articulados a partir do Estágio 4: (i) a comparação entre os quatro estágios, que confronta as 50 regras geradas pelo agente com *prompt3* contra as 100 regras dos demais estágios; e (ii) a análise de sensibilidade do agente ao *prompt*, que examina, mantidos constantes o modelo e a arquitetura, em que medida a variação da formulação da entrada (do *prompt1* ao *prompt3*) afeta a qualidade e a taxa de aprovação das regras geradas. O detalhamento das métricas aplicadas em cada eixo é apresentado na Seção 4.2.4.

⁶ Disponível em www.github.com/daninot/agente-autonomo-rag-regras-sigma

4.2.3 Desenvolvimento do agente

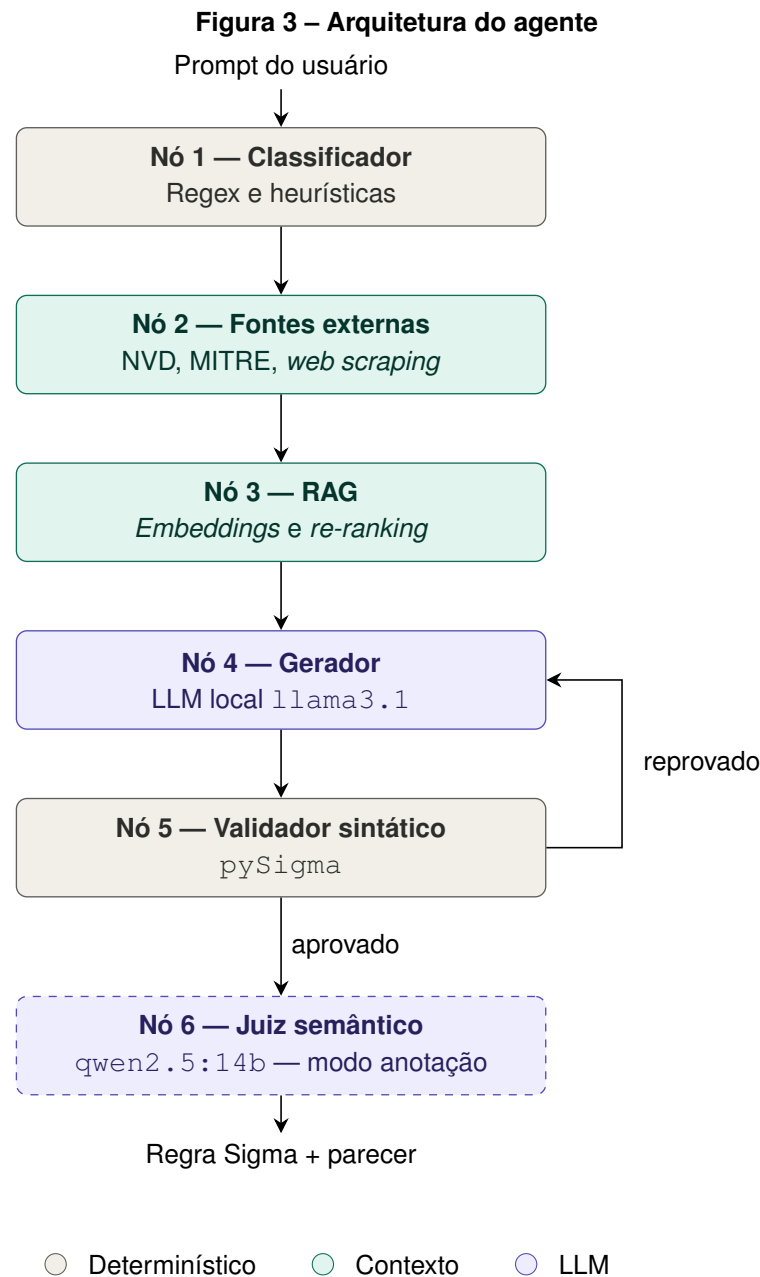
4.2.3.1 Visão geral da arquitetura

O agente desenvolvido neste trabalho é implementado conforme o paradigma de *graph-based agent*, no qual o fluxo de execução é representado por um grafo dirigido cujos nós correspondem a etapas funcionais distintas e as arestas correspondentes determinam a ordem e as condições de transição no fluxo. A implementação utiliza a biblioteca *LangGraph* (LangChain Team, 2024a), parte do ecossistema *LangChain*, que oferece componentes fundamentais para a definição de máquinas de estados executadas sobre um estado compartilhado mutável (*GraphState*). Cada nó recebe o estado, executa sua tarefa, atualiza os campos pertinentes e o devolve ao grafo, que decide o próximo nó a ser executado com base em arestas fixas ou condicionais.

A arquitetura adotada está articulada em torno de seis nós, organizados em um *pipeline* de enriquecimento progressivo da informação. O nó 1, denominado “classificador de entrada”, é determinístico (baseado em expressões regulares e heurísticas, sem invocação de LLM) e interpreta o *prompt* recebido, identifica seu tipo (CVE, *hash* ou texto_livre), extrai URLs de referência e isola a descrição textual da ameaça. O nó 2, denominado “consulta a fontes externas”, enriquece o estado com dados técnicos obtidos junto a APIs de inteligência de ameaças (NVD, MITRE ATT&CK) e, quando necessário, recorre a *web scraping* complementado por uma sub-rotina de destilação por LLM. O nó 3, denominado “RAG”, consulta um banco vetorial contendo exemplos de regras *Sigma* previamente embarcadas, aplicando recuperação semântica por similaridade seguida de *re-ranking* por *cross-encoder*. O nó 4, denominado “geração”, invoca uma LLM local (*Llama 3.1*) que sintetiza a regra *Sigma* em YAML a partir do estado acumulado. O nó 5, denominado “validação sintática”, submete a regra gerada à ferramenta *pySigma* e emite veredito de aprovação ou reprovação. Por fim, o nó 6, denominado “juiz semântico”, opera em modo anotação — registra um parecer qualitativo sobre a regra aprovada, sem capacidade de bloqueá-la ou desencadear nova geração.

A ordem das ações adotadas pelos nós não é arbitrária. A consulta às APIs externas (nó 2) precede deliberadamente a recuperação por RAG (nó 3) para que o material técnico retornado pelas APIs (por exemplo, a descrição oficial de um CVE no NVD) sirva de consulta semântica enriquecida ao banco vetorial, aumentando a relevância dos exemplos recuperados. A geração (nó 4) ocorre apenas após o estado conter as três camadas de contexto (entrada classificada, dados de inteligência e exemplos recuperados) e é seguida imediatamente pela validação sintática (nó 5), de modo que regras malformadas sejam corrigidas antes de qualquer avaliação semântica. O grafo ainda incorpora uma aresta condicional após o nó 5: caso a regra reprove na validação sintática, o controle retorna ao nó 4 para nova tentativa de geração, configurando um laço de auto-correção limitado por um número máximo de tentativas; caso aprove, a regra segue para o nó 6. Trata-se da única ramificação não-linear do grafo.

A Figura 3 apresenta a topologia completa da máquina de estados, com a identificação dos seis nós, suas conexões e a aresta condicional. Os seis nós são executados sequencialmente; a única ramificação não-linear é a aresta condicional após o nó 5, que retorna ao nó 4 em caso de reprovação na validação sintática. O nó 6, com borda tracejada, opera em modo anotação — registra parecer sem bloquear o fluxo.



Fonte: Autoria própria (2026).

As subseções seguintes detalham cada componente: a máquina de estados propriamente dita e o estado compartilhado (??), o RAG (4.2.3.3), as ferramentas externas (4.2.3.3), o laço de validação sintática (4.2.3.5 e 4.2.3.6) e a avaliação semântica complementar (4.2.3.7).

4.2.3.2 Máquina de estados (*LangGraph*), nó 1

Tabela 1 – Campos do *GraphState* agrupados por etapa do *pipeline*

Etapa	Campo	Descrição
Entrada bruta	<code>input_usuario</code>	<i>Prompt input</i>
Classificação (nó 1)	<code>tipo_input</code>	CVE, <i>hash</i> ou texto livre
	<code>termo_busca</code>	CVE ou <i>hash</i> extraído
	<code>url_fornecida</code>	URLs identificadas no <i>prompt</i>
	<code>texto_para_rag</code>	Descrição da ameaça pré-processada
	<code>contexto_pobre</code>	Indicador de contexto textual insuficiente
Enriquecimento (nós 2 e 3)	<code>contexto_api</code>	Dados técnicos de NVD/MITRE
	<code>contexto_rag</code>	Exemplos recuperados do banco vetorial
Geração e validação (nós 4 e 5)	<code>regra_gerada</code>	Regra <i>Sigma</i> em YAML
	<code>erro_validacao</code>	Mensagem do <i>pySigma</i> ou APROVADO
	<code>tentativas</code>	Contador do laço de correção
Avaliação semântica (nó 6)	<code>parecer_juiz</code>	Parecer do juiz em bloco JSON com cabeçalho identificador

Fonte: Autoria própria (2026).

A máquina de estados do agente é construída como um grafo dirigido sobre uma estrutura de dados única e compartilhada, chamada *GraphState*. Trata-se de um dicionário tipado em *Python* (*TypedDict*) que funciona como um “caderno de anotações” visível a todos os nós: cada nó lê os campos de que precisa, executa sua tarefa e devolve apenas os campos que atualizou (LangChain, 2024). Essa organização evita passar informações entre nós por meio de parâmetros explícitos, facilita a manutenção do agente e permite inspecionar todo o seu estado em qualquer momento da execução.

O *GraphState* possui 12 campos, que podem ser agrupados em cinco categorias de acordo com a etapa do *pipeline* em que são preenchidos, conforme a Tabela 1.

As arestas (conexões entre os nós) definem o caminho percorrido pelo estado durante a execução. Como ilustrado na Figura 3, o agente utiliza dois tipos de conexão: conexões fixas, que sempre levam ao próximo nó previamente definido, e uma única conexão condicional, onde o destino depende do conteúdo do estado naquele momento (LangChain, 2024). Cinco conexões fixas ligam, em sequência, o ponto de partida do grafo ao nó 1, e cada nó subsequente ao seguinte, até o nó 5. Após o nó 5, encontra-se a conexão condicional, cuja decisão é tomada por uma função auxiliar chamada `roteador_de_validacao`, descrita a seguir. Por fim, uma última conexão fixa leva o nó 6 ao ponto final do grafo: o produto do agente, a regra *Sigma* (Figura 3).

A função `roteador_de_validacao` segue três regras, verificadas em ordem. Primeira: se o campo `erro_validacao` contém o valor `APROVADO`, o controle segue para o nó 6 e o laço termina com sucesso. Segunda: se o erro persiste, mas o contador `tentativas` já contabiliza 4, o controle também segue para o nó 6, encerrando o laço por esgotamento das tentativas. Entretanto, a regra considerada sintaticamente inválida ainda é gravada em disco (o *script* não a descarta nem ignora) e avaliada pelo juiz semântico, cujo parecer também é salvo para análise posterior. Terceiro: em qualquer outra situação, quando o *pySigma* é executado sobre a regra e detecta um erro, o controle volta ao nó 4 para uma nova tentativa de geração, agora orientada pela mensagem de erro guardada em `erro_validacao`. O limite de quatro tentativas é uma escolha de projeto: alto o suficiente para permitir a convergência em casos recuperáveis, mas baixo o suficiente para impedir laços infinitos diante de *prompts* que não podem ser corrigidos.

Vale destacar que essa conexão condicional é a única ramificação do grafo. Todas as outras decisões internas do agente — qual API consultar no nó 2, quantos exemplos recuperar no nó 3 e qual modelo invocar no nó 4, ocorrem dentro dos próprios nós, sem afetar a estrutura do grafo. Essa separação entre a lógica de fluxo (representada pelo grafo) e a lógica interna (escondida dentro de cada nó) é uma decisão deliberada: ela permite que cada nó seja modificado, substituído ou melhorado de forma independente, sem alterar a arquitetura geral do agente.

4.2.3.3 Ferramentas externas (nó 2)

O nó 2 é responsável por enriquecer o estado do agente com informação técnica obtida de fontes externas. Sua existência atende a duas necessidades complementares. A primeira diz respeito ao conteúdo: identificadores como CVEs e *hashes* são, por si só, apenas códigos; não trazem nenhuma descrição da ameaça que representam. Para que o gerador (nó 4) tenha informação suficiente para escrever uma regra adequada, é preciso traduzir esses identificadores em descrições técnicas, como vetor de ataque, plataformas afetadas, técnicas MITRE ATT&CK associadas e nível de gravidade. A segunda necessidade aparece quando o *prompt* do usuário é pobre em descrição textual, contendo apenas um *link* para um boletim de segurança, por exemplo. Neste caso, o agente precisa de algum mecanismo para reconstruir, a partir do material bruto coletado, uma descrição utilizável da ameaça. E o nó 2 atende às duas necessidades em uma única passagem, organizada em quatro etapas executadas em sequência, conforme o Algoritmo 1.

Algorithm 1 Coleta de contexto técnico (nó 2)

Input: Estado *state* com campos *url_fornecida*, *tipo_input*, *termo_busca*, *contexto_pobre***Output:** Campo *state.contexto_api* atualizado no estado

```

1 trechos  $\leftarrow$  [] cves, cwes  $\leftarrow$  extrair_referencias(state.input_usuario)
   // Etapa 1: URL scraping
2 foreach url  $\in$  state.url_fornecida do
3   | html  $\leftarrow$  HttpGet(url) if request succeeded then
4   |   | texto  $\leftarrow$  extrair_elementos_semanticos(html) adicionar texto to trechos
5   | else
6   |   | palavras  $\leftarrow$  extrair_palavras_do_slug(url) adicionar palavras to trechos
7   | end
8   | atualizar cves, cwes com referências encontradas em url
9 end

   // Etapa 2: APIs estruturadas para cada CVE
10 foreach cve  $\in$  cves do
11 |   adicionar consulta_mitre(cve) to trechos adicionar consulta_nvd(cve) to trechos
12 end
13 if state.tipo_input = hash then
14 |   adicionar consulta_virustotal(state.termo_busca) to trechos // simulada nesta
   |   versão
15 end

   // Etapa 3: busca complementar na web
16 texto  $\leftarrow$  state.termo_busca primeiro não vazio, else state.texto_para_rag adicionar
   SearchDuckDuckGo(texto) to trechos
17 state.contexto_api  $\leftarrow$  concatenar(trechos)

   // Etapa 4: destilação condicional por LLM
18 if state.contexto_pobre and tamanho(state.contexto_api)  $\geq$  LIMIAR then
19 |   descricao  $\leftarrow$  destilar_com_LLM(state.contexto_api) // qwen2.5:7b
20 |   state.contexto_api  $\leftarrow$  descricao || state.contexto_api // prepende
   |   destilacao
21 end
22 return state.contexto_api

```

A primeira etapa é a extração de conteúdo das URLs fornecidas pelo usuário. Para cada URLs listada em *url_fornecida*, o agente envia uma requisição HTTP com um cabeçalho *User-Agent* de navegador comum. *Links* do GitHub no formato *blob* são convertidos automaticamente para *raw.githubusercontent.com*, de modo a obter o conteúdo bruto do arquivo em vez da página renderizada do repositório. O HTML retornado é processado pela biblioteca *BeautifulSoup*, que extrai prioritariamente os elementos mais informativos da página: parágrafos, artigos, cabeçalhos (h1, h3), células de tabela e itens de lista. A escolha por esses elementos considera que documentações de auditoria de *logs* costumam expor os nomes reais dos campos de auditoria em cabeçalhos e tabelas, e esses nomes são exatamente o que o nó

4 precisará para escrever um campo `detection` compatível com a fonte de *log* real. Caso a requisição falhe, há um *fallback* simples: o agente extrai palavras-chave da última parte das URLs (*slug*), preservando algumas pistas sobre a ameaça.

A segunda etapa consulta bases estruturadas de inteligência de ameaças. Para cada CVE detectado no *prompt* ou nas URLss, o agente consulta duas fontes oficiais. A primeira é o repositório *MITRE ATT&CK*, que retorna o vetor de ataque e as técnicas associadas à vulnerabilidade. A segunda é o NVD, que retorna a descrição oficial, as CWEs associadas e a pontuação CVSS. As CWEs identificadas são acumuladas em um conjunto e adicionadas como contexto extra. Quando o *prompt* se refere a um *hash* de arquivo malicioso (`tipo_input = hash`), o fluxo previsto seria a consulta à plataforma *VirusTotal*⁷, mas nesta versão do agente este caminho é simulado com dados fictícios. A plataforma possui API pública e gratuita até certo ponto; ela exige cadastro e geração de uma chave de acesso pessoal, além de impor limites de uso. A decisão de não implementar essa integração se deve à observação empírica de que a grande maioria dos casos das regras Sigma não são classificadas como *hash*, predominando entradas do tipo `cve` e `texto_livre`. Essa limitação está registrada como possibilidade de extensão futura na Seção de trabalhos futuros. A opção de identificação, no entanto, foi mantida para representar uma situação real em que um analista faz o uso do agente e solicita uma regra que contém descrição de *hash*.

A terceira etapa é uma busca complementar na *web* aberta, executada por meio do mecanismo *DuckDuckGo*⁸, com até cinco resultados por consulta. Essa busca não substitui as fontes estruturadas; ela funciona como uma rede de segurança para captar discussões em blogs técnicos, relatórios de empresas de segurança e boletins de segurança de fornecedores, que costumam publicar informações antes das bases oficiais. O termo enviado à busca é o termo isolado (CVE ou *hash*), quando disponível, ou o texto pré-processado pelo nó 1, truncado em 200 caracteres para preservar a qualidade da consulta. A escolha pelo *DuckDuckGo* se deu por dois motivos: não exige chave de API e é compatível com a biblioteca de cliente utilizada no projeto.

A quarta etapa é executada apenas em determinadas condições, atendendo ao caso de *prompts* pobres em descrição textual. Quando o nó 1 sinaliza `contexto_pobre = True` e o material acumulado pelas três etapas anteriores atinge um tamanho mínimo definido pela constante `LIMIAR_MATERIA_PRIMA`, o agente aciona uma sub-rotina de destilação por LLM. Nessa etapa, o material bruto coletado (texto extraído das URLss, descrições do MITRE e do NVD, resultados da busca na *web*) é enviado ao modelo `qwen2.5:7b`, que produz uma descrição resumida da ameaça em linguagem natural. Essa descrição destilada é colocada no topo do campo `contexto_api`, e abaixo o material bruto. Assim, quando o nó 3 ler esse campo, encontrará primeiro a descrição resumida e, em seguida, o material bruto que a originou. Convém assinalar, neste momento, que a decisão de escolher o modelo `textttqwen2.5:7b` para

⁷ Disponível em: www.virustotal.com.

⁸ Disponível em: www.duckduckgo.com.

essa destilação se dá pelo fato de pertencer a uma família distinta da do gerador `Llama3.1`. Essa escolha se deu porque em *pipelines* sequenciais como o aplicado (a saída do destilador alimenta diretamente o gerador), pode acontecer de erros e vieses característicos das etapas iniciais propagarem-se e a acumularem-se nas etapas posteriores (Caselli *et al.*, 2015). O uso do mesmo modelo nas duas etapas poderia amplificar esse risco, uma vez que ambos compartilhariam os mesmos pontos-cegos de treinamento. A escolha por famílias arquiteturalmente distintas (*qwen* e *llama*, treinadas por equipes e em *corpora* diferentes) busca minimizar essa correlação de erros.

Ao final dessas quatro etapas, o campo `contexto_api` contém todo o conteúdo técnico coletado sobre a ameaça, com as descrições estruturadas (MITRE, NVD, destilação) priorizadas sobre o material bruto. Essa priorização permite o nó 3 (RAG), descrito na próxima subseção, utilizar `contexto_api` como base para construir a consulta ao banco vetorial, e blocos estruturados geram *embeddings* de maior qualidade do que texto extraído diretamente de páginas HTML.

4.2.3.4 RAG (nó 3)

O nó 3 implementa a etapa de RAG do agente. Seu propósito específico aqui é entregar ao gerador (nó 4) um pequeno conjunto de regras *Sigma* já existentes, semanticamente próximas da ameaça em questão. Vale salientar que as regras recuperadas (contidas em `rag_knowledge`) não são entregues ao nó 4 como “a resposta correta” para a ameaça em questão; até porque, em geral, não o são. Trata-se de regras de outras ameaças, apenas semanticamente próximas. Elas servem como referência de formatação, contexto semântico e como pista das estruturas YAML que costumam funcionar bem em ameaças similares.

A construção da consulta enviada ao banco vetorial segue uma ordem de prioridade definida por quatro estratégias, escolhidas em função do que está disponível no estado naquele momento. A estratégia preferencial combina o sinal mais informativo do usuário — o termo isolado, como um CVE, ou as seções técnicas extraídas pelo nó 1, com a descrição técnica recuperada das APIs no nó 2. Caso a API não tenha retornado descrição rica, recorre-se apenas às seções técnicas extraídas pelo nó 1. Caso estas também não estejam presentes, se utiliza apenas o termo isolado (CVE ou *hash*). Em último caso, o *prompt* original do usuário é usado como consulta. Esta ordem reflete uma ordem de qualidade: quanto mais rica a consulta enviada ao banco, maior a chance de recuperar exemplos alinhados com a ameaça real.

A recuperação propriamente dita é organizada em um funil de duas etapas, técnica conhecida na literatura como *retrieve-and-rerank* (Nogueira; Cho, 2019). Na primeira etapa, a consulta é convertida em um vetor numérico (*embedding*) pelo modelo `BAAIbge-small-en-v1.5`, da família BGE (Xiao *et al.*, 2023), e comparada por si-

milaridade de cosseno contra todos os vetores armazenados em um banco vetorial ⁹. O banco contém as regras do conjunto *rag_knowledge* (**Seção selecionados**), também convertidas previamente em *embeddings*. Dessa busca ampla resultam vinte candidatos. Na segunda etapa, os vinte candidatos passam por um modelo de *re-ranking* do tipo *cross-encoder* (BAAI/bge-reranker-base), que analisa cada par consulta-candidato de forma conjunta e produz uma pontuação de relevância refinada. Os cinco candidatos com a maior pontuação são selecionados como contexto final e gravados no campo `contexto_rag` do estado.

A escolha pelo funil de duas etapas, ao invés de uma única busca, se dá porque modelos de *embeddings* codificam cada texto de forma independente, produzindo vetores que podem ser pré-calculados e armazenados — isso os torna extremamente rápidos, mas também os faz perder algumas nuances da interação entre a consulta e o documento. Já os modelos *cross-encoder* processam consulta e candidato lado a lado, capturando essas nuances, porém com um custo computacional maior (Nogueira; Cho, 2019). A combinação dos dois aproveita o melhor de cada paradigma: a busca por *embedding* filtra o banco inteiro até vinte candidatos plausíveis em milissegundos, e o *re-ranker* aplica seu poder discriminativo apenas sobre esse pequeno subconjunto.

O nó 3 também executa um diagnóstico de coerência sobre os cinco exemplos selecionados — etapa que não interfere no fluxo do agente, mas registra informação útil para análise posterior. Para cada exemplo, o *script* extrai o campo `logsource` e contabiliza quantas categorias e produtos distintos aparecem entre os cinco. Quando três ou mais categorias diferentes coexistem nos exemplos recuperados, o *script* registra um aviso: nessa situação, há risco de a LLM “misturar” campos de fontes de *log* incompatíveis ao redigir a regra final. Esse diagnóstico não bloqueia a geração, mas serve como sinal de alerta para a análise dos resultados e como evidência empírica da qualidade do *retrieval* em diferentes tipos de entrada.

4.2.3.5 Geração da regra (nó 4)

O nó 4 é o componente que produz a regra *Sigma* propriamente dita. Sua entrada é o estado já enriquecido pelos nós anteriores. O classificador (nó 1) determinou o tipo da entrada e isolou o termo de busca; as ferramentas externas (nó 2) acumularam contexto técnico em `contexto_api`, e o RAG (nó 3) recuperou cinco exemplos de regras reais em `contexto_rag`. Cabe ao nó 4 sintetizar essas informações em um único arquivo YAML que atenda às especificações da plataforma *Sigma*. Para essa tarefa é empregado o modelo `Llama3.1`, executado localmente via *Ollama*, com temperatura ajustada em 0,1 — valor escolhido propositalmente baixo para favorecer respostas determinísticas e reduzir a variabilidade entre execuções.

A invocação da LLM segue a estrutura dual de *prompt*, contendo uma mensagem de sistema (*system prompt*) e uma mensagem de usuário (*user prompt*). A mensagem de sistema é

⁹ O banco vetorial utilizado é o *ChromaDB* (www.trychroma.com), uma das implementações *open-source* disponíveis para essa finalidade.

carregada de um arquivo externo (`sigma_system_prompt.md`¹⁰ e contém instruções permanentes que regem o comportamento do modelo, independentemente da ameaça específica. São elas: estrutura obrigatória do YAML, lista de *tags* válidas para o campo `tags`, regras de uso dos *logsources*, modificadores YAML aceitos pela especificação *Sigma*, padrões proibidos (como o bloco `related:` ou a inserção de *filtros* no campo `condition:`) e diretrizes para nunca inventar campos ou URLs. O conteúdo desse arquivo foi desenvolvido de forma iterativa a partir da análise de erros observados em execuções preliminares sobre o conjunto *extra_test_cases* (Seção 4.2.1) e foi “congelado” antes do início dos experimentos sobre o conjunto *test_cases*, de modo a evitar qualquer contaminação metodológica entre o ajuste do agente e a sua avaliação. A mensagem de usuário, por sua vez, é construída dinamicamente a cada chamada e reúne seis blocos sequenciais:

1. *USER REQUEST*, contendo o *prompt* original do usuário;
2. *USER-PROVIDED REFERENCES*, com a lista exata de URLs autorizadas para o campo `references:` da regra, ou uma instrução explícita para omitir o campo quando nenhuma URL foi fornecida;
3. *FORMATTING TEMPLATE*, com os cinco exemplos recuperados pelo RAG, usados como referência estrutural de formatação;
4. *ADDITIONAL TECHNICAL CONTEXT*, com a informação técnica acumulada pelo nó 2 — MITRE, NVD, busca *web* e, quando aplicável, a descrição destilada da ameaça;
5. *REASONING STEP*, que solicita à LLM a produção de um bloco `thinking...thinking` com raciocínio explícito sobre quatro aspectos críticos da regra a ser gerada — escolha do *logsource*, identificação dos campos de *log*, escolha dos modificadores YAML adequados e mapeamento à técnica MITRE ATT&CK correspondente;
6. *OUTPUT REQUIREMENTS*, com as instruções formais sobre o formato esperado da saída.

A solicitação de um bloco de raciocínio explícito antes da geração do YAML implementa uma forma de *Chain-of-Thought Prompting* (Wei *et al.*, 2022), técnica conhecida por melhorar a qualidade de saídas que envolvem decisões em múltiplas etapas. No caso de regras *Sigma*, essas decisões são interdependentes — a escolha do *logsource* restringe os campos disponíveis, que por sua vez determinam quais modificadores fazem sentido, e o raciocínio explícito força a LLM a deliberar sobre cada etapa antes de se comprometer com o resultado final, em vez de produzir a regra como saída direta.

¹⁰ Apêndice B

Por fim, o nó 4 incorpora um mecanismo de auto-correção orientada por erro. Quando o estado do grafo contém uma mensagem de erro armazenada no campo `erro_validacao` (que ocorre a partir da segunda tentativa de geração, após o validador sintático (nó 5) ter rejeitado a tentativa anterior), o *prompt* de usuário recebe um bloco adicional chamado *ATTENTION*, contendo a mensagem de erro específica e a instrução para corrigi-la. Este acréscimo permite que cada nova geração não seja cega: a LLM recebe *feedback* estruturado sobre o erro anterior e pode focar sua correção naquele problema específico, em vez de regerar a regra do zero. A descrição completa do laço entre os nós 4 e 5 será apresentada na próxima subseção (Seção 4.2.3.6).

4.2.3.6 Validação sintática iterativa e laço de correção (nó 5)

O nó 5 é o ponto em que a regra produzida pelo nó 4 é submetida a um conjunto de verificações. Seu propósito é aprovar regras que cumprem a especificação oficial *Sigma* ou, quando há erros, devolver ao gerador uma mensagem técnica precisa que oriente uma nova tentativa de geração. É a partir deste nó que se forma o laço de auto-correção entre os nós 4 e 5, descrito ao final desta subsubseção.

Antes das verificações propriamente ditas, o nó 5 executa uma etapa de pré-processamento sobre a saída bruta da LLM, projetada a partir de padrões de erro recorrentes observados durante a calibração do agente sobre o conjunto *extra_test_cases* (Seção 4.2.1). Quatro normalizações são aplicadas em sequência:

1. Remoção de eventuais cercas de *markdown* (`" "`) que a LLM possa ter adicionado em torno do YAML;
2. Extração e descarte do bloco `thinking...thinking`, no qual o nó 4 instruiu a LLM a registrar seu raciocínio passo a passo (Seção 4.2.3.5);
3. Tratamento do caso em que a LLM produz mais de um documento YAML separados por `--`, mantendo-se apenas o primeiro;
4. substituição do identificador `id`: da regra por um número único gerado pelo *Python* (*Universally Unique Identifier* (UUID)). Este passo é necessário porque o *system prompt* instrui a LLM que preencha o campo `id`: com um valor temporário qualquer, deixando a geração do identificador real para esta etapa.

Só depois das normalizações o YAML é considerado pronto para validação.

A validação em si é organizada em três etapas progressivas, do mais barato ao mais caro. A primeira etapa utiliza a biblioteca *PyYAML*¹¹ para verificar se o texto é um YAML sintaticamente válido. A segunda etapa avalia a estrutura mínima da regra. Confirma que o resultado é um dicionário, que os três campos obrigatórios da especificação *Sigma* estão presentes

¹¹ Disponível em: www.pyyaml.org.

(`title`, `logsource` e `detection`) e que todas as *tags* declaradas no campo `tags`: pertencem à taxonomia oficial do MITRE ATT&CK (Strom *et al.*, 2018), verificação feita por uma função local que compara cada *tag* contra uma lista de táticas e técnicas conhecidas. A terceira e mais rigorosa etapa utiliza a biblioteca `textitpySigma`¹², mantida pelo *SigmaHQ*, que executa a validação semântica da regra contra a especificação completa da plataforma. A regra é considerada aprovada apenas se passar nas três etapas. Em qualquer falha, uma mensagem de erro específica da etapa em que ocorreu o problema é gravada no campo `erro_validacao` do estado.

O fluxo subsequente é decidido pela função de roteamento que lê o resultado da validação. São três caminhos possíveis: (i) se a regra foi aprovada (`erro_validacao = APROVADO`), o controle segue para o juiz semântico (nó 6), encerrando o laço; (ii) se a regra foi reprovada e o contador `tentativas` já atingiu o limite máximo de 4, o controle também segue para o nó 6, encerrando o laço por exaustão de tentativas; (iii) nas situações restantes (de reprovação), o controle volta para o nó 4 para uma nova tentativa de geração. E esse retorno não é uma simples repetição; como descrito na Seção 4.2.3.5, o nó 4 lê o campo `erro_validacao` e anexa a mensagem de erro ao *prompt*, de modo que a LLM receba *feedback* estruturado sobre o problema específico da tentativa anterior e tente corrigi-lo.

O ciclo formado entre os nós 4 e 5 constitui um mecanismo de auto-correção iterativa orientada por *feedback* externo, padrão amplamente estudado na literatura sob o nome *self-refinement* (Madaan *et al.*, 2023). O caso aqui apresentado difere do mecanismo original em um aspecto importante. No *Self-Refine* clássico, o próprio modelo gera tanto a saída quanto a crítica que a refina; no laço deste agente, o *feedback* vem de um validador externo, determinístico e auditável (o *pySigma*), o que torna o sinal de correção independente das eventuais inseguranças ou vieses da própria LLM. Vale registrar, porém, que esse laço só protege contra erros sintáticos e estruturais. Erros de natureza semântica, como uma regra bem formada mas inadequada à ameaça exigem uma camada adicional de avaliação, papel desempenhado pelo nó (Seção 4.2.3.7).

Por conseguinte, vale uma observação sobre a decisão de encaminhar ao nó 6 as regras que falharam na validação por exaustão de tentativas, em vez de simplesmente descartá-las. Essa escolha permite que toda regra gerada, aprovada ou não, seja salva em disco e analisada pelo juiz semântico, o que viabiliza análise dos modos de falha do agente. Sem esse tratamento, regras reprovadas seriam invisíveis à análise posterior, e a oportunidade de caracterizar empiricamente por que e em que circunstâncias o agente falha seria perdida. O limite de quatro tentativas, por sua vez, é um *trade-off* prático entre tempo de execução e probabilidade de convergência: tentativas adicionais raramente resolvem erros que não foram corrigidos nas primeiras iterações, segundo observações preliminares com `extra_test_cases`.

¹² Disponível em www.github.com/SigmaHQ/pySigma

4.2.3.7 Avaliação semântica por LLL-as-a-judge (nó 6)

O nó 6 implementa uma camada de avaliação complementar à validação sintática do nó 5. Sua razão de existir parte do fato de que o *pySigma* aprova regras que estão *formalmente* corretas, isto é, regras que respeitam a especificação *Sigma* em estrutura, campos obrigatórios, modificadores e operadores. Contudo, ele não tem como avaliar se a regra é adequada à ameaça do cenário: uma regra pode ser sintaticamente impecável e, ainda assim, não fazer sentido contextual. Pode monitorar uma fonte de *logs* errada, usar campos irrelevantes para o ataque descrito, inventar filtros que não têm respaldo na descrição original do usuário, entre outras situações. Captar esse tipo de inadequação, que escapa à validação puramente sintática, é o papel do nó 6.

A solução adotada é a técnica conhecida como LLM-como-juiz (*LLM-as-a-Judge*) (Zheng *et al.*, 2023), em que uma LLM dedicada a essa função recebe a regra, a descrição original da ameaça e a especificação oficial do *Sigma*, e emite um parecer crítico sobre a correção semântica da regra. O modelo escolhido para essa tarefa é o *Qwen2.5:14b*, executado localmente com temperatura zero — valor que garante o máximo de determinismo possível e elimina variabilidade aleatória entre execuções. Duas decisões convergem para escolha do modelo e a primeira é o uso de uma família arquiteturalmente distinta à do gerador (*Llama3.1*), pelos motivos já discutidos na Seção 4.2.3.3 — famílias diferentes reduzem o risco de erros sistemáticos comuns ao gerador e ao auditor. E a segunda decisão é optar por um modelo maior (14 bilhões de parâmetros contra 8 bilhões do gerador), justificada pelo fato de que uma avaliação crítica exige mais capacidade de raciocínio do que geração condicionada.

O *design* do nó 6 opera em modo anotação, vale ressaltar. Isso implica que seu parecer não bloqueia o fluxo do agente nem dispara uma nova rodada de geração, mesmo quando a regra é considerada problemática. Poderiam ser enumeradas mais de uma razão para essa escolha, mas as decisões principais são metodológica e falta de tempo hábil — optou-se que o objetivo do nó 6 seria *medir* a qualidade semântica das regras geradas pelo agente em comparação com o gabarito do *SigmaHQ*, ao invés de corrigi-las em tempo real, pois demandaria mais tempo para testar a qualidade do impacto do julgamento.

Quanto ao parecer ¹³, produto do modo anotação, é estruturado em seis dimensões avaliadas, cada uma classificada como *PASS*, *WARN* ou *FAIL*, com uma justificativa em uma ou duas frases. As dimensões são:

1. *logsource*, que avalia se a fonte de *log* declarada usa corretamente os campos *category*, *product* e *service*, e se os valores escolhidos são válidos;
2. *modifiers*, que avalia se os operadores `|contains|`, `|endswith|` e `|startswith|` foram usados de forma adequada;

¹³ Exemplos disponíveis em Apêndice x

3. *condition*, que avalia se a lógica *booleana* é sintaticamente válida e faz sentido;
4. *structure*, que avalia a presença dos campos obrigatórios da especificação;
5. *semantic_alignment*, que avalia se a lógica de detecção efetivamente captura a ameaça descrita;
6. *invented_filters*, que avalia se a regra adicionou filtros (caminhos ou *hosts* específicos) que não têm respaldo na descrição original da ameaça.

A combinação dessas seis dimensões produz um veredito final com três valores possíveis: *APPROVED* (todas as dimensões PASS), *APPROVED_WITH_WARNINGS* (pelo menos uma WARN, nenhuma FAIL) e *REJECTED* (pelo menos uma FAIL). Um campo adicional de *observações gerais* captura um comentário breve do juiz sobre a qualidade global da regra.

O parecer é solicitado à LLM em formato JSON estruturado e gravado em disco em um arquivo cuja nomenclatura segue o padrão `nome_do_cenário__promptN.parecer.txt`. A extensão `.txt` (em vez de `.json`) é usada para caso o modelo eventualmente produzir um JSON mal-formatado, o conteúdo bruto da resposta é preservado em vez de ser descartado, possibilitando inspeção manual posterior. Cada arquivo contém um cabeçalho identificador (cenário, *prompt*, modelo e temperatura usados) seguido do bloco JSON com o parecer. Esses arquivos são a fonte primária de evidência empírica usada na **Seção de Resultados** para caracterizar os modos de falha do agente.

4.2.4 Métricas de Avaliação

A avaliação do desempenho do agente e a comparação entre os quatro estágios de geração descritos na Seção 4.2.2 se apoiam em métricas determinísticas, métricas semânticas, métricas de similaridade com o gabarito e testes estatísticos para comparação entre estágios. Todas elas têm como fonte primária de dados o registro de metadados produzido automaticamente pelo ambiente de execução do agente, descrito a seguir.

4.2.4.1 Metadados

Durante a execução do agente com os 50 cenários (regras) do conjunto *test_cases*, cada combinação (*cenário*, *promptN*) produz 9 (sem considerar a regra original) objetos salvos no mesmo diretório, conforme descrito na Seção 4.2.2.5. Novamente: considerando os três *prompts*, para cada um, há uma regra *Sigma* gerada (um arquivo `.yaml`), um parecer do juiz semântico (arquivo `.parecer.txt`) e uma linha em um arquivo `.csv`, denominado `metadados_geracao.csv`, que centraliza os dados das 150 execuções. Esta estrutura de dados viabiliza todas as análises subsequentes, visto que cada uma de suas linhas corres-

ponde a uma execução completa do agente, e suas colunas registram os atributos relevantes da execução.

As colunas do registro de metadados podem ser agrupadas em cinco categorias funcionais. A primeira é a identificação, com as colunas `cenario_id` e `prompt_usado`, que identificam a execução de forma única. A segunda é a configuração, com as colunas `tipo_input`, `contexto_pobre` e `usou_web_search`, que registram as decisões tomadas pelos nós 1 e 2 sobre o tratamento da entrada. A terceira é o resultado quantitativo da execução, com as colunas `status` (APROVADO, REPROVADO ou ERRO_EXECUCAO), `tentativas` (número de iterações no laço nó 4 – nó 5), `tempo_total_s` e `erro_validacao_final`. A quarta é a avaliação semântica do nó 6, com a coluna `veredito_juiz` e seis colunas `juiz_dimensao`, uma para cada dimensão avaliada, além da coluna `motivo_reprovacao`. A quinta é o rastreo dos arquivos gerados, com as colunas `regra_referencia`, `arquivo_regra_gerada` e `arquivo_parecer`. Essa estrutura permite que cada métrica seja calculada a partir de operações simples de filtragem, contagem ou agregação sobre as 150 linhas do CSV correspondentes aos 50 cenários executados com três configurações de *prompt* cada.

4.2.4.2 Métricas determinísticas (nó 5)

O primeiro grupo de métricas avalia o desempenho sintático e estrutural do agente, com base nas decisões determinísticas do nó 5 (validador). A partir do `.csv` serão calculados quatro indicadores:

1. a “taxa de aprovação sintática”, definida como a proporção entre execuções em que `status = APROVADO` e o total de execuções;
2. o “número médio de tentativas” até a aprovação (ou até o esgotamento do limite), calculado sobre a coluna `tentativas`;
3. a “distribuição dos modos de falha”, obtida a partir do agrupamento das execuções com `status = REPROVADO` pela mensagem registrada em `erro_validacao_final` — classificando-as nas três etapas internas do nó 5: erro de *parsing* YAML, ausência de campo obrigatório ou erro semântico do *pySigma*;
4. “tempo médio por execução”, calculado sobre a coluna `tempo_total_s`.

Estes quatro indicadores cobrem o desempenho do agente quanto à aprovação sintática das regras geradas. Aspectos como qualidade semântica, similaridade com o gabarito e comparação entre estágios são tratados nas subseções seguintes.

4.2.4.3 Métricas semânticas (nó 6)

O segundo grupo de métricas provém do parecer do juiz semântico, do nó 6, e cobre aspectos distintos da conformidade sintática. Três indicadores serão calculados: (i) a “distribuição de vereditos”, ou seja, as proporções de regras classificadas como `APPROVED`, `APPROVED_WITH_WARNINGS` e `REJECTED`, obtidas diretamente da coluna `veredito_juiz`; (ii) a “taxa de *PASS* por dimensão”, calculada separadamente para cada uma das seis colunas `juiz_dimensão`, permitindo identificar em quais aspectos o agente apresenta maior ou menor solidez (por exemplo, se a dimensão `semantic_alignment` apresenta taxas sistematicamente menores que `structure`, isso indica que as regras tendem a ser bem formadas mas semanticamente desalinhadas com a ameaça); e (iii) a “taxa de qualidade entre nó 5 e 6”, definida como a proporção de regras que foram aprovadas pelo validador sintático (`status = APROVADO`) mas que receberam veredito `REJECTED` ou `APPROVED_WITH_WARNINGS` pelo juiz semântico — em outras palavras, esta última métrica permite quantificar a divergência entre validação sintática e avaliação semântica.

4.2.4.4 Métricas de similaridade com o gabarito

O terceiro grupo de métricas avalia a proximidade entre as regras geradas e as regras de referência correspondentes do gabarito do *SigmaHQ*, em uma comparação direta um-a-um. Diferentemente das métricas anteriores, que avaliam atributos absolutos de cada regra gerada, aqui o cálculo é conjunto sobre a regra gerada (`arquivo_regra_gerada` no `.csv`) e a regra original correspondente (`regra_referencia`). São adotadas duas perspectivas complementares, similaridade semântica geral e acerto estrutural sobre campos específicos.

A similaridade semântica geral é medida pela similaridade do cosseno entre vetores de *embeddings* das duas regras. Os vetores são calculados com o modelo `BAAIbge-small-en-v1.5` usado pelo nó 3 (Seção 4.2.3.4), portanto o modelo que o agente utiliza para recuperar exemplos similares no banco vetorial é o mesmo que se mede a proximidade entre regra gerada e gabarito. O resultado é um valor entre -1 e 1 , no qual valores próximos de 1 indicam regras semanticamente próximas e valores próximos de 0 indicam regras semanticamente distantes. Essa métrica captura proximidade global — se duas regras descrevem ameaças relacionadas, terão *embeddings* próximos, mesmo que escrevam campos individuais de formas diferentes.

Quanto ao acerto estrutural, será medido pelo índice de *Jaccard* sobre conjuntos derivados de campos-chave da regra, comparados par a par entre regra gerada e regra-alvo. Diferentemente do cosseno, que opera sobre vetores contínuos e dá uma visão geral, o índice de *Jaccard* opera sobre conjuntos discretos e responde a perguntas pontuais sobre quais elementos estão presentes em comum. Três comparações serão reportadas separadamente: (i) *Jaccard* sobre o conjunto de *tags* declaradas, que responde se o agente identificou as técnicas

MITRE ATT&CK corretas; (ii) *Jaccard* sobre o conjunto de campos do bloco `detection`, que responde se o agente escolheu os campos de corretos para a detecção; e (iii) *Jaccard* sobre o par (`category`, `product`) do `logsource`, que responde se o agente identificou corretamente a fonte de *log* adequada à ameaça. Reportar as três comparações separadamente permite distinguir, por exemplo, regras que erram fonte de *log* mas acertam as *tags* de regras que fazem o oposto.

A escolha pelas duas métricas em conjunto, em vez de uma só, se dá pela complementaridade conceitual. Enquanto o cosseno fornece uma medida que captura proximidade mesmo quando as duas regras não compartilham campos exatos, *Jaccard*, em contraste, responde se um campo específico foi acertado ou não. Usar somente o cosseno deixaria perguntas estruturais sem resposta direta; usar somente *Jaccard* ignoraria a proximidade semântica entre regras que se referem a ameaças correlatas mas com vocabulários distintos.

5 RESULTADOS

5.0.1 Produtos obtidos

Esta seção apresenta a composição final do conjunto experimental empregado nas análises seguintes.

A Tabela 2 sintetiza a quantidade de regras *Sigma* em cada um dos quatro grupos comparativos, bem como sua origem e o respectivo procedimento de obtenção. Os Estágios 2 e 3, que originalmente contavam com 250 regras cada (50 cenários $\times$ 5 LLMs comerciais), foram reduzidos a 50 regras mediante o procedimento de filtragem descrito na Seção 4.2.2.4, com o propósito de equiparar o número de regras entre os quatro grupos e permitir a aplicação de testes estatísticos pareados.

Tabela 2 – Composição final dos quatro grupos comparativos

Estágio	Designação	N	Origem	Procedimento de obtenção
1	Regras originais	50	Repositório SigmaHQ (test_cases)	Amostragem estratificada <i>round-robin</i>
2	Geração <i>zero-shot</i>	50	5 LLMs comerciais $\times$ 50 cenários	Filtragem <i>best-of-N</i> por LLM-as-a-judge
3	Engenharia de <i>prompt</i>	50	5 LLMs comerciais $\times$ 50 cenários	Filtragem <i>best-of-N</i> por LLM-as-a-judge
4	Agente autônomo com RAG	50	Agente <i>LangGraph</i> (prompt3)	Execução única por cenário
Total		200		

Fonte: Autoria própria (2026).

As Tabelas 3 e 4 ilustram as regras produto do agente desenvolvido assim seus respectivos pareceres, para cada cenário e *prompt*.

Tabela 3 – Regras geradas pelo `sigma_agent_automatico.py`

Arquivo	N	Derivada de
<code>regra_gerada_prompt1.yml</code>	50	<code>prompt1</code>
<code>regra_gerada_prompt2.yml</code>	50	<code>prompt2</code>
<code>regra_gerada_prompt3.yml</code>	50	<code>prompt3</code>
Total	150	

Fonte: Autoria própria (2026).

Tabela 4 – Pareceres gerados pelo juiz do nó 6

Arquivo	N	Derivada de
prompt1.parecer.txt	50	Qwen2.5 14B
prompt2.parecer.txt	50	Qwen2.5 14B
prompt3.parecer.txt	50	Qwen2.5 14B
Total	150	

Fonte: Autoria própria (2026).

Para garantir que houve diversificação na seleção estratificada com *round-robin*, verificou-se a representatividade do conjunto de teste em relação ao repositório *SigmaHQ* e os 50 cenários foram analisados sob três aspectos: (i) a categoria de fonte de *log* (*log-source.category* ou *logsource.product*); (ii) a tática MITRE ATT&CK declarada nas *tags*; e (iii) o nível de severidade declarado no campo *level*. A Tabela 5 apresenta as distribuições observadas em cada dimensão.

Tabela 5 – Diversidade da seleção de regras *Sigma* por *round-robin*

	Dimensão	Categoria	N
(i)	Fonte de <i>log</i>	<i>Endpoint Windows</i>	17
		Aplicação	13
		Rede e <i>proxy</i>	9
		<i>Cloud / SaaS</i>	7
		<i>Endpoint Linux/macOS</i>	4
Total	—	—	50
(ii)	Tática MITRE ATT&CK	<i>initial-access</i>	15
		<i>defense-evasion</i>	14
		<i>privilege-escalation</i>	12
		<i>execution</i>	11
		<i>persistence</i>	9
		<i>command-and-control</i>	5
		outras (7 categorias)	15
Total	—	—	81
(iii)	Severidade (level)	<i>medium</i>	30
		<i>high</i>	16
		<i>critical</i>	2
		<i>low</i>	2
Total	—	—	50

Fonte: Autoria própria (2026).

Quanto à família de fonte de *log*, a distribuição provavelmente reflete a composição do repositório *SigmaHQ*, predominantemente voltado a ambientes corporativos, cobrindo as cinco classes mais relevantes de fontes de telemetria em SOCs modernos.

Quanto à tática MITRE ATT&CK, as três táticas mais frequentes são *initial-access*, *defense-evasion* e *privilege-escalation*, que figuram entre as três táticas mais observadas em incidentes segundo o último Picus Security (2026).

Quanto à severidade, a maior parcela das regras estão classificadas como *medium* ou *high*.

Por fim, como já mencionado anteriormente, uma execução de `sigma_agent_automatizado.py` produz um arquivo `metadados_unificado.csv`¹ que contém, para cada par (cenário, variação de *prompt*), um registro com os dados de execução e avaliação semântica, portanto, constitui boa parte das análises apresentadas nas seções subsequentes deste capítulo.

5.0.2 Desempenho determinístico do agente

Das 150 execuções que o agente foi submetido, (50 cenários $\times$ 3 variações de *prompt*), produziu 75 regras consideradas sintaticamente válidas pelo nó 5, correspondendo a uma taxa global de aprovação de 50,0% (Tabela 6). Entre os três *prompts*, *prompt1* apresentou a maior taxa (58,0%), seguido por *prompt2* (50,0%) e *prompt3* (42,0%). Não foram registradas falhas de execução (ERRO_EXECUCAO=0, da coluna *status*) ao longo dos cinco lotes de execução.

Tabela 6 – Taxa de aprovação sintática do agente, global e por variação de *prompt*

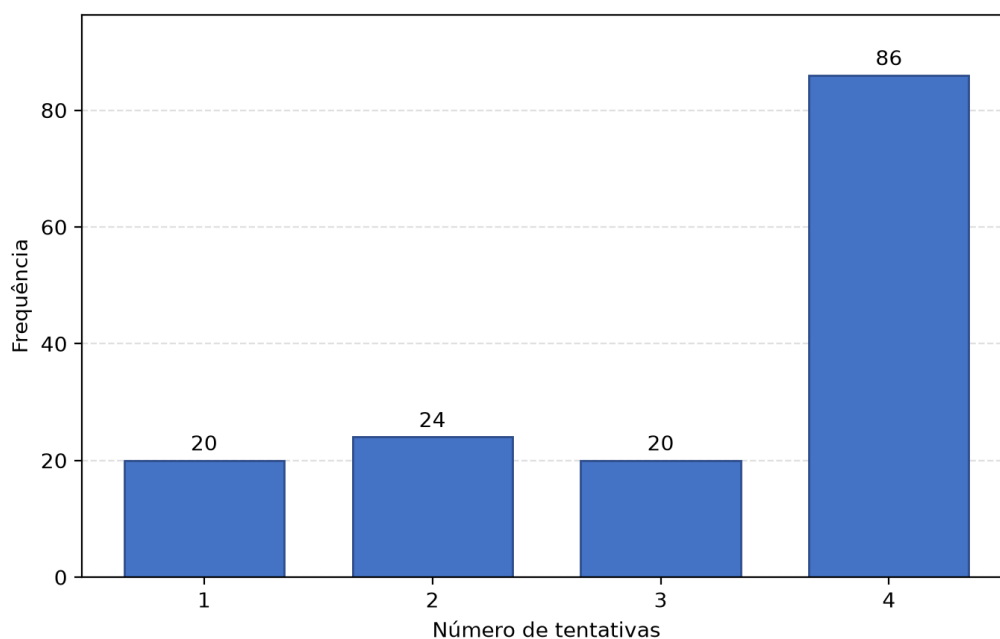
Grupo	N	APROVADO	REPROVADO	ERRO_EXECUCAO	Taxa de aprovação (%)
Global	150	75	75	0	50,0
<i>Prompt 1</i>	50	29	21	0	58,0
<i>Prompt 2</i>	50	25	25	0	50,0
<i>Prompt 3</i>	50	21	29	0	42,0

Fonte: Autoria própria (2026).

Quanto ao esforço de autocorreção entre os nós 4 e 5, foi acionado intensivamente ao longo do experimento. A distribuição do número de tentativas até o final de cada execução é apresentada na Figura 4. Enquanto 20 execuções (13,3%) obtiveram regra válida logo na primeira tentativa, 86 execuções (57,3%) atingiram o limite máximo de 4 tentativas estabelecido pela configuração do agente. Das 86 execuções que esgotaram o orçamento, apenas 11 obtiveram aprovação na quarta tentativa, enquanto as 75 restantes terminaram em estado de falha.

¹ Conferir em www.github.com/daninot/agente-autonomo-rag-regras-sigma/tree/main/data/analises

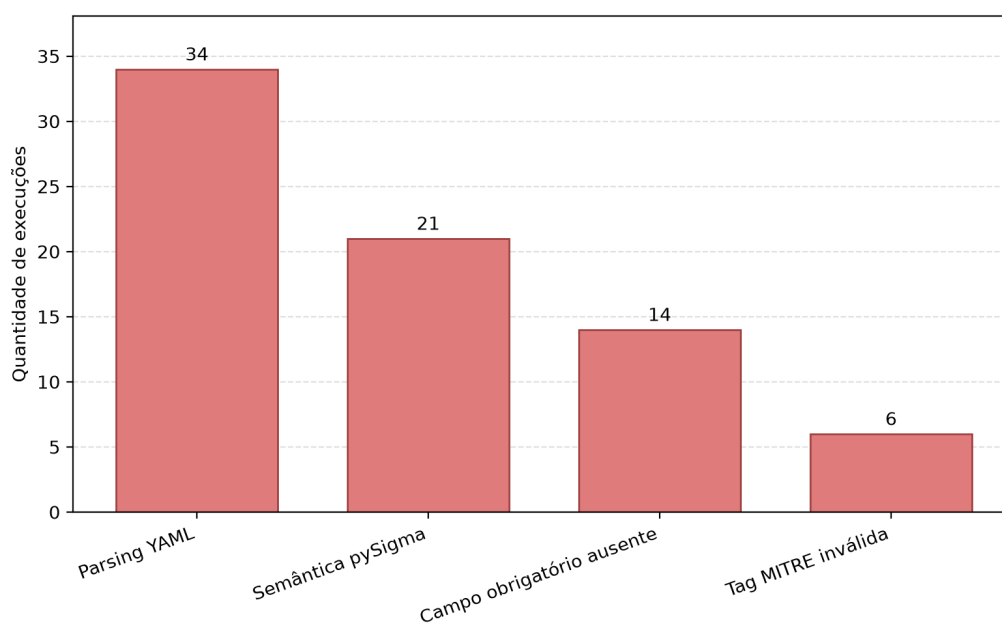
Figura 4 – Distribuição do número de tentativas até a aprovação no nó de validação determinística.



Fonte: Autoria própria (2026).

As 75 execuções reprovadas dividem-se em quatro categorias de erro mutuamente exclusivas, sumarizadas na Tabela 7 e ilustradas na Figura 5. As falhas de *parsing* YAML ² responderam pela maior parte das reprovações (45,3%), seguidas de não conformidades com a especificação da linguagem *Sigma* detectadas pelo *pySigma* (28,0%), ausência de campos obrigatórios na estrutura do documento (18,7%) e referências a *tags* MITRE inválidas (8,0%).

Figura 5 – Distribuição do.



Fonte: Autoria própria (2026).

² Falha de formatação.

Tabela 7 – Distribuição dos erros de validação por categoria

Categoria do erro	Quantidade	Percentual (%)
Parsing YAML	34	45,3
Validação pelo <i>pySigma</i>	21	28,0
Campo obrigatório ausente	14	18,7
Tag MITRE inválida	6	8,0

Fonte: Autoria própria (2026).

Em relação à influência das decisões dos nós iniciais sobre o desfecho final do agente, foi avaliada para as variáveis de configuração `usou_web_search`, `tipo_input` e `contexto_pobre`. A primeira apresentou valor constante (`True`) em todas as 150 execuções, portanto não foi empregada como fator explicativo.

Quanto à `tipo_input`, conforme apresentado na Tabela 8, observou-se taxa de aprovação menor para cenários classificados no nó 1 como CVE (33,3%, n=12) em comparação com cenários classificados como texto livre (51,4%, n=138). Já para `contexto_pobre`, se observou um padrão contraintuitivo, apresentado na Tabela 8: cenários com contexto considerado pobre pelo nó de avaliação inicial produziram taxa de aprovação mais alta (57,1%, n=49) do que cenários com contexto considerado rico (46,5%, n=101). Uma hipótese explicativa para esse comportamento é que, diante de menor base contextual, o agente tende a gerar regras estruturalmente mais simples — e consequentemente mais facilmente validáveis pelo nó determinístico (nó 5), enquanto que contextos ricos induzem a tentativas de geração mais elaboradas, que aumentam a chance de incorrer em erros sintáticos.

Tabela 8 – Resultados da validação por tipo de *input* e qualidade do contexto

Variável	Categoria	APROVADO	REPROVADO	Total	Taxa de aprovação (%)
<code>contexto_pobre</code>	Falso	47	54	101	46,5
	Verdadeiro	28	21	49	57,1
<code>tipo_input</code>	CVE	4	8	12	33,3
	Texto livre	71	67	138	51,4
Total Geral	—	75	75	150	50,0

Fonte: Autoria própria (2026).

5.0.3 Qualidade semântica das regras do agente

Como descrito na Seção 4.2.3.7, cada regra gerada pelo agente é submetida a uma avaliação semântica realizada pelo nó 6 (juiz *LLM-as-a-judge*), que emite um dos três vereditos possíveis: `APPROVED`, `APPROVED_WITH_WARNINGS` ou `REJECTED`. A Tabela 9 sumariza a distribuição desses vereditos nas 150 execuções, e a Figura 7 apresenta a mesma distribuição segmentada por variação de *prompt*.

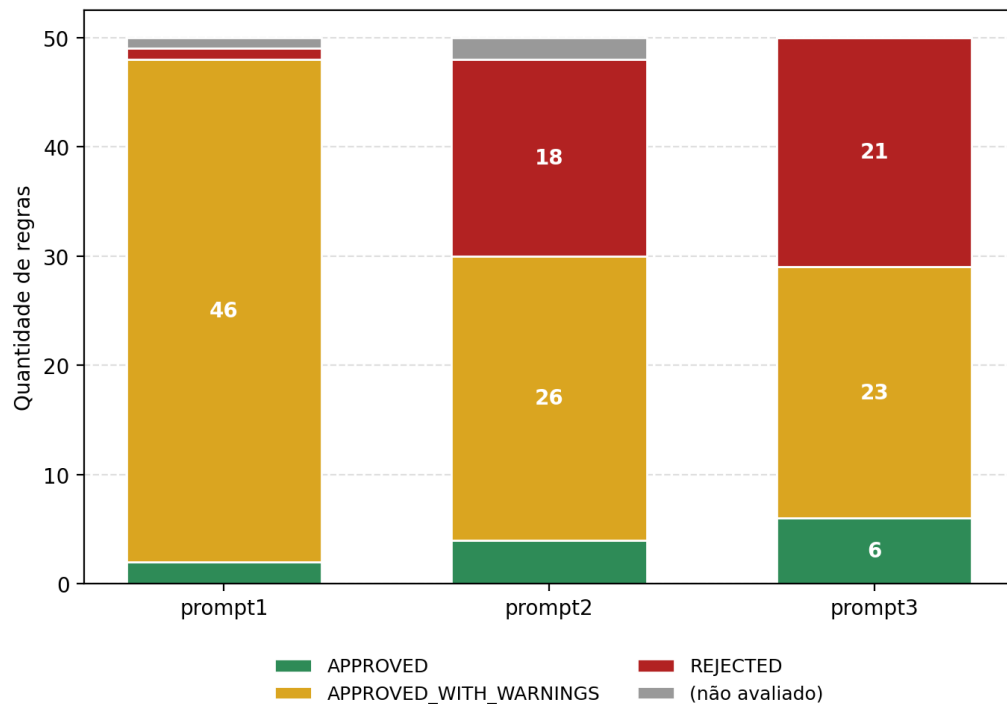
Das 150 regras submetidas ao juiz, apenas 12 (8,0%) receberam o veredito `APPROVED`, enquanto 95 (63,3%) foram classificadas como `APPROVED_WITH_WARNINGS`, 40 (26,7%) foram `REJECTED` e 3 (2,0%) não foram avaliadas.

Tabela 9 – Distribuição dos vereditos emitidos pelo nó 6 (juiz semântico)

Grupo	N	APPROVED	APPROVED_WITH_WARNINGS	REJECTED	Não avaliado
Global	150	12	95	40	3
<i>Prompt 1</i>	50	2	46	1	1
<i>Prompt 2</i>	50	4	26	18	2
<i>Prompt 3</i>	50	6	23	21	0

Fonte: Autoria própria (2026).

Figura 6 – Distribuição dos vereditos do juiz semântico estratificada por variação de *prompt*



Fonte: Autoria própria (2026).

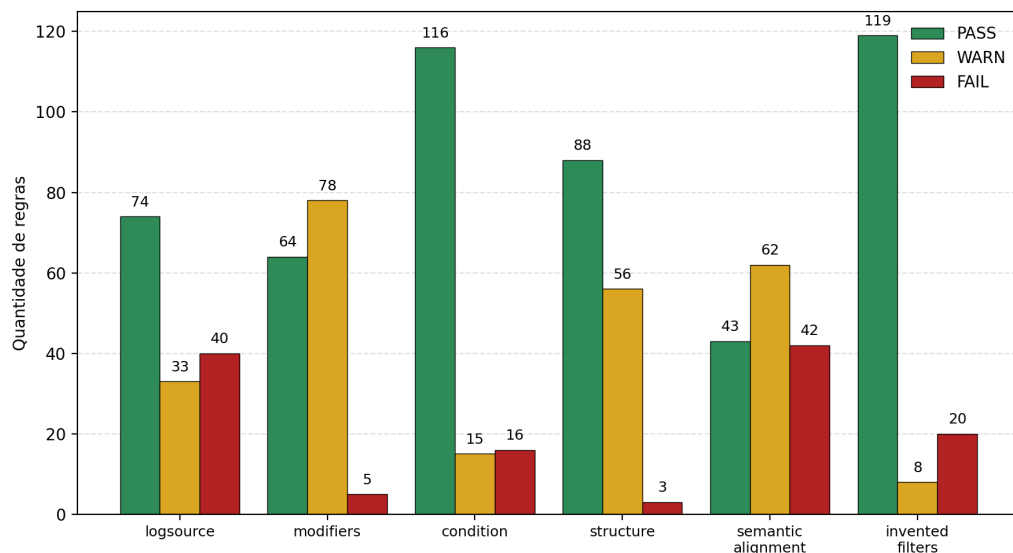
No nó 6, o parecer emitido pelo juiz é decomposto em seis dimensões independentes de avaliação: `logsource`, `modifiers`, `condition`, `structure`, `semantic_alignment` e `invented_filters`. Para cada dimensão o juiz atribui o status de `PASS` (sem objeções), `WARN` (ressalvas menores) ou `FAIL` (problema relevante). A Tabela 10 e a Figura 7 apresentam a contagem absoluta e relativa de cada status nas 147 regras efetivamente avaliadas.

Tabela 10 – Resultado dos vereditos nas seis dimensões de avaliação do juiz semântico

Dimensão	N avaliado	PASS	WARN	FAIL	PASS (%)	WARN (%)	FAIL (%)
Filtros inventados	147	119	8	20	81,0	5,4	13,6
Condição	147	116	15	16	78,9	10,2	10,9
Estrutura	147	88	56	3	59,9	38,1	2,0
Logsource	147	74	33	40	50,3	22,4	27,2
Modificadores	147	64	78	5	43,5	53,1	3,4
Alinhamento semântico	147	43	62	42	29,3	42,2	28,6

Fonte: Autoria própria (2026).

Figura 7 – Distribuição absoluta dos vereditos do juiz semântico



Fonte: Autoria própria (2026).

Verifica-se uma clara distinção entre as áreas de maior confiabilidade do agente e aquelas em que se verificam deficiências persistentes. Os pontos fortes se encontram em dimensões relacionadas à engenharia formal da regra. *invented_filters*, que mede a fidelidade do agente em não introduzir filtros ausentes do cenário obteve 81,0% de PASS; *condition*, que verifica a coerência da lógica *booleana* de detecção, obteve 78,9% de PASS. Já os pontos fracos se concentram em dimensões relacionadas ao conhecimento de domínio: *modifiers*, que se refere ao uso correto de operadores como *|contains* e *|endswith*, obteve de 43,5% de PASS; *logsource*, referente à escolha da fonte de log apropriada para o cenário, 50,3%; e, em especial, *semantic_alignment*, que mede a aderência da regra à intenção descrita no cenário de teste, atingiu apenas 29,3% de PASS.

O cruzamento entre o status determinístico produzido pelo nó 5 e o veredito semântico produzido pelo nó 6, apresentado na Tabela 11, tenta responder a esta pergunta.

Tabela 11 – Cruzamento entre o status do nó 5 (validação determinística) e o veredito do nó 6 (juiz semântico)

Status sintático	APPROVED	APPROVED_WITH_WARNINGS	REJECTED	Não avaliado	Total
Aprovado	7	45	21	2	75
Reprovado	5	50	19	1	75
Total Geral	12	95	40	3	150

Fonte: Autoria própria (2026).

Das 75 regras consideradas APROVADAS pelo validador determinístico do nó 5, ou seja, regras que são YAML válido, em conformidade com a especificação *Sigma*, com todos os campos obrigatórios e *tags* MITRE válidas, apenas 7 (9,3%) receberam o veredito `APPROVED` do juiz semântico. As demais 68 regras (90,7%) apresentaram pelo menos uma ressalva: 45 foram `APPROVED_WITH_WARNINGS`, 21 (28,0%) foram `REJECTED` apesar da validade sintática, e 2 não puderam ser avaliadas. Além disso, 5 regras REPROVADAS pelo validador do nó 5 receberam, ainda assim, veredito `APPROVED` do juiz.

Análise qualitativa dos motivos de reprovação

Em `metadados_unificado.csv`, o campo `motivo_reprovacao` registra para cada execução em que há observações a documentar, seja por reprovação no nó 5, seja por presença de ressalvas ou rejeição no nó 6, o conjunto completo de comentários textuais produzidos pelo juiz, organizados por dimensão. Esse campo foi preenchido em 75 das 150 execuções e contém um total de 201 comentários individuais, distribuídos como apresentado na Tabela 12.

A análise dos comentários textuais corrobora os achados quantitativos dos pareceres Tabela 10. As duas dimensões com maior volume de observações são `semantic_alignment` (49 comentários, predominantemente `WARN` ou `FAIL`) e `logsource` (42 comentários, igualmente distribuídos).

Exemplos representativos extraídos do conjunto de comentários incluem:

`semantic_alignment` [FAIL] — “*The detection logic does not align with the threat description. The rule focuses on specific keywords but misses broader reconnaissance and exfiltration activities*”.

`logsource` [FAIL] — “*The log source category is incorrectly set to registry_event instead of database*”.

`modifiers` [WARN] — “*|contains is used appropriately, but it might be better to use exact match if specific paths or file names are known*”.

A amostra completa de comentários, organizada por dimensão e status, está disponível no arquivo de dados suplementar `amostras_comentarios.csv`³.

³ Disponível em: www.github.com/daninot/agente-autonomo-rag-regras-sigma/blob/main/data/analises/amostras_comentarios.csv

Cabe notar que enquanto a Tabela 10 contabiliza os status atribuídos pelo juiz nas 147 regras avaliadas, a Tabela 12 contabiliza apenas os comentários textuais associados, gerados quando há observação para documentar.

Tabela 12 – Volume de comentários textuais do juiz por dimensão, segmentado por status

Dimensão	PASS	WARN	FAIL	Total de comentários
Alinhamento semântico	0	28	21	49
<i>Logsource</i>	0	23	19	42
Modificadores	0	38	3	41
Estrutura	0	29	1	30
Filtros inventados	0	7	13	20
Condição	0	10	9	19

Fonte: Autoria própria (2026).

5.0.4 Similaridade com o gabarito

Esta seção descreve as duas métricas de proximidade empregadas para confrontar as regras geradas em cada um dos cinco grupos comparativos com as regras-gabarito do conjunto de teste: similaridade semântica de cosseno entre *embeddings*, calculada tanto sobre o documento `YAML` completo quanto exclusivamente sobre o bloco `detection` e correspondência estrutural em cinco campos categóricos da regra. Os *embeddings* foram produzidos pelo modelo `BAAI/bge-small-en-v1.5`.

Para fins de confirmação, o conjunto experimental são os cinco grupos de regras *Sigma* contra as 50 regras-gabarito do conjunto de teste. São eles: o E2, composto por 50 regras geradas em modo *zero-shot* por cinco modelos de linguagem comerciais e filtradas por procedimento de *best-of-N* via *LLM-as-a-judge* (uma regra por cenário); o E3, construído da mesma forma, porém com uso de engenharia de *prompt*; e os três subgrupos do E4, correspondentes às 50 regras geradas pelo agente proposto para cada uma das três variações de *prompt* avaliadas (E4-p1, E4-p2 e E4-p3).

5.0.4.1 Similaridade semântica

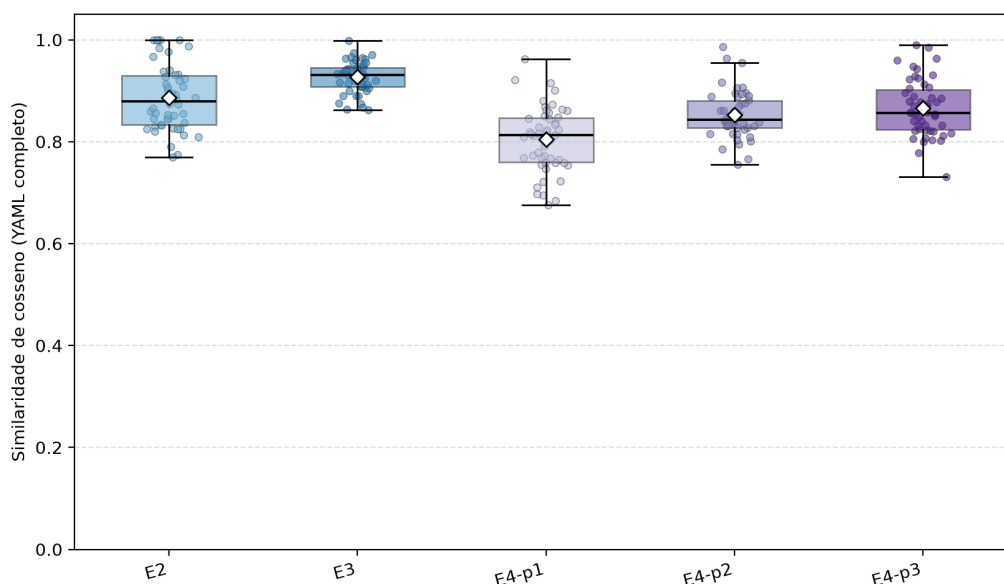
A Tabela 13 apresenta as estatísticas descritivas média, mediana e desvio-padrão da similaridade de cosseno entre cada regra gerada e sua respectiva regra-gabarito, calculadas sobre as 50 regras de cada grupo, para as duas métricas: similaridade do `YAML` completo e similaridade só do bloco `detection`. As Figuras 8 e 9 ilustram as distribuições por meio de *boxplots* sobrepostos aos 50 valores individuais de cada grupo.

Tabela 13 – Resultados estatísticos de similaridade de cosseno global e de *detection* por grupo

Grupo	Similaridade Global			Similaridade <i>Detection</i>		
	Média	Mediana	DP	Média	Mediana	DP
E2	0,887	0,879	0,064	0,829	0,827	0,154
E3	0,927	0,931	0,030	0,948	0,961	0,048
E4-p1	0,805	0,812	0,063	0,462	0,704	0,398
E4-p2	0,853	0,842	0,046	0,421	0,336	0,425
E4-p3	0,866	0,856	0,055	0,347	0,000	0,413

Fonte: Autoria própria (2026).

Figura 8 – Distribuição da similaridade de cosseno entre os YAMLS completos das regras geradas e das regras-gabarito, por grupo.

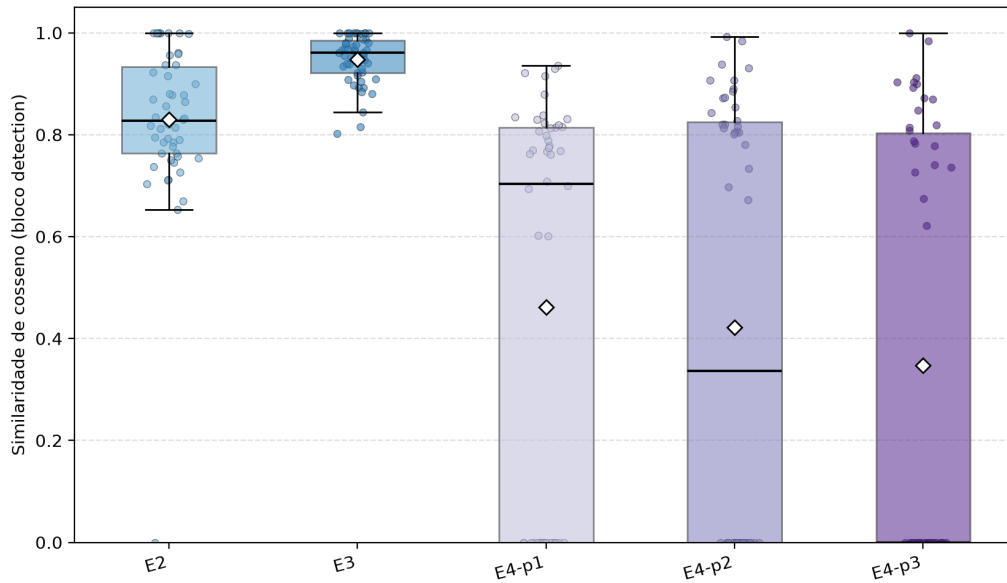


Fonte: Autoria própria (2026).

Quanto à similaridade do *YAML* completo, todos os cinco grupos apresentaram medianas acima de 0,80, com E3 registrando o valor mais alto (0,931) e o E4-p1 o mais baixo (0,812), uma diferença de 0,12 pontos. Em se tratando da similaridade do bloco *detection*, a dispersão entre grupos foi maior, já que as medianas variaram de 0,961 no E3 a 0,000 no E4-p3, com desvios-padrão entre 0,048 (E3) e 0,425 (E4-p2). Os três subgrupos do E4 apresentaram desvios-padrão superiores a 0,39, maiores do que os observados em E2 (0,154) e E3 (0,048).

A Tabela 14 apresenta a distribuição dos cenários em três faixas de similaridade da *detection*: alta ($> 0,5$), média (0,1 a 0,5) e baixa ($\leq 0,1$). Em todos os cinco grupos, a frequência de cenários na faixa média foi nula; as observações se concentraram mais nos extremos.

Figura 9 – Distribuição da similaridade de cosseno entre os blocos `detection` das regras geradas e das regras-gabarito, por grupo.



Fonte: Autoria própria (2026).

Tabela 14 – Distribuição dos resultados em níveis (Alta, Média e Baixa) por grupo

Grupo	N	Alta (>0,5)		Média (0,1–0,5]		Baixa ($\leq$ 0,1)	
		Qtd.	%	Qtd.	%	Qtd.	%
E2	50	49	98,0	0	0,0	1	2,0
E3	50	50	100,0	0	0,0	0	0,0
E4-p1	50	29	58,0	0	0,0	21	42,0
E4-p2	50	25	50,0	0	0,0	25	50,0
E4-p3	50	21	42,0	0	0,0	29	58,0

Fonte: Autoria própria (2026).

Em E2 e E3, todos ou quase todos dos cenários caiu na faixa alta (98,0% e 100,0%, respectivamente). Nos três subgrupos do E4, a distribuição foi aproximadamente equilibrada entre as faixas alta e baixa: 58,0% / 42,0% no E4-p1, 50,0% / 50,0% no E4-p2, e 42,0% / 58,0% no E4-p3.

A Tabela 15 apresenta as mesmas estatísticas descritivas da similaridade do bloco `detection`, recalculadas após exclusão dos cenários em que a regra do grupo apresentou YAML inválido (considerando apenas os pares em que ambos os blocos `detection` eram extraíveis).

Tabela 15 – Estatísticas descritivas da similaridade do bloco `detection`, considerando apenas cenários com YAML válido na regra gerada

Grupo	N (válidos)	Média	Mediana	DP
E2	49	0,846	0,830	0,100
E3	50	0,948	0,961	0,048
E4-p1	29	0,796	0,807	0,081
E4-p2	25	0,843	0,827	0,077
E4-p3	21	0,827	0,819	0,094

Fonte: Autoria própria (2026).

Sob essa restrição, as medianas dos três subgrupos do E4 aumentaram para 0,807 (E4-p1), 0,827 (E4-p2) e 0,819 (E4-p3), enquanto E2 manteve mediana 0,830 e E3, 0,961. Os desvios-padrão dos subgrupos do E4 se reduziram a valores entre 0,077 e 0,094, próximos ao do E2 (0,100).

Após o filtro, o número de cenários considerados em cada grupo é o mesmo apresentado na Tabela 18: 49 no E2, 50 no E3, 29 no E4-p1, 25 no E4-p2 e 21 no E4-p3.

5.0.4.2 Correspondência estrutural

A Tabela 16 apresenta a correspondência entre cada regra gerada e sua regra-gabarito em cinco campos: `logsource.category`, `logsource.product`, `logsource.service` (taxa de acerto exato), `tags` (índice médio de *Jaccard*) e `level` (taxa de acerto exato). O denominador inclui as 50 regras de cada grupo; regras com YAML inválido foram contabilizadas como erro em todos os campos.

Tabela 16 – Correspondência estrutural entre regras geradas e regras-gabarito, calculada sobre o total de 50 regras por grupo.

Grupo	N considerado	Logsource (%)			Tags (Jaccard)	Nível (%)
		Categoria	Produto	Serviço		
E2	50	58,3	70,5	53,3	0,314	58,0
E3	50	100,0	100,0	100,0	0,333	54,0
E4-p1	50	5,6	27,3	20,0	0,067	12,0
E4-p2	50	19,4	40,9	26,7	0,108	28,0
E4-p3	50	16,7	25,0	13,3	0,062	16,0

Fonte: Autoria própria (2026).

O bloco `detection` foi propositalmente excluído desta análise, uma vez que sua estrutura aninhada e a possibilidade de expressar a mesma intenção lógica por meio de diferentes combinações de campos e seleções tornam métricas de *match* exato e de *Jaccard* inadequa-

das; a comparação semântica desse bloco foi conduzida na Seção anterior por meio de *embeddings*.

O E3 registrou taxas de 100% de acerto em todos os três campos de *logsource*; E2 registrou 58,3%, 70,5% e 53,3% em *category*, *product* e *service*, respectivamente. Os três subgrupos do E4 apresentaram taxas entre 5,6% e 19,4% em *category*, entre 25,0% e 40,9% em *product* e entre 13,3% e 26,7% em *service*. O índice médio de *Jaccard* sobre *tags* variou de 0,06 (E4-p3) a 0,33 (E3). A taxa de acerto exato em *level* foi de 58,0% no E2 e 54,0% no E3; nos subgrupos do E4, os valores foram 12,0%, 28,0% e 16,0%.

A Tabela 17 traz a mesma análise, agora com o denominador limitado às regras cujo *YAML* é válido em cada grupo.

Tabela 17 – Correspondência estrutural restrita às regras com *YAML* válido por grupo.

Grupo	N considerado	Logsource (%)			Tags (Jaccard)	Nível (%)
		Categoria	Produto	Serviço		
E2	49	60,0	72,1	53,3	0,320	59,2
E3	50	100,0	100,0	100,0	0,333	54,0
E4-p1	29	10,5	50,0	27,3	0,115	20,7
E4-p2	25	41,2	81,8	50,0	0,216	56,0
E4-p3	21	50,0	61,1	22,2	0,148	38,1

Fonte: Autoria própria (2026).

Com essa restrição, as taxas de acerto exato dos subgrupos de E4 foram: em *category*, 10,5% (E4-p1), 41,2% (E4-p2) e 50,0% (E4-p3); em *product*, 50,0%, 81,8% e 61,1%; em *service*, 27,3%, 50,0% e 22,2%; e em *level*, 20,7%, 56,0% e 38,1%. O índice de *Jaccard* médio sobre *tags* subiu para 0,11, 0,22 e 0,15 nos três subgrupos. As taxas correspondentes do E2 e do E3 permaneceram próximas às reportadas na Tabela 16.

A Tabela 18 apresenta a taxa de *YAML* válido por grupo, que define os denominadores aplicados nas Tabelas 15 e 17.

Tabela 18 – Taxa de regras com *YAML* válido por grupo.

Grupo	N	YAML válidos	Taxa (%)
E2	50	49	98,0
E3	50	50	100,0
E4-p1	50	29	58,0
E4-p2	50	25	50,0
E4-p3	50	21	42,0

Fonte: Autoria própria (2026).

As taxas observadas foram de 98,0% (E2), 100,0% (E3), 58,0% (E4-p1), 50,0% (E4-p2) e 42,0% (E4-p3).

5.1 Discussões

Os resultados expostos no capítulo anterior possibilitam organizar quatro discussões interligadas, cada uma emergindo da convergência de evidências provenientes dos três instrumentos de avaliação utilizados — validador determinístico (Seção 5.0.2), juiz semântico (Seção 5.0.3) e métricas externas de similaridade e correspondência (Seção 5.0.4). A primeira discute a relação entre validade formal e qualidade semântica de uma regra *Sigma*; a segunda identifica a barreira sintática como o gargalo dominante do agente; a terceira examina o equilíbrio entre conservadorismo e ambição revelado pela comparação entre as três variações de *prompt*; e a quarta reflete sobre o viés inerente ao procedimento de filtragem aplicado aos grupos comparativos baseados em LLMs comerciais.

5.1.1 Validação sintática como condição necessária mas não suficiente

A separação entre o nó 5 (validação determinística) e o nó 6 (juiz semântico) foi uma boa decisão, apoiada pelos dados, pelo menos para analisar a qualidade das regras geradas. Das 75 regras aprovadas pelo nó 5, apenas 7 (9,3%) receberam veredito `APPROVED` do juiz; 21 (28,0%) foram `REJECTED` mesmo sendo sintaticamente válidas. No caminho inverso, 5 das regras reprovadas pelo nó 5 receberam `APPROVED` do juiz. Forma e significado, portanto, não caminham juntos em todos os casos, pois parte das regras válidas é semanticamente ruim, e algumas regras quebradas refletem bem a intenção do cenário. Validar apenas a sintaxe captura só uma parte da qualidade real da saída.

A correspondência exata em `logsource.category` no E4 (entre 5,6% e 19,4%) e os 29,3% de `PASS` em `semantic_alignment` apontam, por caminhos diferentes, para a mesma fragilidade: alinhar a regra gerada ao cenário é o ponto mais fraco do agente. As duas métricas têm correlação esperada (não são validações independentes), mas a convergência reforça o diagnóstico.

5.1.2 A barreira sintática como gargalo principal

A taxa de `YAML` válido (Tabela 18) e a similaridade da `detection` filtrada (Tabela 15) mudam a forma de ler os resultados do E4. Sobre as 50 regras, a mediana da similaridade da `detection` foi 0,704, 0,336 e 0,000 nos três subgrupos. No entanto, considerando só as regras com `YAML` válido, a mediana fica entre 0,807 e 0,827, próxima do E2 (0,830).

Isso não significa equivalência semântica entre o E4 e o E2. O filtro do E2 seleciona o que já é válido, enquanto o E4 entrega entre 42% e 58% de saídas inválidas. Mas mostra que, quando o agente entrega `YAML` válido, a qualidade da `detection` fica em patamar comparável ao do E2. A bimodalidade observada nos cinco grupos — faixa intermediária (0,1 a

0,5) vazia em todos eles, reforça a leitura de que o agente opera em dois regimes claros, "regra válida com detecção aceitável" ou "saída sintaticamente quebrada".

5.1.3 Variações de *prompt*

Os três subgrupos do E4 mostram uma inversão de desempenho entre as duas dimensões avaliativas. Na validação determinística, `prompt1` (58,0%) > `prompt2` (50,0%) > `prompt3` (42,0%). Na avaliação semântica, o `prompt1` gerou apenas 2 regras APPROVED puras contra 6 do `prompt3` — que também concentrou o maior número de REJECTED.

Uma leitura coerente é que o `prompt1`, por pedir saídas mais simples, leva o agente a produzir regras conservadoras, que passam na validação com mais frequência, mas raramente atingem qualidade plena (46 das 50 receberam veredito intermediário). O `prompt3` pede saídas mais elaboradas e gera tentativas mais ambiciosas; quando acertam, acertam melhor; quando falham, falham por completo. O `prompt2` fica no meio em ambas as dimensões.

O tamanho amostral por subgrupo (50 regras) não permite afirmação estatística firme sobre as diferenças. Mas o padrão se repete em métricas distintas (validação, dimensões do juiz, similaridade), o que sugere consistência. A escolha entre as variações não é "qual é a melhor", mas qual regime serve ao uso. Em *pipelines* automáticos de SOC, em que validade é prioridade, o `prompt1` é mais estável. Em uso assistido por analista humano, o `prompt3` pode compensar pelo maior número de regras de alta qualidade, mesmo com mais descartes.

5.1.4 Viés do procedimento de filtragem

Sobre a montagem dos grupos E2 e E3, vale pontuar honestamente sobre o viés de que LLMs tendem a avaliar bem saídas parecidas com o que elas próprias produziram (Zheng *et al.*, 2023).

Também há uma diferença real entre as classes de modelo comparadas. As cinco LLMs comerciais (*ChatGPT*, *Claude*, *DeepSeek*, *Gemini* e *Microsoft Copilot*) são bem maiores que *Llama 3.1 8B* usado pelo agente, e passam constantemente por ajustes extensos por *feedback* humano, além de conseguirem acessar a internet — que os permite consultar o repositório *SigmaHQ* oficial para elaborar as regras. Essa diferença de escala (o agente foi desenvolvido em *setup* local, com restrições de infraestrutura) tende a se manifestar especialmente em tarefas que exigem aderência a formatos estritos como *YAML*, em que o modelo precisa simultaneamente raciocinar sobre o domínio (qual detecção é apropriada para o cenário) e respeitar uma gramática rígida (estrutura *Sigma*, campos obrigatórios, sintaxe de modificadores). Modelos maiores costumam ter desempenho superior em ambas as dimensões.

Há ainda um terceiro fator: o repositório *SigmaHQ* é público e indexado. É provável que parte dessas regras tenha entrado nos dados de treinamento das LLMs comerciais, o que ajuda

a explicar a correspondência de 100% do E3 em `logsource`. O E4 usa essas mesmas regras via RAG como recuperação externa, não como conhecimento já internalizado pelo modelo, o que dá menos garantia de reprodução exata.

O que os dados sustentam, portanto, é uma afirmação restrita: o agente, na configuração atual, não supera abordagens comerciais com filtragem por *ensemble*; e parte dessa diferença vem de vantagens estruturais (escala, treinamento, possível exposição prévia ao domínio).

5.2 Limitações e Trabalhos futuros

Este capítulo apresenta as principais limitações deste trabalho e os encaminhamentos correspondentes para investigações futuras.

5.2.1 Limitações

- a) Tamanho amostral — o conjunto de teste contém 50 cenários, número limitado para inferência estatística firme sobre as diferenças observadas entre os grupos comparativos. Conclusões definitivas sobre a superioridade de um método sobre outro requerem amostra maior;
- b) Limitações de infraestrutura — o agente foi desenhado para execução local, sob restrições de *hardware* doméstico (GPU NVIDIA RTX 3060 com 12 GB de VRAM dedicada). Essa restrição determinou a escolha do *Llama 3.1 8B* como modelo gerador principal, modelo significativamente menor que as LLMs comerciais usadas nos grupos E2 e E3. Parte da diferença de desempenho observada reflete essa assimetria de escala, não atribuível à arquitetura do agente em si;
- c) Ausência de treinamento especializado — o modelo *Llama 3.1 8B* foi utilizado em sua versão pré-treinada, sem qualquer ajuste específico ao domínio de geração de regras *Sigma*. Técnicas de adaptação supervisionada poderiam aumentar a familiaridade do modelo com a estrutura formal da linguagem *Sigma* e reduzir a fração de saídas sintaticamente inválidas observada no E4 (42% a 58% conforme a variação de *prompt*);
- d) Viés do procedimento de filtragem — a montagem dos grupos E2 e E3 envolvendo seleção das melhores regras geradas por LLMs comerciais favoreceu saídas alinhadas com a regra-gabarito — propriedade mensurada pelas próprias métricas externas. A comparação direta entre o agente e abordagens comerciais permanece, portanto, assimétrica;
- e) Avaliação semântica restrita a *LLM-as-a-judge* — a avaliação semântica conduzida pelo nó 6 e pelos cinco juízes da filtragem foi inteiramente realizada por modelos de linguagem. A literatura indica boa correlação entre *LLM-as-a-judge* e avaliação humana

para tarefas estruturadas (Zheng *et al.*, 2023), mas essa correspondência não é total, e nuances de qualidade de uma regra *Sigma* podem escapar a um avaliador automático.

5.2.2 Trabalhos futuros

- a) Ampliação amostral — replicação do experimento com 100 ou mais cenários. Preferencialmente cobrindo diferentes famílias de *logsource* de forma equilibrada para suportar inferência estatística mais robusta e permitir análises estratificadas por tipo de cenário.
- b) Execução com modelos de maior escala — reprodução do experimento substituindo o *Llama 3.1 8B* por modelos abertos de maior escala (como variantes do *Llama 3.1 70B*), executados em infraestrutura adequada. Esse experimento isolaria o efeito da escala do modelo sobre o desempenho do agente.
- c) Fine-tuning supervisionado e geração estruturada — duas direções complementares para reduzir a fração de `YAML` inválido produzida pelo agente. A primeira consiste em *fine-tuning* supervisionado do modelo gerador via técnicas como LoRA ??, usando pares de regra *Sigma* e instrução em linguagem natural, abordagem que demanda construção de *dataset* adequado e validação cuidadosa contra superajuste.
- d) Validação com avaliação humana — submissão de uma amostra estratificada de regras (em torno de 30 regras, distribuídas entre os grupos) à avaliação de analistas de detecção experientes, segundo uma rubrica definida *a priori*. Os resultados serviriam para calibrar o quanto a avaliação semântica automatizada deste trabalho corresponde ao julgamento de profissionais da área.
- e) Reposicionamento do juiz semântico como bloqueador no fluxo do agente — na arquitetura atual, o juiz semântico (nó 6) atua como avaliador pós-execução: emite parecer sobre a regra finalizada, mas não interfere no fluxo de geração. Os resultados da Seção 5.0.2 mostraram que, das 75 regras aprovadas pelo validador sintático (nó 5), apenas 9,3% receberam `APPROVED` do juiz e 28,0% foram `REJECTED` apesar da validade sintática. Ou seja, parte das regras formalmente válidas chega ao usuário com problemas semânticos. Uma direção promissora é reposicionar o juiz como segundo bloqueador no laço de autocorreção: regras `REJECTED` retornariam ao nó gerador junto com as justificativas por dimensão; regras `APPROVED_WITH_WARNINGS` poderiam ser devolvidas para refinamento, sob limite máximo de tentativas. A modificação transformaria o juiz, de instrumento de avaliação, em instrumento de controle de qualidade da saída.

6 CONCLUSÃO

REFERÊNCIAS

- Accenture. **Securing the Digital Economy: Reinventing the Internet for Trust**. Dublin, Ireland, 2019. Relatório corporativo. Disponível em: <https://newsroom.accenture.com/news/2019/cybercrime-could-cost-companies-us-5-2-trillion-over-next-five-years-according-to-new-research-from-accenture>
- ALCANTARA, L. *et al.* Syrius: Synthesis of rules for intrusion detectors. **IEEE Transactions on Reliability**, IEEE, v. 71, n. 1, p. 370–381, 2021.
- BANERJEE, S.; LAVIE, A. METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. *In: Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. [S.l.: s.n.], 2005. p. 65–72.
- BOOTH, H.; RIKE, D.; WITTE, G. A. **The National Vulnerability Database (NVD): Overview**. Gaithersburg, MD, 2013.
- BROWN, T. B. *et al.* Language models are few-shot learners. **Advances in Neural Information Processing Systems**, v. 33, p. 1877–1901, 2020.
- BUSINESS, V. **2018 Data Breach Investigations Report**. New York, USA, 2018. Disponível em: https://www.verizon.com/business/resources/reports/DBIR_2018_Report.pdf.
- BUSINESS, V. **2025 Data Breach Investigations Report (DBIR)**. USA, 2025. C. David Hylender, Philippe Langlois, Alex Pinto, Suzanne Widup (eds.). Disponível em: <https://www.verizon.com/business/resources/T437/reports/2025-dbir-data-breach-investigations-report.pdf>.
- CASELLI, T. *et al.* When it's all piling up: Investigating error propagation in an NLP pipeline. *In: Proceedings of the 1st Workshop on NLP Applications: Completing the Puzzle (WNACP 2015)*. [S.l.: s.n.], 2015.
- CHOWDHARY, K. R. Natural language processing. *In: Fundamentals of Artificial Intelligence*. New Delhi: Springer, 2020. p. 603–649.
- DIMITROV, D.; NIKOLOV, D. Leveraging yara and sigma rules to detect chinese state-sponsored hacking groups of the "typhoon" type. *In: Environment. Technology. Resources. Proceedings of the 16th International Scientific and Practical Conference*. Rezekne, Latvia: RTU Press, 2025. II, p. 107–113. Disponível em: <https://doi.org/10.17770/etr2025vol2.8617>.
- DU, M.; HU, W.; HEWLETT, W. AutoCombo: Automatic malware signature generation through combination rule mining. *In: Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. [S.l.]: ACM, 2021. p. 3777–3786.
- EXABEAM. **AI SIEM: How SIEM with AI/ML is Revolutionizing the SOC**. 2025. Disponível em: <https://www.exabeam.com/explainers/siem/ai-siem-how-siem-with-ai-ml-is-revolutionizing-the-soc/>. Acesso em: 20 jun. 2025.
- Forum of Incident Response and Security Teams. **Common Vulnerability Scoring System version 3.1: Specification Document**. [S.l.], 2019. Disponível em: <https://www.first.org/cvss/v3-1/specification-document>.
- GAMBLIN, J. **2025 CVE Data Review**. 2026. Análise consolidada dos dados da National Vulnerability Database (NVD): 48.185 CVEs publicados em 2025. Disponível em: <https://jerrygamblin.com/2026/01/01/2025-cve-data-review/>.

- GONZÁLEZ-GRANADILLO, G.; GONZÁLEZ-ZARZOSA, S.; DIAZ, R. Security information and event management (SIEM): Analysis, trends, and usage in critical infrastructures. **Sensors**, v. 21, n. 14, p. 4759, 2021.
- HASANOV, I. *et al.* Application of large language models in cybersecurity: A systematic literature review. **IEEE Access**, IEEE, v. 12, p. 176751–176778, 2024.
- IBM Security. **Cost of a Data Breach Report 2024**. Armonk, NY, 2024. Pesquisa conduzida pelo Ponemon Institute. Disponível em: <https://www.ibm.com/downloads/documents/us-en/107a02e94948f4ec>.
- IBM Security. **Cost of a Data Breach Report 2025**. Armonk, NY, 2025. Pesquisa conduzida pelo Ponemon Institute. Disponível em: <https://www.ibm.com/reports/data-breach>.
- JACCARD, P. The distribution of the flora in the alpine zone. **New Phytologist**, Wiley, v. 11, n. 2, p. 37–50, 1912.
- JOHNSON, J.; DOUZE, M.; JÉGOU, H. Billion-scale similarity search with GPUs. **IEEE Transactions on Big Data**, IEEE, v. 7, n. 3, p. 535–547, 2019.
- JORDAN, M. I.; MITCHELL, T. M. Machine learning: Trends, perspectives, and prospects. **Science**, American Association for the Advancement of Science, v. 349, n. 6245, p. 255–260, 2015.
- JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition**. 2. ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2009. ISBN 9780131873216.
- KAUFMAN, S. *et al.* Leakage in data mining: Formulation, detection, and avoidance. **ACM Transactions on Knowledge Discovery from Data**, ACM, v. 6, n. 4, p. 1–21, 2012.
- LangChain. **Graph API overview — LangGraph documentation**. 2024. <https://docs.langchain.com/oss/python/langgraph/graph-api>. Acesso em: 17 jun. 2026.
- LangChain Team. **LangGraph: Build resilient language agents as graphs**. 2024. <https://langchain-ai.github.io/langgraph/>. Acesso em: 10 de junho.
- LangChain Team. **LangGraph: Building Stateful, Multi-Actor Applications with LLMs**. 2024. Disponível em: <https://langchain-ai.github.io/langgraph/>.
- LEMPINEN, M.; PYYNY, E.; JUNTUNEN, A. **Chatbot for Assessing System Security with OpenAI GPT-3.5**. 2023. 34 p.
- LEWIS, P. *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. *In: Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2020. v. 33, p. 9459–9474.
- LI, J. *et al.* **GRIDAI: Generating and Repairing Intrusion Detection Rules via Collaboration among Multiple LLM-based Agents**. 2025.
- LIDDY, E. D. Natural language processing. *In: Encyclopedia of Library and Information Science*. 2. ed. New York: Marcel Decker, Inc., 2001.
- LIN, C.-Y. ROUGE: A package for automatic evaluation of summaries. *In: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*. [S.l.: s.n.], 2004. p. 74–81.
- LIU, P. *et al.* Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. **ACM Computing Surveys**, ACM, v. 55, n. 9, p. 1–35, 2023.

MADAAN, A. *et al.* Self-Refine: Iterative refinement with self-feedback. *In: Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2023. v. 36.

Mandiant. Relatório técnico anual, **M-Trends 2026: Data, Insights, and Strategies From the Frontlines**. 2026. Tempo médio para exploração (time-to-exploit) estimado em -7 dias; exploits como vetor inicial mais comum pelo 6º ano consecutivo (32%). Disponível em: <https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026>.

MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. **Introduction to Information Retrieval**. Cambridge, UK: Cambridge University Press, 2008. ISBN 9780521865715.

MCGOWAN, M. **Detecting malicious activities with Sigma rules**. 2025. Disponível em: https://lantern.splunk.com/Security/UCE/Guided_Insights/Threat_hunting/Detecting_malicious_activities_with_Sigma_rules. Acesso em: 20 jun. 2025.

National Institute of Standards and Technology. **National Vulnerability Database – General Information**. 2025. Disponível em: <https://nvd.nist.gov/general>.

National Institute of Standards and Technology. **NIST Updates NVD Operations to Address Record CVE Growth**. 2026. Aumento de 263% nas submissões de CVE entre 2020 e 2025; adoção de modelo de priorização baseado em risco. Disponível em: <https://www.nist.gov/news-events/news/2026/04/nist-updates-nvd-operations-address-record-cve-growth>.

NOGUEIRA, R.; CHO, K. Passage re-ranking with BERT. **arXiv preprint arXiv:1901.04085**, 2019.

NURUSHEVA, A. *et al.* Machine learning algorithms in siem systems for enhanced detection and management of security events. *In: Bulletin of L. N. Gumilyov Eurasian National University. Mathematics, Computer Science, Mechanics Series*. Astana, Kazakhstan: L. N. Gumilyov Eurasian National University, 2024. v. 148, n. 3, p. 6–17.

PALO ALTO NETWORKS. **What Is the Role of AI and ML in Modern SIEM Solutions?** 2025. Disponível em: <https://www.paloaltonetworks.com/cyberpedia/role-of-artificial-intelligence-ai-and-machine-learning-ml-in-siem>. Acesso em: 20 jun. 2025.

PAPINENI, K. *et al.* BLEU: a method for automatic evaluation of machine translation. *In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*. [S.l.: s.n.], 2002. p. 311–318.

PICUS SECURITY. **What Is a YARA Rule?** 2025. Disponível em: <https://www.picussecurity.com/resource/glossary/what-is-a-yara-rule>. Acesso em: 20 jun. 2025.

Picus Security. **The Top Ten MITRE ATT&CK Techniques**. 2026. Disponível em: <https://www.picussecurity.com/resource/the-top-ten-mitre-attack-techniques>.

PODZINS, O.; ROMANOV, A. Why siem is irreplaceable in a secure it environment? *In: 2019 Open Conference of Electrical, Electronic and Information Sciences (eStream)*. [S.l.: s.n.], 2019. p. 1–5.

RAJAPAKSHA, S.; RANI, R.; KARAFILI, E. A RAG-based question-answering solution for cyber-attack investigation and attribution. *In: Computer Security – ESORICS 2024*. [S.l.: Springer, 2024. (Lecture Notes in Computer Science), p. 238–256.

Rapid7 Labs. Relatório técnico anual, **The 2026 Global Threat Landscape Report: Decoding the Accelerated Cyber Attack Cycle**. 2026. Mediana entre divulgação de vulnerabilidade

e inclusão no catálogo CISA KEV reduzida de 8,5 para 5,0 dias; exploração confirmada de vulnerabilidades CVSS 7-10 cresceu 105% (de 71 em 2024 para 146 em 2025). Disponível em: <https://www.rapid7.com/research/report/global-threat-landscape-report-2026/>.

REIMERS, N.; GUREVYCH, I. Sentence-BERT: Sentence embeddings using siamese BERT-networks. *In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019. p. 3982–3992.

ROTH, F. **Rule Creation Guide**. 2022. Disponível em: <https://github.com/SigmaHQ/sigma/wiki/Rule-Creation-Guide>. Acesso em: 20 jun. 2025.

ROY, S. *et al.* **SoK: The MITRE ATT&CK Framework in Research and Practice**. 2023. arXiv:2304.07411 [cs.CR]. Disponível em: <https://arxiv.org/abs/2304.07411>.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Hoboken, NJ: Pearson, 2020. ISBN 9780134610993.

SADDI, V. R. *et al.* Examine the role of generative ai in enhancing threat intelligence and cyber security measures. *In: Proceedings of the 2024 2nd International Conference on Disruptive Technologies (ICDT)*. India: IEEE, 2024. p. 537–542.

SANS Institute. White paper, **SANS 2025 Detection and Response Survey**. 2025. Survey anual independente e vendor-neutral (3ª edição), conduzido por Josh Lemon. 73% das organizações apontam falsos positivos como o principal desafio de detecção; 76% citam fadiga de alertas como preocupação central do SOC. Disponível em: <https://www.sans.org/white-papers/2025-detection-response-survey-webcast-forum>.

SCHWARTZ, Y. *et al.* LLMCloudHunter: Harnessing LLMs for automated extraction of detection rules from cloud-based CTI. *In: Proceedings of the ACM Web Conference 2025 (WWW '25)*. Sydney, NSW, Australia: ACM, 2025. p. 1922–1941. ISBN 979-8-4007-1274-6/25/04.

SCHWARTZ, Y. *et al.* **LLMCloudHunter: Harnessing LLMs for Automated Extraction of Detection Rules from Cloud-Based CTI**. 2024.

SHAH, A. H. *et al.* Automated log analysis and anomaly detection using machine learning. *In: Fuzzy Systems and Data Mining VIII*. Amsterdam, Netherlands: IOS Press, 2022, (Frontiers in Artificial Intelligence and Applications, v. 358). p. 137–147.

SHEERAZ, M. *et al.* Revolutionizing SIEM security: An innovative correlation engine design for multi-layered attack detection. **Sensors**, v. 24, n. 15, p. 4901, 2024.

SHEPARDSON, D. **SMarriott CEO apologizes for data breach, unsure if China responsible**. 2019. Disponível em: <https://www.reuters.com/article/us-usa-cyber-congress/marriott-ceo-apologizes-for-data-breach-unsure-if-china-responsible-idUSKCN1QO217/>. Acesso em: 20 jun. 2025.

SIGMAHQ. **About Sigma**. 2025. Disponível em: <https://sigmahq.io/docs/guide/about.html>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma - Generic Signature Format for SIEM Systems**. 2025. Disponível em: <https://github.com/SigmaHQ/sigma/tree/master>. Acesso em: 20 jun. 2025.

SIGMAHQ. **Sigma Rules**. 2025. Disponível em: <https://sigmahq.io/docs/basics/rules.html>. Acesso em: 20 jun. 2025.

STROM, B. E. *et al.* **MITRE ATT&CK: Design and Philosophy**. [S.l.], 2018. Revised March 2020.

TARIQ, S. *et al.* Alert fatigue in security operations centres: Research challenges and opportunities. **ACM Computing Surveys**, v. 57, n. 9, p. 1–38, 2025.

TENDIKOV, N. *et al.* Security information event management data acquisition and analysis methods with machine learning principles. **Results in Engineering**, v. 22, p. 102254, 2024.

The MITRE Corporation. **CVE Program Celebrates 25 Years of Impact in Cybersecurity**. 2024. Disponível em: <https://www.mitre.org/news-insights/news-release/cve-program-celebrates-25-years-impact-cybersecurity>.

The MITRE Corporation. **About CWE – Common Weakness Enumeration Overview**. 2025. Disponível em: <https://cwe.mitre.org/about/index.html>.

The MITRE Corporation. **CVE List Basics: Identifier Syntax and Numbering Authorities**. 2025. Disponível em: <https://www.cve.org/ResourcesSupport/AllResources/CNARules>.

VASWANI, A. *et al.* Attention is all you need. **Advances in Neural Information Processing Systems**, v. 30, 2017.

VIELBERTH, M. *et al.* Security operations center: A systematic study and open challenges. **IEEE Access**, v. 8, p. 227756–227779, 2020.

WANG, H. *et al.* RulePilot: An LLM-powered agent for security rule generation. *In: Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*. Rio de Janeiro, Brazil: ACM, 2026.

WANG, L. *et al.* A survey on large language model based autonomous agents. **Frontiers of Computer Science**, Springer, v. 18, n. 6, p. 186345, 2024.

Wazuh, Inc. **Wazuh - The Open Source Security Platform**. 2024. <https://wazuh.com/>. Copyright © 2015-2024 Wazuh, Inc.; Accessed: 26 June 2025.

WEI, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models. *In: Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2022. v. 35, p. 24824–24837.

XIAO, S. *et al.* C-Pack: Packaged resources to advance general Chinese embedding. **arXiv preprint arXiv:2309.07597**, 2023.

YAO, S. *et al.* ReAct: Synergizing reasoning and acting in language models. *In: International Conference on Learning Representations (ICLR)*. [s.n.], 2023. Disponível em: https://openreview.net/forum?id=WE_vluYUL-X.

ZHENG, L. *et al.* Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *In: Advances in Neural Information Processing Systems*. [S.l.: s.n.], 2023. v. 36.

APÊNDICE A – Exemplos de *prompts* usados

A seguir, a reprodução de exemplos de *prompt1*, *prompt2* e *prompt3* utilizados pelo agente deste trabalho para gerar regras *Sigma*.

A.1 Exemplos de *prompts* para `file_event_win_malware_darkgate_autoit3_save_temp.yml`

`prompt1.txt`

```
Generate ONE SIGMA rule (https://sigmahq.io/docs/basics/rules.html)  
for this event:  
- https://www.bleepingcomputer.com/news/security/hackers-exploit-  
windows-smartscreen-flaw-to-drop-darkgate-malware/  
- https://www.trendmicro.com/en\_us/research/24/c/cve-2024-21412-  
darkgate-operators-exploit-microsoft-windows-sma.html
```

Prompt2.txt

You are a Senior Threat Detection Engineer. Produce ONE valid SIGMA rule in YAML, following <https://sigmahq.io/docs/basics/rules.html>.

Threat to detect DarkGate malware loader being saved and executed from the C:

temp directory - attackers exploit Windows SmartScreen (CVE-2024-21412) to drop and run DarkGate via AutoIt3 scripts or interpreter from a temporary path.

Target environment - Product: windows; log source category: file_event

- Detection uses TWO parallel selections combined with OR (1 of selection_*):

(a) the target file path is within the temp directory AND ends with an AutoIt3-related extension or binary name;

(b) the executing image path is within the temp directory AND ends with the same AutoIt3-related indicators.

- Each selection combines |contains (temp directory path) with |endswith (AutoIt3 file types).

References

- <https://www.bleepingcomputer.com/news/security/hackers-exploit-windows-smartscreen-flaw-to-drop-darkgate-malware/>

- https://www.trendmicro.com/en_us/research/24/c/cve-2024-21412-darkgate-operators-exploit-microsoft-windows-sma.html

Output Return ONLY the YAML. No markdown fences, no explanations, no extra text.

Prompt3.txt

Generate a SIGMA rule to detect DarkGate malware loader dropped and executed from C:

temp via AutoIt3 - attackers exploit CVE-2024-21412 (Windows SmartScreen bypass) to stage and run DarkGate using AutoIt3 scripts or interpreter from a temporary path.

Target product: windows; log source category: file_event.

Detection uses TWO parallel selections with OR (1 of selection_*): one matching the target filename, another matching the executing image - both requiring the temp directory path (|contains) combined with AutoIt3-related file types (|endswith).

References to include:

- <https://www.bleepingcomputer.com/news/security/hackers-exploit-windows-smartscreen-flaw-to-drop-darkgate-malware/>
- https://www.trendmicro.com/en_us/research/24/c/cve-2024-21412-darkgate-operators-exploit-microsoft-windows-sma.html

A.2 Exemplos de *prompts* para web_sql_injection_in_access_logs.yml

prompt1.txt

Generate ONE SIGMA rule (<https://sigmahq.io/docs/basics/rules.html>) for this event:

- <https://www.acunetix.com/blog/articles/exploiting-sql-injection-example/>
- <https://www.acunetix.com/blog/articles/using-logs-to-investigate-a-web-application-attack/>
- <https://brightsec.com/blog/sql-injection-payloads/>
- <https://github.com/payloadbox/sql-injection-payload-list>
- <https://book.hacktricks.xyz/pentesting-web/sql-injection/mysql-injection>

prompt2.txt

You are a Senior Threat Detection Engineer. Produce ONE valid SIGMA rule in YAML, following <https://sigmahq.io/docs/basics/rules.html>.

Threat to detect

SQL injection attempts in web server access logs - GET requests whose URIs contain SQL injection payloads including schema enumeration, UNION-based extraction, blind injection, and URL-encoded variants of common SQLi strings.

Target environment

- Log source category: webserver
- Detection combines THREE conditions with AND:
 - (a) HTTP method is GET;
 - (b) the URI contains any of a comprehensive list of SQL injection payload strings - include both plain-text and URL-encoded variants, covering UNION SELECT, information_schema enumeration, version/database fingerprinting, and boolean-based payloads;
 - (c) exclude 404 responses to reduce noise from scanners hitting non-existent endpoints. Use 'selection and keywords and not 1 of filter_main_*'.

References

- <https://www.acunetix.com/blog/articles/exploiting-sql-injection-example/>
- <https://www.acunetix.com/blog/articles/using-logs-to-investigate-a-web-application-attack/>
- <https://brightsec.com/blog/sql-injection-payloads/>
- <https://github.com/payloadbox/sql-injection-payload-list>
- <https://book.hacktricks.xyz/pentesting-web/sql-injection/mysql-injection>

Output Return ONLY the YAML. No markdown fences, no explanations, no extra text.

prompt3.txt

Generate a SIGMA rule to detect SQL injection attempts in web server access logs - GET requests whose URIs contain SQLi payloads including schema enumeration, UNION-based extraction, blind injection, and URL-encoded variants.

Log source category: webserver.

Detection combines: (a) HTTP method GET; (b) a comprehensive keyword list covering UNION SELECT, information_schema, version/database fingerprinting, and boolean payloads in both plain-text and URL-encoded form; (c) exclusion of 404 responses to suppress scanner noise. Use `selection and keywords and not 1 of filter\_main\_\*`.

References to include:

- <https://www.acunetix.com/blog/articles/exploiting-sql-injection-example/>
- <https://www.acunetix.com/blog/articles/using-logs-to-investigate-a-web-application-attack/>
- <https://brightsec.com/blog/sql-injection-payloads/>
- <https://github.com/payloadbox/sql-injection-payload-list>
- <https://book.hacktricks.xyz/pentesting-web/sql-injection/mysql-injection>

**APÊNDICE B – Melhores regras geradas pelas LLM comerciais
selecionadas no filtro por *LLM-as-a-judge***

Tabela 19 – Lista completa das regras filtradas e os modelos selecionados por *prompt*

Regra Sigma	Prompt 1	Prompt 2
bitbucket_audit_unauthorized_full_data_- export_triggered	Deepseek	Claude
azure_app_device_code_authentication	Claude	Claude
zeek_susp_kerberos_rc4	Claude	Claude
posh_ps_modify_group_policy_settings	Claude	Claude
win_firewall_as_add_rule_wmiprvse	ChatGPT	Claude
huawei_bgp_auth_failed	Claude	Claude
rpc_firewall_printing_lateral_movement	Claude	Claude
registry_set_change_security_zones	Claude	Claude
app_sqlinjection_errors	ChatGPT	Claude
db_anomalous_query	Claude	Claude
file_delete_win_delete_exchange_- powershell_logs	ChatGPT	Claude
net_dns_katz_stealer_domain	Claude	Claude
opencanary_ssh_login_attempt	Claude	Claude
proxy_exploit_cve_2023_1389_unauth_- command_injection_tplink_archer_ax21	Deepseek	Deepseek
registry_event_runkey_winekey	Claude	Claude
appframework_django_exceptions	Claude	Claude
github_delete_action_invoked	Claude	Deepseek
java_local_file_read	Claude	Claude
kubernetes_audit_exec_into_container	Claude	Claude
kubernetes_audit_secrets_modified_or_- deleted	Claude	Deepseek
av_exploit_cve_2021_34527_print_nightmare	Deepseek	Claude
microsoft365_new_federated_domain_added_- audit	Deepseek	Claude
win_appxdeployment_server_appx_- downloaded_from_file_sharing_domains	Gemini	Claude
file_event_lnx_doas_conf_creation	Gemini	Deepseek
posh_pm_invoke_obfuscation_stdin	Claude	Claude
win_security_exploit_cve_2023_36884_- office_windows_html_rce_share_access_- pattern	Gemini	Claude
appframework_ruby_on_rails_exceptions	Gemini	Claude

Continua na próxima página

Tabela 19 – Continuação da página anterior

Regra Sigma	Prompt 1	Prompt 2
pipe_created_sysinternals_psexec_ default_pipe_susp_location	ChatGPT	Claude
file_event_win_susp_startup_folder_ persistence	Gemini	Claude
proc_creation_macos_remote_access_tools_ meshagent_arguments	Claude	Claude
create_remote_thread_win_powershell_ generic.yml	Copilot	Claude
image_load_susp_clickonce_unsigned_ module_loaded	Copilot	Deepseek
web_sql_injection_in_access_logs	Deepseek	Claude
proc_creation_lnx_mal_shai_hulud_ indicator	Claude	Claude
zeek_http_executable_download_from_webdav	Deepseek	Deepseek
file_event_win_malware_darkgate_autoit3_ save_temp	NONE	Claude
nodejs_rce_exploitation_attempt	Deepseek	Claude
proc_creation_win_reg_query_registry	Deepseek	Deepseek
app_python_sql_exceptions	Deepseek	Deepseek
file_event_macos_python_path_ configuration_files	Claude	Claude
spring_application_exceptions	Deepseek	Claude
okta_policy_rule_modified_or_deleted	Claude	Deepseek
cisco_duo_mfa_bypass_via_bypass_code	Claude	Claude
file_rename_win_non_dll_to_dll_ext	Claude	Deepseek
net_connection_win_silenttrinity_stager_ msbuild_activity	Claude	Claude
posh_pm_alternate_powershell_hosts	Claude	Claude
proc_access_win_lsass_susp_access_flag	Claude	Claude
velocity_ssti_injection	Claude	Claude
zeek_dce_rpc_mitre_bzar_persistence	Claude	Claude
net_dns_susp_telegram_api	Deepseek	Deepseek